



Hybrid Optimization Approaches for the Electric Autonomous Dial-a-Ride Problem

DIPLOMARBEIT

zur Erlangung des akademischen Grades

Diplom-Ingenieur

im Rahmen des Studiums

Software Engineering

eingereicht von

Theo Schneider

Matrikelnummer 12402263

an der Fakultät für Informatik

der Technischen Universität Wien

Betreuung: Ao.Univ.Prof. Dipl.-Ing. Dr.techn. Günther Raidl

Mitwirkung: Univ.Ass. Dipl.-Ing. Laurenz Tomandl, BSc

Projektass.in Dipl.-Ing.in Maria Bresich, BSc

Wien, 12. Oktober 2026

Theo Schneider

Günther Raidl



Hybrid Optimization Approaches for the Electric Autonomous Dial-a-Ride Problem

DIPLOMA THESIS

submitted in partial fulfillment of the requirements for the degree of

Diplom-Ingenieur

in

Software Engineering

by

Theo Schneider

Registration Number 12402263

to the Faculty of Informatics

at the TU Wien

Advisor: Ao.Univ.Prof. Dipl.-Ing. Dr.techn. Günther Raidl

Assistance: Univ.Ass. Dipl.-Ing. Laurenz Tomandl, BSc

Projekttass.in Dipl.-Ing.in Maria Bresich, BSc

Vienna, October 12, 2026

Theo Schneider

Günther Raidl

Erklärung zur Verfassung der Arbeit

Theo Schneider

Hiermit erkläre ich, dass ich diese Arbeit selbständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit – einschließlich Tabellen, Karten und Abbildungen –, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind, auf jeden Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

Ich erkläre weiters, dass ich mich generativer KI-Tools lediglich als Hilfsmittel bedient habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Im Anhang „Übersicht verwendeter Hilfsmittel“ habe ich alle generativen KI-Tools gelistet, die verwendet wurden, und angegeben, wo und wie sie verwendet wurden. Für Textpassagen, die ohne substantielle Änderungen übernommen wurden, habe ich jeweils die von mir formulierten Eingaben (Prompts) und die verwendete IT-Anwendung mit ihrem Produktnamen und Versionsnummer/Datum angegeben.

Wien, 12. Oktober 2026

Theo Schneider

Danksagung

Ich bedanke mich ganz herzlich bei meinem Betreuer, Günther Raidl, sowie meinen Mitbetreuern Laurenz Tomandl und Maria Bresich für ihre Unterstützung während der gesamten Forschung für diese Diplomarbeit.

Außerdem möchte ich meiner Familie für ihre ständige Unterstützung und ihr Verständnis danken, wodurch mir die bestmöglichen Bedingungen für mein Studium geboten wurden.

Acknowledgements

I deeply thank my advisor, Günther Raidl, and my co-advisors Laurenz Tomandl and Maria Bresich for their help throughout this thesis.

I also want to thank my family for their continued support and understanding, providing the best possible conditions for my studies.

Kurzfassung

Das Electric Autonomous Dial-a-Ride Problem (EADARP) untersucht die Optimierung der Routen- und Zeitplanung einer Flotte elektrischer, selbstfahrender Fahrzeuge, dessen Ziel es ist, Kunden von einer Abhol- zu einer Abgabestelle zu befördern. Das Ergebnis soll alle Kundenanforderungen erfüllen, die Fahrkosten minimieren und betriebliche Einschränkungen berücksichtigen.

Im Vergleich zu bereits existierenden Methoden, die Nachbarschaften der aktuellen Lösung erkunden, um bessere Lösungen zu finden, funktioniert die Construct, Merge, Solve & Adapt (CMSA) Metaheuristik anders. Sie generiert und verfeinert einen diversen Satz an Komponenten, mit denen man ein ganzzahliges lineares Programm (ILP) formulieren kann, das die beste mögliche Lösung aus den bekannten Komponenten extrahiert.

Diese Arbeit untersucht einen CMSA-basierten Ansatz für das EADARP, einschließlich der Entwicklung hybrider Methoden, die CMSA mit anderen Ansätzen kombiniert, um potentiell bessere Ergebnisse zu liefern. Dafür werden drei unterschiedliche CMSA-basierte Algorithmen entwickelt: *(i)* die direkte Anwendung der CMSA Metaheuristik für das EADARP, *(ii)* einen hybriden CMSA-Column Generation Ansatz, der die exakte Technik Column Generation benutzt, um dem CMSA Pool vielversprechende Komponente hinzuzufügen, und *(iii)* einen hybriden CMSA-LNS Algorithmus, der die schnellen Lösungsentdeckungsmechanismen der LNS mit dem exakten ILP von CMSA kombiniert, um hochwertige Lösungen zu finden.

Um die entwickelten Ansätze zu evaluieren, wurden Experimente auf Benchmark-Instanzen der Fachliteratur durchgeführt. Der CMSA-CG Ansatz ist auf kleinen Instanzen vielversprechend, da Column Generation die Suche auf Lösungen lenken kann, die sonst unerreichbar wären. Leider erweist sich das Lösen des exakten Pricing Subproblems als rechenintensiv. Dadurch wird die Anwendung dieses Ansatzes auf größeren Instanzen in der Praxis nicht mehr sinnvoll möglich. Der CMSA Algorithmus liefert auf kleinen bis mittleren Instanzen gute Ergebnisse und findet durchgehend hochwertige Lösungen auf allen Instanzen. Für größere Instanzen stößt CMSA jedoch aufgrund der verwendeten Konstruktionsheuristik auf Skalierbarkeitsprobleme. Der CMSA-LNS Ansatz kann dank der LNS Hybridisierung diese Hürde überwinden, übertrifft die anderen Ansätze bei mittleren Instanzen durchgehend und bleibt auch auf größeren Instanzen kompetitiv.

Abstract

The Electric Autonomous Dial-a-Ride Problem (EADARP) consists of optimizing the routing and scheduling of a fleet of electric self-driving vehicles to transport a set of customers between a pickup and a dropoff location with the objective of serving all given requests while minimizing driving costs and respecting operational constraints.

Compared to existing methods to solve this problem, which work by exploring neighborhoods of the current incumbent, the Construct, Merge, Solve & Adapt (CMSA) metaheuristic relies on generating and refining a diverse pool of components, which is used to formulate an Integer Linear Program (ILP) to extract the best possible solution based on the known components.

This thesis investigates a CMSA approach for the EADARP, including the development of hybrid methods combining CMSA with other techniques to deliver improved solutions. More precisely, this thesis proposes three different algorithms based on CMSA: *(i)* a direct application of CMSA to the EADARP, *(ii)* a hybrid CMSA-Column Generation algorithm that uses exact Column Generation to find promising components that are added to the CMSA pool, and *(iii)* a hybrid CMSA-LNS approach that combines the fast exploration of different solutions from the LNS with the exact ILP solving of CMSA to find high-quality solutions.

Experimental results obtained on benchmark instances from the literature are used to measure the performance of each approach. The CMSA-CG approach shows promising results on small instances, with Column Generation guiding the search towards otherwise unreachable solutions. Unfortunately, solving the exact Pricing Subproblem problem proves computationally expensive, hindering the application of this method to larger instances. On its own, CMSA already performs quite well up to medium-sized instances, consistently finding high-quality solutions on all instances. As the size of instances grows, CMSA encounters scalability issues due to the chosen construction heuristic. The CMSA-LNS algorithm can overcome this limitation, benefiting from the LNS hybridization, and consistently outperforms the other approaches on medium instances, remaining competitive even on large instances.

Contents

Kurzfassung	xi
Abstract	xiii
Contents	xv
1 Introduction	1
1.1 Research Questions	3
1.2 Structure of the Thesis	3
2 Related Work	5
3 The Electric Autonomous Dial-a-Ride Problem	9
3.1 Problem Definition	9
3.2 Set Covering Formulation for the EADARP	15
3.3 Working with Dynamic Instances	20
4 The Construct, Merge, Solve & Adapt heuristic	23
4.1 Introduction to CMSA	23
4.2 Adapting CMSA to the EADARP	24
4.3 Handling Large Instances	29
5 Hybridizing CMSA with Column Generation	33
5.1 Introduction to Column Generation	33
5.2 Applying Column Generation to the EADARP	34
5.3 Labeling Algorithm to Solve the Pricing Subproblem	38
5.4 Integrating Column Generation into CMSA	51
6 Hybridizing CMSA with an LNS-based approach	55
6.1 An LNS Approach for the EADARP	55
6.2 Combining CMSA and LNS	56
6.3 Enhancing LNS for Large Instances	59
7 Experimental Analysis	61
	xv

7.1	Test Environment	61
7.2	Parameter Tuning	62
7.3	Description of the Experiments	66
7.4	Experimental Results	67
8	Conclusion & Summary	89
	Overview of Generative AI Tools Used	93
	Übersicht verwendeter Hilfsmittel	95
	List of Figures	97
	List of Tables	99
	List of Algorithms	101
	Acronyms	103
	Bibliography	105

Introduction

The ever-growing need for personal mobility leads to an increased interest in efficient transportation, especially within urban areas. Besides owning a car, other transportation options include public transport, as well as taxi and ride-sharing services. The option of using a shared vehicle to transport multiple customers between their pickup and dropoff locations represents a promising middle ground between the rigidity of public transport and the high financial and environmental costs of individually owned vehicles. In this context, optimizing the routing and scheduling of the vehicles becomes crucial to achieve economies of scale in operational costs and ensure customer satisfaction.

This problem has been formalized by Cordeau and Laporte [15], who proposed the first formulation of the Dial-a-Ride Problem (DARP). The DARP consists of optimizing the routing and scheduling of a fleet of vehicles to transport a set of customers between a pickup and a dropoff location with the objective of serving all given requests while minimizing driving costs and respecting operational constraints.

With the increasing interest in electric mobility and the advances in the domain of self-driving vehicles, the DARP has been adapted to consider a fleet of electric and autonomous vehicles, leading to the formulation of the Electric Autonomous Dial-a-Ride Problem (EADARP) [10]. These changes impose additional constraints based on each vehicle's state of charge. Solutions need to insert some charging stops in the planned routes to recharge the battery of the vehicles. The EADARP is a well-studied problem, with multiple approaches having already been developed to find solutions efficiently [10, 12, 27, 40, 41, 42].

The Dynamic Electric Autonomous Dial-a-Ride Problem (DynEADARP), in which not all requests are known beforehand but become known over time during the planning horizon, has been proposed as a further variant of the EADARP to more accurately describe a real-world system [22, 45]. Taking only the known requests into account, the algorithms solving the DynEADARP have to find a compromise between serving all

current requests most efficiently and remaining flexible enough to accommodate new incoming requests.

Most methods to solve the EADARP and the DynEADARP rely either on exact methods that find the optimal solution at high costs or on heuristic algorithms that perform a guided search of the solution space to find a good solution more efficiently. In practice, exact methods do not scale well with the size of real-world instances, making these approaches intractable. The majority of heuristic methods that have been applied to the EADARP work by generating feasible solutions and evaluating them to guide the search towards promising solutions.

In contrast to search-based methods, the Construct, Merge, Solve & Adapt (CMSA) heuristic developed by Blum et al. [7] considers parts of a solution, so-called components, as the main object of interest. This different approach relies on generating and refining a diverse pool of components, which can be used to find the combination yielding the best possible solution. CMSA has already been successfully applied to other combinatorial optimization problems, such as the Minimum Common String Partition Problem and the Minimum Covering Arborescence Problem [7], as well as other Vehicle Routing Problems [2].

Aside from its use as a standalone heuristic framework, CMSA also presents many opportunities for hybridization with other existing approaches. In their work on the Variable Sized Bin Packing Problem, Akbay et al. [3] have looked into the combination of CMSA with Column Generation, an exact technique which can propose new solution components with high potential to guide the heuristic search. There is also potential in combining CMSA with other heuristic techniques, taking advantage of search-based techniques such as Large Neighborhood Search (LNS) to deliver feasible solutions that the CMSA approach will then refine.

This thesis aims to investigate a CMSA approach for the EADARP. Aside from applying this technique directly, this thesis will also investigate hybrid methods combining CMSA with other methods, such as Column Generation and Large Neighborhood Search, to deliver improved solutions for the EADARP.

This thesis will build on the formulation of the DynEADARP developed by Tomandl et al. [45], with a focus on the specific handling of lateness and time windows. In the scope of this thesis, we will investigate static instances of this formulation, where all requests are known in advance. As such, we will look into a variant of the EADARP which is based on the DynEADARP formulation of Tomandl et al. [45]. The technique developed by Tomandl et al. [45] to solve dynamic instances uses a rolling horizon approach that requires to continuously solve the static problem formulation. As such, the algorithms developed in this thesis for static instances remain applicable for dynamic instances, although the core of the refinements developed in this thesis will be targeted towards static instances.

Results obtained by conducting computational experiments on test instances from the literature will be used to evaluate the performance of the developed approaches.

1.1 Research Questions

The goals of this thesis are summarized in the following Research Questions (RQ):

RQ1: What adaptations can be made to the general CMSA framework to deliver improved solutions for the EADARP?

RQ2: How can CMSA make use of exact Column Generation to develop a guided search strategy to solve the EADARP?

RQ3: Which algorithms can be used to efficiently solve the Pricing Subproblem obtained when applying Column Generation to the EADARP?

RQ4: How can CMSA be combined with the LNS developed by Tomandl et al. [45] to improve its performance?

1.2 Structure of the Thesis

Chapter 2 will close off this introduction by looking into related work, investigating existing literature to solve optimization problems in the class of Dial-a-Ride Problems. This chapter will also review applications of methods used in this thesis, such as Construct, Merge, Solve & Adapt and Column Generation, to other problems.

Chapter 3 presents the EADARP, the problem that this thesis will investigate, clearly defining the problem formulation that will be used as the foundation of the developed algorithms. This chapter will also introduce an alternative mathematical formulation of the EADARP which will prove useful in later chapters.

The next chapters will present each of the three approaches developed in this thesis. Chapter 4 will start by introducing the Construct, Merge, Solve & Adapt (CMSA) metaheuristic and developing a standalone CMSA algorithm for the EADARP. Then, Chapter 5 and Chapter 6 will present the two hybridization strategies that have been investigated in this thesis to improve the performance of CMSA by combining it with other approaches.

Afterwards, Chapter 7 will present the experimental results obtained in this thesis, comparing the approaches developed previously with one another and using the LNS approach from Tomandl et al. [45] as a baseline for comparison. This chapter will also present the parameter tuning that was executed to find fitting parameter values for each algorithm.

Finally, Chapter 8 will conclude this report, summarizing and discussing the main results that were achieved in this thesis.

Related Work

This chapter reviews the related work. First, we will give an overview of existing techniques to tackle optimization problems in the area of Vehicle Routing Problems and Dial-a-Ride Problems, before looking into existing literature on Construct, Merge, Solve & Adapt and Column Generation, which will be used throughout this thesis.

The DynEADARP was introduced as a variant of the static EADARP to account for the dynamic arrival of requests. The EADARP itself is a variant of the Dial-a-Ride problem (DARP), introduced by Cordeau and Laporte [15]. These problems all belong to the class of Vehicle Routing Problems (VRPs), combinatorial optimization problems trying to optimize the routing of vehicles under operational constraints. This section will investigate existing literature related to the DARP, EADARP, and DynEADARP.

The DARP aims to optimize the routing and scheduling of vehicles to transport customers from a pickup to a dropoff location. Many different approaches have been developed to solve multiple variants of this problem, starting from the tabu search originally introduced by Cordeau and Laporte [15]. Exact methods to solve the DARP rely on Integer Linear Program formulations, using either Branch-and-Cut [14, 35, 38] or Branch-and-Price [18, 31, 37] to compute optimal solutions. In particular, Rist and Forbes [35] introduced a new formulation for DARP-like problems relying on the concept of fragments, which will also be used in this thesis. Heuristic approaches have also been investigated to obtain good quality solutions with a lower runtime. These approaches include metaheuristics based on local search, such as Variable Neighborhood Search [32] or Adaptive Large Neighborhood Search [19]. Other techniques using Dynamic Programming [36] or Genetic Algorithms [25] have been investigated. Problem formulations with multiple concurrent objective functions have also been considered [30, 33]. Cordeau and Laporte [13], Molenbruch et al. [29] and Ho et al. [21] provide comprehensive reviews on existing literature regarding the DARP.

The EADARP was first introduced as a variant of the classical DARP by Bongiovanni et al. [10]. They formulated this problem as a Mixed Integer Linear Program (MILP), which

can be solved exactly using Branch-and-Cut with solvers such as Gurobi or CPLEX. The drawback of this approach is that the runtime of such solvers scales exponentially with the instance size, making it intractable for larger instances. Continuing with exact methods, Su et al. [41] presented a Branch-and-Price framework that utilizes an exponentially sized formulation of the EADARP along with exact column generation to more efficiently find optimal solutions. One of the main advancements of this approach was to adapt the labeling algorithms developed by Desaulniers et al. [17] for another VRP to fit with the EADARP. This approach also uses the formalism of battery-restricted fragments to efficiently solve the Pricing Subproblem to optimality. Finally, Stallhofer and Parragh [40] developed a MILP approach relying on an event-based graph representation of the EADARP, obtaining competitive results on benchmark instances.

Faced with the limitations of exact methods, heuristic approaches have also been designed to tackle the EADARP. Su et al. [42] proposed a heuristic based on deterministic annealing that uses an efficient method to evaluate routes in linear time to iteratively improve the found solutions. Using a different approach, Limmer [27] developed a bi-level Large Neighborhood Search (LNS) which separates the scheduling of charging stops from the assignment of requests. The charging stops are only updated occasionally by the outer-level LNS, while the inner loop optimizes the distribution of pickups and dropoffs with fixed charging stops. This multi-level approach allows for better scalability. Bresich et al. [12] proposed another LNS-based approach that incorporates an on-the-fly algorithm to insert charging stops during route evaluation. This on-the-fly approach is handled separately from the LNS, making it suitable for multiple metaheuristics. This approach was further investigated by Bresich et al. [11] with a focus on heuristic and machine learning driven pruning techniques to sparsen the instance graph to better handle large instances. Finally, Jacobsen et al. [24] investigated a deep reinforcement learning approach to learn route construction policies independently from heuristics developed by domain experts.

Aside from static formulations where all requests are known in advance, dynamic variants of VRPs have also emerged, in which requests become available over time. These dynamic problems introduce new challenges due to the fact that a solution may need to be updated whenever a new request becomes available. Solutions at a given time have to be flexible enough to accommodate new requests as efficiently as possible. In particular, some sub-optimal decisions at the current time step may become profitable in the future, due to new incoming requests. Multiple solution approaches have been investigated to solve the dynamic DARP. Based on the original tabu search for the DARP [15], Attanasio et al. [5] have proposed parallel algorithms utilizing tabu search to maximize the proportion of satisfied requests. Other approaches include two-phase heuristics handling new request insertion separately from an ongoing route optimization process by Coslovitch et al. [16] or Tafreshian et al. [43]. Multiple authors have also looked into approaches modeling dynamic problem formulations as Markov Decision Processes, such as Sayarshad and Gao [39] and Ackermann and Rieck [1] for the dynamic DARP, or Kullman et al. [26] for the dynamic Ride-Hailing Problem. Amiri et al. [4] investigated a Column Generation

based approach for the dynamic DARP. For a general overview of dynamic approaches for VRPs, we refer to the reviews developed by Berbeglia et al. [6] and Psaraftis et al. [34].

The first dynamic formulation of the EADARP has been introduced by Bongiovanni et al. [9], who proposed a two-phase heuristic. The outer phase assigns newly incoming requests to vehicles and inserts them into the associated routes, while the inner heuristic relies on local search strategies in order to improve the solution. This heuristic was enhanced further with the use of a trained Machine Learning model to adequately choose improvement operators in the inner solution-improving phase [8]. Huang et al. [22] proposed a complex genetic programming approach which dynamically evolves heuristics used to construct valid solutions to the dynamic EADARP. Using an LNS-based approach, Tomandl et al. [45] further extended it by training a Reinforcement Learning model to control the request start times as well as guide their charging behaviour via a controlled objective penalty.

Compared to existing approaches for EADARP variants, this thesis aims to investigate heuristic approaches based on Construct, Merge, Solve & Adapt for the EADARP. Construct, Merge, Solve & Adapt (CMSA) was introduced by Blum et al. [7]. It is a metaheuristic framework that works by continuously refining a pool of solution components until a satisfactory solution is reached. Components are usually some partial solutions that can be combined with one another to form a solution to the original problem. The Construct step uses a randomized construction operator to generate a diverse set of feasible solutions. Then, the Merge step iterates over all previously generated solutions to extract the components that a solution is composed of and add them to the pool. The Solve step aims at improving the incumbent solution by finding the combination of components currently in the pool that yields the best feasible solution. Finally, the Adapt step handles the management of the pool, discarding components that have remained unused for a long time.

CMSA has been successfully applied to solve combinatorial optimization problems from different domain areas, such as the Minimum Common String Partition Problem [7], or graph problems including the Rooted Max Tree Coverage Problem [47] and the Maximum Happy Vertices Problem [44]. Further variants of CMSA include the use of self-adaptive parameters to dynamically control the Construct step for the Electric Vehicle Routing Problem with Simultaneous Pickup and Deliveries [2].

One of the approaches developed in this thesis relies on Column Generation to improve the performance of CMSA by adding promising components to the pool. This approach has been successfully applied by Akbay et al. [3] for the Variable Sized Bin Packing Problem. Column Generation is an iterative procedure used in mathematical programming to solve certain large Linear Programs [46]. This procedure starts from a small initial pool of decision variables which will be refined by adding new variables with good potential of improving the solution. Variables that improve the solution are found by solving the so-called Pricing Subproblem. It is obtained by formulating the problem of finding a new variable to add to the pool as a separate optimization problem.

There are many approaches in the literature that use Column Generation to solve VRPs. Exact methods relying on an exponentially sized formulation of the original MILP use Column Generation in a Branch-and-Price framework. Branch-and-Price has been used to solve many problem formulations in the area of DARPs. From the Branch-Cut-and-Price framework developed by Ropke and Cordeau [37] for the Pickup and Delivery Problem with Time Windows to multiple variants of the DARP considering additional driver-related constraints [31] or an objective function based on the occupancy rate of vehicles [18], many approaches have developed Column Generation techniques for VRPs. Su et al. [41] have applied this concept to the static EADARP, while Amiri et al. [4] investigated acceleration techniques for Column Generation applied to the dynamic DARP.

A common topic among many of these approaches is the use of a labeling algorithm to solve the Pricing Subproblem within the Column Generation framework. Similar labeling algorithms are also used in heuristic approaches. Desaulniers et al. [17] proposed exact labeling algorithms for electric VRPs to efficiently generate feasible routes for the vehicles. They developed multiple formulations taking into account constraints on the allowed number of recharging stops per route, and the possibility for vehicles to execute partial recharges. Su et al. [42] built upon these algorithms to develop a route evaluation scheme to efficiently minimize the excess user ride time for the EADARP. Bresich et al. [12] modified this labeling scheme in their LNS approach for the EADARP to evaluate the solutions given by the repair operations of their LNS.

The Electric Autonomous Dial-a-Ride Problem

This chapter describes the EADARP, giving the background information needed to understand the main challenges of this optimization problem and the approaches used to solve it.

In the existing literature, different problem variants for the EADARP have been introduced. Section 3.1 introduces the formulation of the EADARP variant which will be considered in this thesis, explaining the constraints and objectives of this problem. The formulation is based on the one developed by Tomandl et al. [45]. Although the formulation used by Tomandl et al. [45] includes dynamic aspects, specifically regarding the dynamic arrival of requests, this thesis will focus on the static formulation, where all requests are known from the start.

Section 3.2 presents an alternative formulation of the EADARP, similar to the classical Set Covering Problem, which will be used in the following chapters.

Section 3.3 goes into more details on the dynamic aspect of the problem, explaining the approach that is used to solve dynamic instances by iteratively solving static sub-instances.

The notations presented in this chapter are used consistently within the rest of this thesis and are summarized in Table 3.1.

3.1 Problem Definition

The Electric Autonomous Dial-a-Ride Problem (EADARP) looks into organizing a fleet of vehicles to transport customers from a pickup to a dropoff location. It is set in the context of a mobility provider owning a fleet of electric autonomous vehicles and trying to

optimize the routes taken by the vehicles to reduce the operational costs and guarantee a certain level of quality of service.

In contrast to classical vehicle routing problems, the EADARP allows ride-sharing. This means that a vehicle may serve multiple requests simultaneously by picking up new customers even when there are already passengers on board of the vehicle, so long as the overall capacity of the vehicle is respected.

The EADARP considers a fleet of n_K vehicles, each located at a given initial position. The goal of this problem is to build a route for each vehicle, such that all requests are fulfilled. Each route starts from the initial position of the associated vehicle.

Each of the n customer requests is then assigned to one of the n_K vehicles, which will handle the request. A request consists of transporting a customer from their pickup location to a dropoff point. Each request also contains a time window during which the customer would prefer to be picked up. This time constraint has to be considered when adding the request to a route.

A vehicle may also drive to a charging station to recharge its battery. In this case, charging stops are inserted into the route of the vehicle. To avoid waiting time for the customers currently in the vehicle, a vehicle can only go to a charging station if it does not currently have any passengers on board.

When every request has been assigned to a vehicle, the final state of all constructed routes is fixed. A solution is thus defined as a set of n_K routes, each starting at the initial position of the associated vehicle. Each route is a sequence of locations at which the vehicle should stop, along with a time of arrival at which the vehicle enters the location. All stopping positions are either pickup locations, dropoff locations, or charging stations. To be considered feasible, a solution has to respect all constraints defined in the problem formulation. The exhaustive list of considered constraints is described in detail in Section 3.1.3. They make sure that the routes constructed for each vehicle are technically feasible and that a given quality of service is respected for the customers.

Finally, to evaluate how good a given solution is, the problem formulation also defines an objective function. This function computes a numerical cost that estimates how good a solution is based on predefined criteria. In the case of the EADARP, the objective function consists of the sum of the driving costs of all vehicles and a penalty factor to compensate for the fact that a vehicle is allowed to arrive at a pickup location after the time window has passed. This objective is described in more detail in Section 3.1.2.

3.1.1 Instance Definition

An instance of the EADARP can be formalized as a complete, directed graph $G = (V, A)$. The nodes V are split into pickup locations, dropoff locations, charging stations, and the initial position of each vehicle, while the set of directed edges $A = \{(u, v) \mid u, v \in V, u \neq v\}$ contains the shortest path between each pair of locations. Each edge $(u, v) \in A$ is associated with the weights $d_{u,v}$ and $b_{u,v}$, representing respectively the driving time and

the battery consumption to go directly from u to v . Both driving times and battery consumption respect the triangle inequality, meaning that making a detour to location v on the path between locations u and w cannot be more efficient than taking the direct path from u to w . Mathematically, this gives:

$$\begin{aligned} \forall u, v, w \in V, \quad d_{u,w} &\leq d_{u,v} + d_{v,w} \\ b_{u,w} &\leq b_{u,v} + b_{v,w} \end{aligned}$$

The set of n_K available vehicles is denoted as $K = \{1, \dots, n_K\}$. All vehicles are considered to be homogeneous and have the same maximum battery capacity Q and customer load capacity C . Each vehicle $k \in K$ starts from the initial position v_k^{ini} with the initial battery level $B_{k,0}$. For static instances, each vehicle $k \in K$ is available from the start at instant $t = 0$. For dynamic instances, which will be discussed in detail in Section 3.3, we use a rolling horizon where vehicles might already be on the road at the start of the optimization. To account for this period of unavailability, we introduce t_k^{avail} , the time at which vehicle k becomes available for planning. For static instances, we simply set $t_k^{\text{avail}} = 0$ for all $k \in K$.

Given the number n of requests in the instance, the set of customer requests is denoted as $R = \{1, \dots, n\}$. Each request $i \in R$ becomes available for planning at time t_i^{arr} . A request consists of a transportation demand from a pickup location $v_i^{\uparrow} \in V$ to a dropoff location $v_i^{\downarrow} \in V$. Each request also has a given customer load $l_i \geq 0$, a maximum user ride time d_i^{max} and a pickup time window $[t_i^{\text{start}}, t_i^{\text{end}}]$, which is valid at the location $v_i^{\uparrow}$. The set of all pickup locations is denoted as $P = \{v_i^{\uparrow} \mid i \in R\}$, while $D = \{v_i^{\downarrow} \mid i \in R\}$ is the set of dropoff locations.

All of the n_S charging stations have a homogeneous charging rate $\alpha > 0$. From the current battery level b of a vehicle, the recharging time $t^{\text{rch}}(b)$ needed to get back to full battery can be computed as follows:

$$t^{\text{rch}}(b) = \frac{Q - b}{\alpha} \quad (3.1)$$

With the notations defined here, the set of nodes V in the instance graph is defined as follows:

$$V = P \cup D \cup S \cup \{v_k^{\text{ini}} \mid k \in K\}$$

3.1.2 Objective of the EADARP

The objective function for this problem consists of two components. On the one hand, the total driving cost of all vehicles should be minimized. Simultaneously, vehicles are allowed to arrive late at pickup locations, but this lateness should be minimized to ensure a good quality of service. These two concurrent objectives are combined into a single objective function by considering a weighted sum of both factors.

Given a solution, its objective value can be computed as:

$$\sum_{k \in K} d_k^{\text{travel}} + \rho \cdot \sum_{i \in R} d_i^{\text{late}} \quad \text{with } d_i^{\text{late}} = \max\{0, t_i - t_i^{\text{end}}\} \quad (3.2)$$

where d_k^{travel} is the sum of all driving costs in the route assigned to vehicle k and d_i^{late} is the delay by which request i is picked up at location $v_i^\uparrow$. The parameter ρ is a fixed penalty factor that decides how costly late pickups are.

3.1.3 Constraints

The complexity of the EADARP comes from the operational constraints that have to be respected. They are composed of technical constraints regarding the state of charge of a vehicle and its customer load capacity, as well as time-related constraints on the handling of customers.

For a solution to be considered feasible, it needs to satisfy all of the following constraints.

- The battery level b of each vehicle at every location should always remain feasible. It should respect $0 \leq b \leq Q$.
- The customer load capacity of a vehicle should never be exceeded. The current load l of a vehicle should always respect $0 \leq l \leq C$.
- Each request $i \in R$ has to be served by exactly one vehicle. This also implies that the same vehicle must pick up the customer and drop them off and that the pickup has to happen before the dropoff.
- Each request $i \in R$ has an associated maximum user ride time d_i^{max} . This time is a limit on how long a customer is allowed to stay on board of a vehicle. Once a request has been picked up, it has to be dropped off within d_i^{max} units of time.
- A request $i \in R$ cannot be picked up before the start of its time window t_i^{start} , but it may be picked up after the end of the time window t_i^{end} at the cost of a lateness penalty that is considered in the objective function.
- Vehicles may only wait when there is no customer on board. Waiting does not consume any battery.
- Charging is only allowed when the vehicle is empty.
- Vehicles always charge to their full capacity. There is no partial recharge.

Context	Symbol	Meaning
	R	set of customer requests
	P	set of pickup locations
	D	set of dropoff locations
	K	set of vehicles
	S	set of charging stations
	T^{plan}	planning horizon
	ρ	penalty factor for lateness
	α	recharging rate at a charging station
$(u, v) \in A$	$d_{u,v}$	driving time from u to v
	$b_{u,v}$	battery consumption from u to v
$i \in R$	t_i^{arr}	time at which request i becomes available
	$v_i^{\uparrow}$	pickup location of request i
	$v_i^{\downarrow}$	dropoff location of request i
	l_i	customer load of request i
	d_i^{max}	maximum user ride time of customer i
	$[t_i^{\text{start}}, t_i^{\text{end}}]$	pickup time windows of request i at $v_i^{\uparrow}$
$k \in K$	v_k^{ini}	initial location of vehicle k
	$B_{k,0}$	initial battery level of vehicle k
	t_k^{avail}	time at which vehicle k becomes available
	C	maximum customer load capacity of a vehicle
	Q	maximum battery capacity of a vehicle

Table 3.1: Summary of the notations used for the EADARP

3.1.4 Fragments

To make the handling of routes easier, each route can be further decomposed into so-called *fragments*. Introduced by Rist and Forbes [35] for the classical DARP and used in many recent approaches for the EADARP [12, 41, 42, 45], fragments are closed sequences of stops where the vehicle starts and ends empty.

A fragment starts when a first request is picked up by a vehicle and ends at the next dropoff location in which the vehicle becomes empty again. In practice, a second type of fragment is also considered. A stop at a charging station can be handled like a fragment, because the vehicle is always empty when entering and leaving a charging station.

With this formalism, a route for vehicle k is then defined as the initial location of the vehicle v_k^{ini} followed by a finite sequence of fragments, abstracting away the details on the order of stops within a fragment or the specific arrival time at each location.

This formalism is also useful to simplify the handling of some of the constraints of the EADARP. The customer load feasibility of a route can be considered fragment by fragment, as the vehicle is always empty between two consecutive fragments. Also, the battery feasibility can be easily verified by computing the battery consumption within a

fragment and computing the state of charge of a vehicle between fragments. The arrival times within a fragment are fully determined by the arrival time at the first location of the fragment. Indeed, as waiting is only allowed when the vehicle is empty, this means that there can be no waiting within the fragment. As such, constraints regarding user ride time can be handled on the scale of a fragment only, instead of looking at a whole route. Also, the lateness created at any location can be determined based on the arrival time at the first location in the fragment.

3.1.5 Illustration with an Example

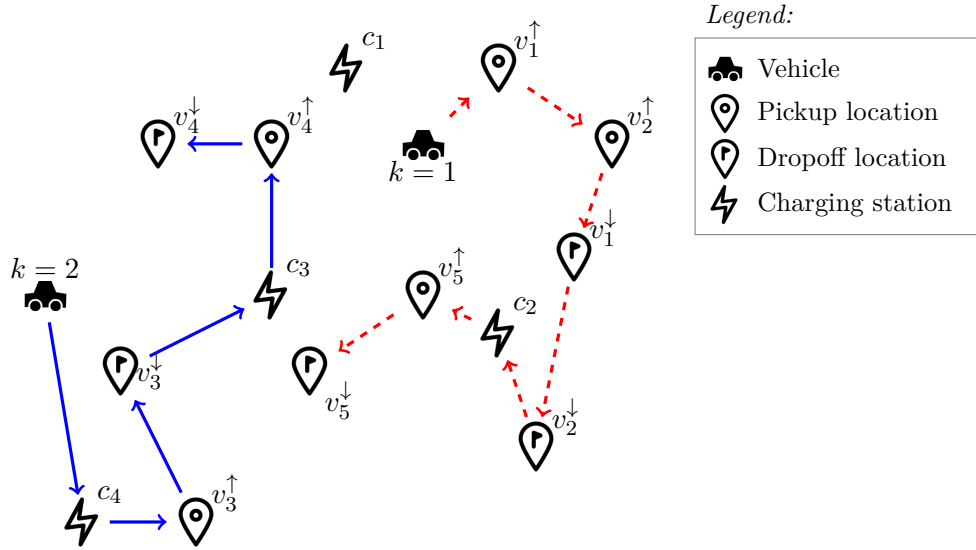


Figure 3.1: Example of an EADARP instance¹

Figure 3.1 shows an example of a simplified EADARP instance, along with a valid solution. A solution is composed of one route for each vehicle, such that all requests are served. For this example instance, the two routes in the solution are:

$$\begin{aligned}
 r_1 : & \left\{ \underbrace{v_1^\uparrow, v_2^\uparrow, v_1^\downarrow, v_2^\downarrow}_{f_1}; c_2(80); \underbrace{v_5^\uparrow, v_5^\downarrow}_{f_2} \right\} & t_{f_1} = 2, t_{c_2} = 11, t_{f_2} = 93 \\
 r_2 : & \left\{ c_4(46); \underbrace{v_3^\uparrow, v_3^\downarrow}_{f_3}; c_3(70); \underbrace{v_4^\uparrow, v_4^\downarrow}_{f_4} \right\} & t_{c_4} = 7, t_{f_3} = 71, t_{c_3} = 92, t_{f_4} = 178
 \end{aligned}$$

Each route is composed of fragments, with the associated time of arrival at the first location of the fragment. In the representation above, transitions between fragments are indicated by a semicolon ';' instead of a simple comma separating two consecutive locations within a fragment.

¹The icons used in the figure are under CC0 license and were taken from <https://cc0-icons.jonh.eu/> and <https://opengameart.org/content/game-icons-expansion>

In route r_1 , the vehicle starts by driving to $v_1^\uparrow$ and will arrive there at time $t_{f_1} = 2$. The internal driving and arrival times in the fragment f_1 are fully determined by the time of arrival at $v_1^\uparrow$. For charging stations, we also indicate the charging time in the definition of the fragment.

Vehicle 2 starts by driving towards charging station c_4 and recharges to full battery. The recharging duration of 46 time units is obtained by considering the initial state of charge of vehicle 2, the battery consumption to drive to c_4 and applying Equation (3.1).

This simplified example does not take into account all constraints of the problem. In particular, the time windows at pickup locations, the maximum user ride time of each request and the waiting times that may happen at some locations have not been considered in this minimalistic example.

3.2 Set Covering Formulation for the EADARP

This section presents the Set Covering formulation of the EADARP, an alternative way to define this optimization problem as an ILP which will be used in the following chapters.

Instead of handling the assignment of requests to vehicles to build routes, this formulation assumes that the set of all feasible routes for an EADARP instance is given and defines an Integer Linear Program whose goal is to find the best way to cover all requests by choosing one route for each available vehicle.

3.2.1 Formulation

Consider the set Ω of all routes that are feasible for the given instance. Because a route is always designed for a specific vehicle, this set can be decomposed into the union of all feasible routes for a given vehicle. Denoting as Ω_k the set of all feasible routes for vehicle k , we have:

$$\Omega = \bigcup_{k \in K} \Omega_k$$

Furthermore, given a route $\omega \in \Omega$ and a request $i \in R$, the constant $\theta_{i,\omega} \in \{0, 1\}$ indicates with value 1 that request i is served by route ω . Meanwhile, the cost of a solution can be obtained by computing individual costs of each route independently from other routes in the solution. As such, we define for each route $\omega \in \Omega$ the associated cost c_ω . This cost can be computed similarly to the objective function defined in Equation (3.2) and is given by:

$$c_\omega = \sum_{(u,v) \in \omega} d_{u,v} + \rho \cdot \sum_{i \in \text{Reqs}(\omega)} d_{i,\omega}^{\text{late}}$$

For each pair $(k \in K, \omega_k \in \Omega_k)$ of a vehicle and a possible route for it, we introduce a binary variable $y_{k,\omega} \in \{0, 1\}$. Then the EADARP can be formulated using an ILP as defined in Equations (3.3) to (3.6).

$$\min \sum_{k \in K} \sum_{\omega \in \Omega_k} c_{\omega} y_{k,\omega} \quad (3.3)$$

$$\text{s.t.} \quad \forall i \in R, \quad \sum_{k \in K} \sum_{\omega \in \Omega_k} \theta_{i,\omega} y_{k,\omega} \geq 1 \quad (3.4)$$

$$\forall k \in K, \quad \sum_{\omega \in \Omega_k} y_{k,\omega} \leq 1 \quad (3.5)$$

$$\forall k \in K, \forall \omega \in \Omega_k, \quad y_{k,\omega} \in \{0, 1\} \quad (3.6)$$

In this formulation, the constraints in Equation (3.4) ensure that all requests are served, while Equation (3.5) makes sure that at most one route is chosen for each vehicle. If a vehicle does not get any route assigned, it remains unused, which can also be interpreted as assigning the empty route to it.

This compact formulation hides away most of the complexity of the EADARP by considering that the set of all feasible routes is known in advance. In particular, all concerns regarding the state of charge of a vehicle and the scheduling constraints are abstracted away. These constraints only intervene in the generation of feasible routes.

3.2.2 Extracting Solutions from the Set Covering ILP

In the rest of this thesis, we will often consider a restricted version of the Set Covering formulation presented above. Instead of assuming that all feasible routes are known in the set Ω , we will solve a sub-instance of the problem where only a subset $\Omega' \subseteq \Omega$ of feasible routes is available.

Also note that the ILP described in Section 3.2.1 is a set covering formulation, with the inequality in Equation (3.4) imposing that each request is served by *at least* one route instead of exactly one. Had we formulated the constraint Equation (3.4) with an equality instead, we would have obtained the Set Partitioning formulation of the EADARP.

In the current case where we consider only a subset of all feasible routes, the Set Covering formulation is more advantageous than the Set Partitioning formulation, because it is less restrictive. Indeed, two routes for different vehicles could fit very well together, with the exception that both serve a same given request. In the case of a set partitioning formulation, this solution would not be allowed, even though it might be the best solution available from the current set of known routes. The set covering formulation allows for this situation to happen. Such a solution is promising to generate high-quality feasible solutions, as it has a better objective value than other solutions while still containing duplicate requests. The feasible solutions obtained by removing the duplicate requests have a high potential, which is why we want to consider them as valid solutions to the Set Covering ILP.

To ensure that the final solution that is returned remains feasible according to the EADARP constraints, an additional post-processing step is needed, which detects such

cases where a request is covered by multiple routes and repairs the solution. In the rest of this section, we will call *duplicate request* a request which is covered by multiple routes in an optimal solution of the Set Covering ILP.

In our situation, the post-processing strategy generates all possible combinations of the selected routes where each request is covered only once. This serves the goal of both finding the best incumbent possible and also enriching the set of known routes with multiple promising routes. The process to find all duplicate requests and generate the new solutions is detailed in Algorithm 3.1.

This algorithm iterates over all vehicles and finds which route (if any) has been selected to be part of the solution. Whenever a request i is encountered that has already been covered by a previous route, the algorithm generates the two possibilities for which i is covered exactly once. Either (1) request i remains served in the previous route, in which case we have to remove it from the current route before adding said route to the solution (see line 15), or (2) request i should continue being served by the current route, which means updating the routes already present in the solution to remove request i from them (see line 25).

In the case (1), special care is needed if there is more than one duplicate request in the same route r . Indeed, for the first duplicate request i_1 , the current route r has not yet been added to the set of solutions, so we only have to remove i_1 from r and add the newly obtained route to *sols*. For a subsequent duplicate request i_2 in the same route, we cannot simply remove i_2 from r , as that would cancel the work that was done on i_1 . Instead, we have to modify the updated route that was already added to *sols*. the example below will illustrate the procedure when multiple duplicate requests are present in the same route. Algorithm 3.1 also accounts for the case when a request is covered more than twice.

At the end of the algorithm, the list of all valid routes built from the ILP solution is returned. If we are only interested in the optimal solution of the Set Covering ILP, we only have to iterate over this list of solutions to find the one with the best objective value. In the worst case, this algorithm can generate an exponential number of solutions, because we double the number of partial solutions that are considered each time that a duplicate request is found. In practice, we do not expect this to be a problem, since only few solutions to the set covering ILP will contain duplicate requests. This is confirmed by experimental results, as discussed in Section 7.4.1.

The following example should make this procedure more understandable. Consider an EADARP instance with $n_K = 3$ vehicles and $n = 5$ requests. Assume that a given ILP formulation returns an optimal solution composed of the following three routes r_1, r_2, r_3 , respectively for vehicle 1, 2, 3:

$$\begin{aligned} r_1 &= \{v_1^\uparrow, v_2^\uparrow, v_1^\downarrow, v_2^\downarrow\} \\ r_2 &= \{v_3^\uparrow, v_3^\downarrow, v_4^\uparrow, v_4^\downarrow\} \\ r_3 &= \{v_2^\uparrow, v_5^\uparrow, v_2^\downarrow, v_3^\uparrow, v_5^\downarrow, v_3^\downarrow\} \end{aligned}$$

Algorithm 3.1 Post-Processing step for the Set Covering ILP

Input: a dictionary *routes* of all known routes grouped by vehicle with their associated value from the ILP solution

Return: all feasible solutions generated from the routes selected in the ILP

```

1: sols  $\leftarrow []$ ; PUSH(sols, [])  $\triangleright$  Initialize with empty solution
2: covered  $\leftarrow \{i \mapsto [] \mid \forall i \in R\}$   $\triangleright$  Track for  $i \in R$  the list of selected routes serving  $i$ 
3: for all  $k \in K$  do
4:   has_route  $\leftarrow$  false
5:   for all  $r \in \text{routes}[k]$  do
6:     if VALUE( $r$ ) = 1 then  $\triangleright$  Route  $r$  was selected
7:       has_route  $\leftarrow$  true
8:       multi_cover  $\leftarrow$  false
9:       for all  $i \in \text{REQS}(r)$  do
10:        if  $i \in \text{covered}$  then  $\triangleright$  Request  $i$  was already covered previously
11:          first_mc  $\leftarrow \neg \text{multi\_cover}$   $\triangleright$  Is it the first duplicate req for  $r$ ?
12:          multi_cover  $\leftarrow$  true
13:          new_sols = COPY(sols)
14:           $\triangleright$  Branch on which route should keep serving request  $i$ :  $\triangleleft$ 
15:           $\triangleright$  (1) Remove  $i$  from  $r$  and keep the older routes unchanged  $\triangleleft$ 
16:          if first_mc = true then  $\triangleright$  Route  $r$  has not yet been added to sols
17:             $r' \leftarrow \text{REMOVEREQ}(r, i)$ 
18:            for all partial_sol  $\in$  new_sols do
19:              UPDATE(partial_sol,  $k, r'$ )
20:          else  $\triangleright$  Route  $r$  was already seen, we have to modify it in sols
21:            for all partial_sol  $\in$  new_sols do
22:              current_r  $\leftarrow$  GETROUTEBYVEHICLE(partial_sol,  $k$ )
23:               $r' \leftarrow \text{REMOVEREQ}(\text{current\_r}, i)$ 
24:              UPDATE(partial_sol,  $k, r'$ )
25:           $\triangleright$  (2) Keep  $r$  unchanged and remove  $i$  from the older routes  $\triangleleft$ 
26:          for all partial_sol  $\in$  sols do
27:            old_r  $\leftarrow$  FINDCOVER(partial_sol,  $i$ )
28:             $r'' \leftarrow \text{REMOVEREQ}(\text{old\_r}, i)$ 
29:            UPDATE(partial_sol, old_r.k,  $r''$ )
30:          sols  $\leftarrow$  sols  $\cup$  new_sols
31:          PUSH(covered[ $i$ ],  $r$ )  $\triangleright$  Indicate that  $i$  is covered by  $r$ 
32:          if not multi_cover then  $\triangleright$  Every request in  $r$  is covered once
33:            for all partial_sol  $\in$  sols do
34:              PUSH(partial_sol,  $r$ )
35:          break  $\triangleright$  There is always at most one route selected per vehicle
36:          if not has_route then  $\triangleright$  No route was selected for  $k$ , add the empty route
37:            for all partial_sol  $\in$  sols do
38:              PUSH(partial_sol, empty_routek)
39: return sols

```

A solution composed of these routes is not a valid solution, because the requests 2 and 3 are served by two routes each. Then, running Algorithm 3.1 on this solution would generate four valid solutions to the original instance. Let us detail the operation of this algorithm. For the vehicles $k \in \{1, 2\}$, no duplicate request is detected, so at the start of the for loop for $k = 3$, $sols$ only contains a single partial solution s defined as:

$$s = \begin{cases} r_1 \\ r_2 \\ \emptyset \end{cases}$$

In the loop for $k = 3$, we first detect that request 2 is a duplicate request, served by both r_1 and r_3 . As this is the first duplicate request for r_3 , we generate the two solutions that serve request 2 exactly once as follows:

$$s_1 = \begin{cases} r_1 = \{v_1^\uparrow, v_2^\uparrow, v_1^\downarrow, v_2^\downarrow\} \\ r_2 = \{v_3^\uparrow, v_3^\downarrow, v_4^\uparrow, v_4^\downarrow\} \\ r'_3 = \{v_5^\uparrow, v_3^\uparrow, v_5^\downarrow, v_3^\downarrow\} \end{cases}$$

$$s_2 = \begin{cases} r'_1 = \{v_1^\uparrow, v_1^\downarrow\} \\ r_2 = \{v_3^\uparrow, v_3^\downarrow, v_4^\uparrow, v_4^\downarrow\} \\ r_3 = \{v_2^\uparrow, v_5^\uparrow, v_2^\downarrow, v_3^\uparrow, v_5^\downarrow, v_3^\downarrow\} \end{cases}$$

Continuing the algorithm, we encounter request 3, which is also covered twice, by r_2 and r_3 . Once again, the algorithm has to branch on which route should keep serving request 3. As we have already added a route for vehicle $k = 3$ when handling request 2, we have to modify the current route in s_1 and s_2 . In particular, note that the route for vehicle 3 in s_1 is r'_3 and not r_3 , as request 2 was already removed from it.

From s_1 , we generate two new solutions $s_{1,1}$ and $s_{1,2}$ as follows. In $s_{1,1}$, we remove request 3 from vehicle 3 and let vehicle 2 serve this request. In $s_{1,2}$, we do the opposite and remove request 3 from vehicle 2. We get the following solutions:

$$s_{1,1} = \begin{cases} r_1 = \{v_1^\uparrow, v_2^\uparrow, v_1^\downarrow, v_2^\downarrow\} \\ r_2 = \{v_3^\uparrow, v_3^\downarrow, v_4^\uparrow, v_4^\downarrow\} \\ r''_3 = \{v_5^\uparrow, v_5^\downarrow\} \end{cases}$$

$$s_{1,2} = \begin{cases} r_1 = \{v_1^\uparrow, v_2^\uparrow, v_1^\downarrow, v_2^\downarrow\} \\ r'_2 = \{v_4^\uparrow, v_4^\downarrow\} \\ r'_3 = \{v_5^\uparrow, v_3^\uparrow, v_5^\downarrow, v_3^\downarrow\} \end{cases}$$

Similarly for s_2 , we generate the solutions $s_{2,1}$ and $s_{2,2}$ as follows:

$$s_{2,1} = \begin{cases} r'_1 = \{v_1^\uparrow, v_1^\downarrow\} \\ r_2 = \{v_3^\uparrow, v_3^\downarrow, v_4^\uparrow, v_4^\downarrow\} \\ r_3''' = \{v_2^\uparrow, v_5^\uparrow, v_2^\downarrow, v_5^\downarrow\} \end{cases}$$

$$s_{2,2} = \begin{cases} r'_1 = \{v_1^\uparrow, v_1^\downarrow\} \\ r'_2 = \{v_4^\uparrow, v_4^\downarrow\} \\ r_3 = \{v_2^\uparrow, v_5^\uparrow, v_2^\downarrow, v_3^\uparrow, v_5^\downarrow, v_3^\downarrow\} \end{cases}$$

As request 5 is only covered once, the algorithm terminates and returns the four solutions $sols = \{s_{1,1}, s_{1,2}, s_{2,1}, s_{2,2}\}$. All of them are valid solutions for the given instance and only serve each request once. Naturally, the service times, battery consumption, and overall cost of the new routes have to be recomputed to obtain the total cost of each of the new solutions.

3.3 Working with Dynamic Instances

The formulations presented up to this point have defined the static version of the DynEADARP, where all requests are known from the start. This static formulation is the main setting that will be investigated in this thesis. For completeness sake, this section will discuss how to handle the dynamic formulation. In the following chapters, unless specified otherwise we will always be targeting the developed algorithms towards the static instances. Still, as this section will show, all techniques developed in this thesis are also applicable to dynamic instances, only needing some minor adaptations to best fit their specificities.

In DynEADARP instances, each request $i \in R$ has a time t_i^{arr} at which it becomes available for planning. For real-time planning purposes, it would not be reasonable to wait for all requests to become available before computing a solution. Indeed, there may be cases where the first request available has to be served before the last request has been received.

As such, algorithms solving DynEADARP instances have to find a balance between making decisions early to serve currently open requests and remaining flexible to accommodate new requests. On the one side, making greedy decisions early ensures that the first requests are served to near-optimal costs, but it might also lead to choices that become sub-optimal in the long run, once more requests are available. On the other hand, waiting for new requests to become available might make it easier to incorporate them into the final solution, but waiting too long might create some lateness when serving earlier requests, which would deteriorate the final solution.

3.3.1 Dynamic Framework

To handle this trade-off in dynamic instances, this thesis adopts the same dynamic framework as Tomandl et al. [45]. This approach starts from an empty sub-instance at instant $t = 0$ and progressively builds routes for the currently available requests by increasing the considered time instant t until reaching the full instance at $t = T^{\text{plan}}$. To avoid making sub-optimal short-sighted decisions, the whole solution to any given sub-instance is not used in the final solution. Instead, only parts of the solution that cannot be pushed back to a later time instant are fixed. Considering that each route in the current sub-solution is composed of a sequence of fragments, only the fragments that have to be started before the arrival of the next request are definitively adopted in the final solution. This dynamic framework retains flexibility to accommodate new requests that may become available in the future by only fixing the decisions which cannot be postponed.

In this approach, we consider that a started fragment cannot be modified anymore. Once a vehicle starts a fragment, it is not possible to modify any stop within the fragment. The vehicle is considered reserved until it reaches the end of its current fragment. In the instance definition in Section 3.1.1, we have introduced t_k^{avail} to account for this period of unavailability. For dynamic instances, t_k^{avail} is the time at which vehicle k reaches the end of its assigned fragments and becomes available for planning.

Formally, the customer requests are considered in increasing order of the time t^{arr} at which they become available. At a given time $t \in [0, T^{\text{plan}}]$, a subset $R_t \subseteq R$ of requests is available for planning. Among them, a partial solution s , established at a previous time instant, already serves some of the requests in R_t . The set of currently open requests is more restricted and can be denoted as $O_t \subseteq R_t$. Furthermore, the partial solution s also gives the location and time at which each vehicle will become available, along with its state of charge.

The set O_t along with the locations, times of availability, and battery level of each vehicle forms a static EADARP instance which can be solved independently from the original problem. This sub-instance can be solved using any of the techniques discussed in this thesis, yielding a solution subsol that assigns all requests in O_t to routes.

To preserve flexibility, only the fragments that have to start before the next request becomes available should be fixed in the partial solution s . This can be achieved by finding the time t' at which the next request becomes available and updating the partial solution s by considering each route in subsol and adding all fragments from the route which have a starting time earlier than t' .

Finally, in the last instant that is considered, all routes in the sub-solution are fixed, which guarantees that the partial solution constructed by this framework is a valid solution for the original instance.

3.3.2 Illustration of the Dynamic Framework

Table 3.2 illustrates the proposed dynamic framework on a small example with $n = 4$ requests and $n_K = 2$ vehicles. For simplicity, battery considerations are omitted from this example.

time t	R_t	O_t	$subsol$	t'	Solution s
$t_1 = 10$	$\{1\}$	$\{1\}$	$\left\{ \underbrace{\{v_1^\uparrow, v_1^\downarrow\}}_{f_1} \quad t_{f_1} = 20 \right.$	15	$\emptyset$
$t_2 = 15$	$\{1, 2\}$	$\{1, 2\}$	$\left\{ \begin{array}{l} \emptyset \\ \underbrace{\{v_1^\uparrow, v_1^\downarrow\}}_{f_2} \quad t_{f_2} = 20 \\ \underbrace{\{v_2^\uparrow, v_2^\downarrow\}}_{f_3} \quad t_{f_3} = 32 \end{array} \right.$	30	$\left\{ \begin{array}{l} f_2 \\ \emptyset \end{array} \right.$
$t_3 = 30$	$\{1, 2, 3\}$	$\{2, 3\}$	$\left\{ \begin{array}{l} \emptyset \\ \underbrace{\{v_2^\uparrow, v_2^\downarrow\}}_{f_4} ; \underbrace{\{v_3^\uparrow, v_3^\downarrow\}}_{f_5} \quad t_{f_4} = 38, t_{f_5} = 44 \end{array} \right.$	40	$\left\{ \begin{array}{l} f_2 \\ f_4 \end{array} \right.$
$t_4 = 40$	$\{1, 2, 3, 4\}$	$\{3, 4\}$	$\left\{ \begin{array}{l} \emptyset \\ \underbrace{\{v_4^\uparrow, v_3^\uparrow, v_4^\downarrow, v_3^\downarrow\}}_{f_6} \quad t_{f_6} = 42 \end{array} \right.$	T^{plan}	$\left\{ \begin{array}{l} f_2 \\ f_4 \rightarrow f_6 \end{array} \right.$

Table 3.2: Example of the dynamic framework

At each of the considered time instants, a solution to the sub-instance induced by O_t is found in the column $subsol$. Then, if some fragment in $subsol$ has to start before the next request comes in, it is added to the final solution s . This is the case at instant t_3 , where the fragment f_4 has to start at time $t_{f_4} = 38 < t_4 = 40$. Otherwise, the fragment is ignored, because there might be a better way to serve all open requests in the future. For example at instant t_1 , the fragment f_1 is ignored because $t_{f_1} = 20 \geq 15 = t_2$. In that case, the very same fragment is proposed at the next instant t_2 , but another situation would also be possible. As an example, fragment f_5 is ignored at instant t_3 and request 3 is integrated into a different fragment at the following instant t_4 .

In a given sub-solution, it may be the case that some of the fragments are fixed, while others are ignored, such as at time t_2 , where fragment f_2 is added to the fixed solution, while fragment f_3 is discarded. Also, it might be that some vehicles do not get a new fragment. In that case, they remain unused and are available again for the next instant.

In the final instant t_4 , all requests are available and $R_{t_4} = R$ holds, so all fragments are fixed to obtain a valid final solution serving all requests.

The Construct, Merge, Solve & Adapt heuristic

In contrast to existing search-based approaches to solve the EADARP, this thesis proposes to develop hybrid algorithms based on the Construct, Merge, Solve & Adapt (CMSA) heuristic, which was introduced by Blum et al. [7].

Before presenting hybridization strategies relying on CMSA in the later chapters, this chapter will first introduce this technique and develop a standalone algorithm to apply CMSA for the EADARP.

This chapter starts by describing the general CMSA framework in Section 4.1. Section 4.2 explains how to tailor this framework to solve the EADARP, explaining the adaptations made to the general framework to best fit this optimization problem. Finally, Section 4.3 discusses two specific strategies developed to address the main bottleneck of CMSA on larger instances, namely to speed up the Construct step.

4.1 Introduction to CMSA

Construct, Merge, Solve & Adapt is a metaheuristic framework that works by continuously refining a pool of solution components until a satisfactory solution is reached. Components are usually some partial solutions that can be combined with one another to form a solution to the original problem. As its name indicates, CMSA is composed of four steps that are applied sequentially. These steps are then repeated until some termination criterion is met.

The Construct step uses a randomized construction operator to generate new feasible solutions. These new solutions are used to diversify the components available in the pool. Enough randomization is needed to ensure that the heuristic does not remain

stuck around a local optimum which is not globally optimal. The Construct step should generate enough components to populate the CMSA pool.

Then, the Merge step iterates over all previously generated solutions to concretely add them to the pool. This step varies depending on the relation between components and actual solutions. The goal is to extract from a solution the components that form it and merge these new components into the pool.

Afterwards, the Solve step aims at improving the current best solution by finding the combination of components currently in the pool that yields the best overall solution to the problem. Formally, the components present in the pool form a sub-instance of the original problem and the goal of the Solve step is to find the optimal solution of this induced sub-instance. This is usually achieved by means of an exact Integer Linear Program (ILP) that finds the optimal solution available from the pool.

Finally, the Adapt step handles the management of the pool. It tracks components that have remained unused for the longest time and discards them based on some criteria. This step ensures that the pool remains small enough, so that the Solve step can always find the optimal solution in a reasonable time.

The different steps of CMSA are illustrated in Figure 4.1. In this example, a solution is composed of a diamond, a square, and a circle of any color. The pool from the previous CMSA iteration is depicted in the top left corner, with some components already present. The Construct step generates a random new solution (in gray in the figure). This new solution is then merged into the pool. In this case, all new components are added to the pool, but it might also happen that some components of a new solution are already present in the pool. Afterwards, the Solve step finds the best solution that can be obtained by combining components from the pool. Finally, the Adapt step removes components from the pool according to the specification of the algorithm.

4.2 Adapting CMSA to the EADARP

Designing a CMSA framework starts with the definition of solution components, which will form the pool. In the case of the EADARP, a natural idea for components is to consider each feasible route for a vehicle. Another alternative for components would be to consider fragments, as defined in Section 3.1.4. Using fragments as components has the advantage of a finer-grained representation, as fragments are independent from the vehicles, but they require additional considerations in their integration with CMSA. Indeed, fragments cannot be considered as isolated structures, as their objective value depends on the state at the location before the fragment. Furthermore, the combination of fragments to form a feasible solution to the EADARP is also more complex than for routes. This is why the decision was made in this thesis to use routes as components for the CMSA algorithm. Based on a set of available routes, a solution is formed by choosing one route for each vehicle and by verifying that the obtained solution satisfies all constraints from the problem formulation. The pool of components tracks for each

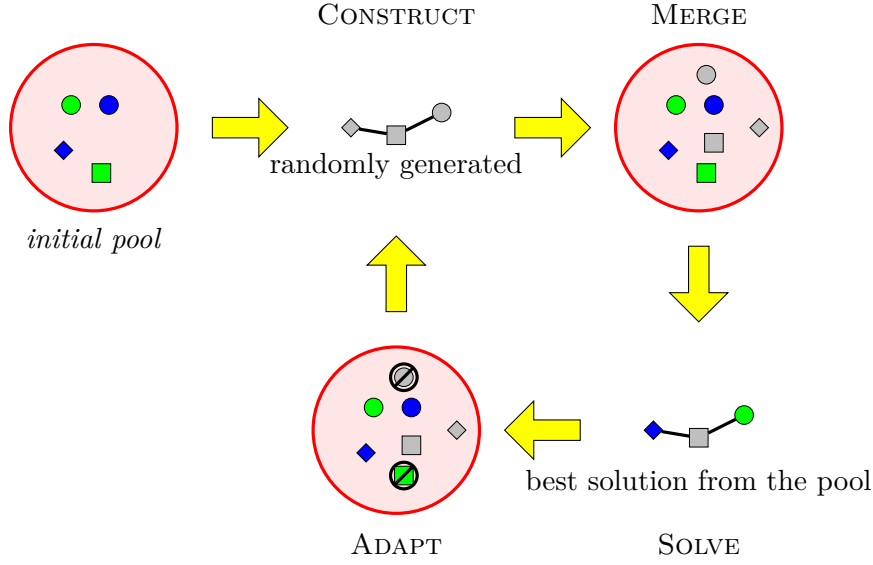


Figure 4.1: The different steps of a CMSA iteration

vehicle which routes are available. Each component additionally stores relevant internal information, such as the partial cost of the route and a list of all served requests.

The CMSA framework for the EADARP is described in Algorithm 4.1. Each of the steps is explained with more details in the rest of this section. The input argument *config* is used as a placeholder for configurable parameters of the CMSA algorithm, such as the age and size limit of the CMSA pool.

4.2.1 Construct

The goal of the Construct step is to generate new diverse components. As such, a simple approach using a randomized construction heuristic for the EADARP is sufficient to generate new feasible solutions which can be decomposed into a set of components. At each iteration in the CMSA framework, the Construct step randomly generates a new solution. This can be achieved by starting from an empty solution and calling different repair operators, such as those from the LNS approach of Tomandl et al. [45].

Given a partial (or empty) solution, the repair operation iterates over all requests that are not yet part of the solution and inserts them one by one at the best feasible position. The different repair operators determine the order in which these unserved requests are considered. In this thesis, we consider the following orders:

- **Random**: The unserved requests are randomly shuffled.
- **Chronological**: Unserved requests are sorted in ascending value of their starting pickup time window t_i^{start} .

Algorithm 4.1 CMSA Framework for the EADARP

Input: an EADARP instance *inst* and a *config*
Return: the best incumbent solution found

```

1: pool  $\leftarrow \emptyset$   $\triangleright$  Pool of components for the CMSA
2: incumbent  $\leftarrow \emptyset$   $\triangleright$  Current best solution
3: while termination criteria not met do
4:    $\triangleright$  Construct  $\triangleleft$ 
5:   new_sol, new_routes  $\leftarrow$  CONSTRUCT(inst, config)
6:   if new_sol is better than incumbent then
7:     incumbent  $\leftarrow$  new_sol
8:    $\triangleright$  Merge  $\triangleleft$ 
9:   for all r  $\in$  new_routes do
10:    pool  $\leftarrow$  pool  $\cup$  {r}  $\triangleright$  Update pool
11:    $\triangleright$  Solve  $\triangleleft$ 
12:   ilp_sol  $\leftarrow$  SOLVEMASTERILP(inst, pool)
13:   if ilp_sol is better than incumbent then
14:     incumbent  $\leftarrow$  ilp_sol
15:    $\triangleright$  Adapt  $\triangleleft$ 
16:   ADAPTPOOL(pool, config)  $\triangleright$  Check age and discard components
17: return incumbent

```

- Greedy: All unserved requests are considered simultaneously, and only the one with the best insertion position is inserted.

Algorithm 4.2 presents an overview of this procedure for the random and chronological insertion orders. For the random insertion, we have:

$$order^{rnd}(list) = SHUFFLE(list)$$

Meanwhile the chronological insertion functions like a sorting function:

$$order^{chron}(list) = SORT(list, i \rightarrow t_i^{start})$$

The greedy construction presented in Algorithm 4.3 requires to go over all requests to find the best insertion position. As such, this operator is quadratic in the number of unserved requests. On larger instances, this quadratic runtime becomes too expensive and the greedy insertion is disabled.

Given a request to insert into the solution, the repair operator iterates over all routes to insert the request at the position that minimizes the objective value of the partial solution. The best insertion position is found by considering all possible positions for the pickup and dropoff locations of the request, regardless of battery considerations. Then, charging stops are inserted in the solution using the on-the-fly insertion strategy from

Algorithm 4.2 Request insertion procedure for random and chronological order**Input:** a partial (or empty) solution s and an ordering function $order$ **Return:** a feasible solution and a set of feasible routes

```

1:  $routes \leftarrow \emptyset$ 
2: for all  $i \in order(UNSERVEDREQUESTS(s))$  do
3:    $best\_ins \leftarrow \emptyset$ 
4:   for all  $k \in SHUFFLE(K)$  do ▷ Consider all routes in randomized order
5:      $ins_k \leftarrow FINDBESTINSERTION(s, k, i)$ 
6:     if  $ins_k$  is better than  $best\_ins$  then
7:        $best\_ins \leftarrow ins_k$ 
8:    $r \leftarrow INSERT(s, i, best\_ins)$  ▷ Realize best insertion and collect modified route
9:    $PUSH(routes, r)$ 
10: return  $s, routes$ 

```

Bresich et al. [12] and Tomandl et al. [45] to generate a feasible solution. The partial objective value is evaluated on this solution. The order in which the routes are considered at line 4 of Algorithm 4.2 is randomized to add some diversification to the insertion process.

Algorithm 4.3 Greedy request insertion procedure**Input:** a partial (or empty) solution s **Return:** a feasible solution and a set of feasible routes

```

1:  $routes \leftarrow \emptyset$ 
2: while  $UNSERVEDREQUESTS(s) \neq \emptyset$  do
3:    $best\_ins \leftarrow \emptyset$ 
4:   for all  $i \in UNSERVEDREQUESTS(s)$  do ▷ Consider all unserved requests
5:     for all  $k \in SHUFFLE(K)$  do
6:        $ins_{i,k} \leftarrow FINDBESTINSERTION(s, k, i)$ 
7:       if  $ins_{i,k}$  is better than  $best\_ins$  then
8:          $best\_ins \leftarrow ins_{i,k}$ 
9:    $r \leftarrow INSERT(s, best\_ins)$  ▷ Realize best insertion and collect modified route
10:   $PUSH(routes, r)$ 
11: return  $s, routes$ 

```

To populate the CMSA pool with more components with a single call to a construction heuristic, we adapt the repair operators from [45] to also collect partial feasible routes that have been generated in the request insertion procedure. Indeed, whenever a request is inserted into a partial solution, we obtain a new partial solution with one more request. In this new partial solution, one route has been modified and represents a new feasible route that can be considered in the CMSA pool. By collecting these routes, we can add n

routes of increasing length to the pool at each call to the construction heuristic, instead of only adding the $n_K < n$ routes that are present in the final constructed solution.

4.2.2 Merge

Once new solutions have been generated, the Merge step iterates over all routes in the solution and adds them to the pool. As we want to be able to easily find which are the least promising components to discard in the Adapt step, the CMSA pool also keeps track of the age of each component. This age evaluates how much time has passed since a component was last used in the incumbent. With this, we are able to sort components by their most recent use. If a component has remained unused for a long time, we assume that it is unlikely to be part of an optimal solution and can discard it from the pool.

Concretely, we define the age of a component as the number of iterations since it was added to the pool or since the last time that this component was used in the incumbent. To limit the computational overhead of updating the age of components in the pool, in the algorithm we only store the last iteration at which a component was used, instead of keeping track of the age of every component in the pool at each iteration. We can deduce the age of a component by subtracting this number from the current iteration.

Some routes from the solution delivered by the Construct step may already be present in the pool. In that case, their age is updated accordingly, by setting their last iteration of use to the current iteration.

4.2.3 Solve

Having updated the pool with new components, the Solve step aims to find the best combination of components to form the new incumbent. This is achieved by solving a restricted form of the Set Covering formulation of the EADARP defined in Section 3.2. Recall that the Set Covering formulation assumes that the set Ω of all feasible routes for the given instance is known. By restricting this set to the subset $\Omega' \subseteq \Omega$ of routes that are currently available in the pool, we obtain a restricted ILP formulation of the EADARP.

An optimal solution to this reduced problem might not be optimal for the more general formulation, but we are guaranteed that it remains feasible. As such, we can find the best possible solution that can be obtained from the pool by solving the Set Covering formulation restricted to Ω' , including the post-processing step discussed in Section 3.2.2, which ensures that all solutions delivered by the Solve step are valid for the original problem.

4.2.4 Adapt

Finally, the Adapt step manages the pool by discarding unused components. Multiple strategies may be used to decide when a component should be discarded. In this thesis, the following two possibilities have been considered. First, whenever a component exceeds

a configured age limit, it is discarded from the pool, because it is considered unlikely to be part of an optimal solution. Secondly, a fixed parameter sets a maximum size for the pool. If the pool gets bigger than this arbitrary parameter, then the oldest components in the pool are discarded until the maximum size is reached. This second strategy ensures that the Solve step remains fast throughout the algorithm, by limiting the number of variables involved in the restricted Set Covering formulation.

With all internal operations defined, the CMSA algorithm can iteratively run until a given stopping criterion is reached. This criterion combines hardware limits like the runtime of the experiments and some convergence criteria, such as the number of iterations since the last improvement. These parameters are then tuned to fit the experimental goals and the test instances.

4.3 Handling Large Instances

Preliminary benchmark results obtained by executing the CMSA algorithm described above on various randomly generated instances of different sizes have highlighted a weakness of the CMSA approach applied to larger instances of the EADARP.

When running the pure CMSA approach on larger instances with at least $n = 500$ requests, the main bottleneck of the CMSA approach lies in the fact that the Construct step is computationally expensive. Indeed, the Construct step inserts all n requests into an empty solution, considering for each request all n_K possible routes to find the insertion position yielding the best possible solution. As the number of requests and vehicles in the instance grows, this construction heuristic becomes expensive. Because of this, the CMSA algorithm can only perform a few iterations within the allotted runtime. Meanwhile, the LNS approach from Tomandl et al. [45] only has to execute one construction step to initialize the solution and then only performs much cheaper destroy and repair cycles.

The rest of this section presents two strategies that have been developed to mitigate this weakness and speed up the Construct step, at the cost of deteriorating the quality of the generated solutions. The first strategy introduces a limit on the number of routes that are considered when inserting a request into a partial solution. The second strategy speeds up the Construct step when an incumbent already exists by calling the request insertion heuristic on a partial solution obtained by destroying the incumbent.

4.3.1 Limiting the Number of Routes Considered for Insertion

Recall the procedure to insert all requests into a partial solution to generate a feasible solution presented in Algorithm 4.2. In this algorithm, we consider every possible insertion position in each route of the solution.

To avoid considering every route, we can set a limit on the number of routes that are to be considered. Depending on the value that is decided for this limit, this can significantly reduce the number of positions that are considered for insertion, thereby speeding up the

Construct step. As a trade-off, this strategy also has a negative impact on the quality of the generated solution. Indeed, each request is inserted at the position having the least impact among the considered routes, instead of the best insertion position in the whole partial solution.

Formally, given an EADARP instance with n_K vehicles, we introduce the integer parameter $k_{\max} \in \{1, \dots, n_K\}$ that determines the maximum number of routes that are considered when inserting a request into a partial solution. The lower the value, the faster the Construct step, but the worse the objective value of the generated solution. Conversely, setting $k_{\max} = n_K$ is equivalent to disabling this strategy and considering all routes for insertion.

The changes in Algorithm 4.2 are presented in Algorithm 4.4. Note how line 4 in the original algorithm is modified to take the value of $k_{\max}$ into account in lines 4 to 8 of Algorithm 4.4. For the greedy request insertion, Algorithm 4.3 is updated by applying the same changes and only considering up to $k_{\max}$ routes in the internal for loop.

Algorithm 4.4 Limited request insertion procedure

Input: a partial solution s , an ordering function $order$ and a parameter $k_{\max}$
Return: a feasible solution and a set of feasible routes

```

1:  $routes \leftarrow \emptyset$ 
2: for all  $i \in order(UNSERVEDREQUESTS(s))$  do
3:    $best\_ins \leftarrow \emptyset$ 
4:    $counter \leftarrow 0$ 
5:   for all  $k \in SHUFFLE(K)$  do
6:      $counter \leftarrow counter + 1$ 
7:     if  $counter > k_{\max}$  then  $\triangleright$  Only consider up to  $k_{\max}$  routes
8:       break
9:      $ins_k \leftarrow \text{FINDBESTINSERTION}(s, k, i)$ 
10:    if  $ins_k$  is better than  $best\_ins$  then
11:       $best\_ins \leftarrow ins_k$ 
12:     $r \leftarrow \text{INSERT}(s, i, best\_ins)$ 
13:     $PUSH(routes, r)$ 
14: return  $s, routes$ 

```

The parameter $k_{\max}$ determining the maximum number of routes considered for insertion has to be tuned to find the best trade-off between speeding up the Construct step and generating solutions with a worse solution quality. This tuning is discussed in more detail in Section 7.2.

4.3.2 Partially Destroying the Incumbent to Generate New Solutions

The second strategy that will be presented here aims to avoid generating new solutions from scratch at every Construct step. In the very first CMSA iteration, no solution is

known, so the Construct step has to generate a new feasible solution by starting from an empty solution. In subsequent calls to the Construct step, we want to take advantage of the fact that the CMSA framework keeps track of a feasible solution, which is the current best found solution, and use this incumbent solution as a starting point for the Construct step.

The idea here is similar to a destroy-repair cycle in an LNS, but with a different objective. The goal of an LNS cycle is to find improving solutions by destroying parts of the current solution and repairing them into a better configuration. Here, the objective is to generate a diverse set of solutions, regardless of their actual quality, in the hope that some routes from different solutions will fit together to deliver an improved solution.

Concretely, the LNS operators from Tomandl et al. [45] destroy a solution by removing a small number of requests chosen randomly and reinserting them using one of the repair operators discussed in Section 4.2.1. To ensure that the Construct step continues to deliver a diverse set of solutions, the destroy step developed here to obtain a partial solution from the incumbent solution will remove a larger proportion of the requests in the solution. The repair process functions the same as the LNS and reinserts the removed requests to generate a feasible solution.

Formally, given an EADARP instance with n requests, we introduce the integer parameter $n_{\text{destroy}} \in \{1, \dots, n\}$ that determines the number of requests that will be destroyed in the incumbent to obtain a starting point for the Construct step. Algorithm 4.5 presents the procedure to apply this parameter in pseudocode. At line 4, we obtain a partial solution s' from the incumbent s by randomly removing n_{destroy} requests from s . Then, at line 5, all missing requests are reinserted into s' by applying a repair operator, such as the ones presented in Section 4.2.1, to obtain a complete feasible solution.

Algorithm 4.5 Construction heuristic with partial destroy

Input: a (possibly empty) solution s , a parameter n_{destroy} and a repair operator repair

Return: a feasible solution and a set of feasible routes

- 1: $\text{routes} \leftarrow \emptyset$
 - 2: $s' \leftarrow \text{COPY}(s)$
 - 3: **if** $s' \neq \emptyset$ **then**
 - 4: $\text{DESTROY}(s', n_{\text{destroy}})$
 - 5: $s', \text{routes} \leftarrow \text{repair}(s')$
 - 6: **return** s', routes
-

Setting $n_{\text{destroy}} = n$ disables this strategy and always generates a new solution from scratch. The lower the value for n_{destroy} , the faster the Construct step, as there are fewer requests to insert into the solution. As a trade-off, a lower value also means a higher risk of generating solutions similar or equal to the incumbent, which would weaken the Solve step of CMSA. A lower value also means that fewer partial feasible routes are generated

in the Construct step, but this should only have a reduced impact on the diversity of components in the CMSA pool, because only routes with very few requests served are generated in the first few request insertions of the construction heuristic.

Same as for the parameter $k_{\max}$ discussed above, n_{destroy} has to be tuned to find a compromise between a faster Construct step and a more diverse set of solutions. The actual tuning to evaluate the performance of this strategy is discussed in Section 7.2.

Both the $k_{\max}$ and n_{destroy} strategies can be applied simultaneously. Indeed, the $k_{\max}$ strategy modifies the internal mechanism of the reinsertion operators by limiting the number of considered routes, while the n_{destroy} strategy works on a higher level by reducing the number of unserved requests that have to be inserted when constructing a feasible solution.

Hybridizing CMSA with Column Generation

As mentioned in Chapter 1, CMSA has a high potential for hybridization with other approaches. A first promising approach consists of bridging the gap between exact and heuristic methods by integrating insights from solving sub-instances to optimality into the heuristic CMSA approach. In particular, Column Generation relies on similar concepts to the Solve step of CMSA, making it a good candidate for hybridization.

This chapter starts by introducing Column Generation as an independent approach to solve the EADARP. Section 5.1 gives a short introduction to Column Generation. Then, Section 5.2 explains how to apply this approach for the EADARP. Finally, Section 5.3 presents the labeling algorithm that has been developed to solve the Pricing Subproblem.

The rest of this chapter investigates how to combine Column Generation with the CMSA framework developed in Chapter 4, with Section 5.4 discussing how to enhance the Solve step of CMSA by using Column Generation.

5.1 Introduction to Column Generation

The introduction given in this section is based on Chapter 11 in the book “Integer Programming (Second Edition)” [46] by Wolsey. We refer to this book for more details on the concepts touched upon here. Note that this book uses examples with maximization problems, while the EADARP is a minimization problem. The concepts remain fully applicable. The direction of some inequalities only needs to be adapted, such as for the reduced cost that will be discussed in this section. For a minimization problem, the goal is to find solutions that decrease the objective value, which is why we want to find solutions with a negative reduced cost.

Column Generation is a procedure used in mathematical programming to solve the Linear Programming relaxation of an ILP with a very large number of variables (also known as *columns*). Solving the full problem directly is not feasible because of the large number of columns. Instead, Column Generation starts from a small feasible set of columns and iteratively adds new columns, until an optimal solution is reached. The idea is to generate new columns cleverly by taking advantage of the dual variables delivered by the primal simplex algorithm.

Assuming an LP formulation with decision variable x , applying the primal simplex algorithm to solve this problem yields an optimal solution x^* . Each constraint in the Master Problem also gets an associated dual variable, which quantifies the marginal value of this constraint in the final solution x^* . In the case of a non-zero dual value, the constraint is activated, meaning that the inequalities in this constraint are tight. For a constraint of the form $Ax \geq b$, this means that the equality condition is reached and it holds that $Ax^* = b$. This means that this constraint is limiting for the improvement of the objective value and that focusing on it can lead to a better solution. Conversely, if the dual value is zero, the constraint is not activated, meaning that there is some slack in the constraint and it is not limiting for finding better solutions. For more details regarding duality, we refer to [46], in particular to Chapter 2.

At each iteration, the current set of columns defines a Restricted Master Problem, which is a restriction of the original LP. This problem is solved to optimality using the primal simplex algorithm. Then the procedure has to generate a new column. This is achieved by finding the column which has the lowest negative reduced cost, i.e., which has the highest potential to improve the current solution by decreasing the objective value. This reduced cost is computed based on the objective function of the original problem and on the dual variables obtained from the simplex algorithm. Because of the large number of columns, iterating over all available columns to find the minimum reduced cost is not feasible. Instead, the problem of finding the column with the lowest reduced cost is expressed as its own optimization problem. This problem is called the Pricing Subproblem. A solution to the Pricing Subproblem indicates which column to add to the set for the next iteration.

If no column with negative reduced cost is found, this means that the optimal solution of the original LP has been found. The Column Generation procedure continues until either a predefined stopping criterion is reached, or until no column with negative reduced cost can be found.

5.2 Applying Column Generation to the EADARP

To apply Column Generation to the EADARP, we have to consider an ILP formulation of this problem with a large number of decision variables. Recall the Set Covering formulation of the EADARP, which was introduced in detail in Section 3.2.1. This formulation introduces a binary decision variable for every feasible route in the given instance. The number of possible routes in the instance graph, not considering feasibility,

is exponential in the size of the graph. Even restricting this to the set of feasible routes according to the constraints of the EADARP, this number remains too large to consider enumerating all feasible routes. As such, this formulation fits the requirements for Column Generation.

The rest of this section will explain how the Restricted Master Problem and the Pricing Subproblem are obtained from the Set Covering formulation.

5.2.1 Restricted Master Problem

Considering an EADARP instance with a set of vehicles K and a set of requests R , the Restricted Master Problem (RMP) is obtained from the Set Covering formulation by only considering the subset of routes that are currently known and by relaxing integrality constraints. If we denote $\Omega' = \bigcup_{k \in K} \Omega'_k$ the set of known routes, where Ω'_k is the set of routes designed for vehicle k , the RMP is defined as the following Linear Program:

$$\min \sum_{k \in K} \sum_{\omega \in \Omega'_k} c_{\omega} y_{k,\omega} \quad (5.1)$$

$$\text{s.t.} \quad \forall i \in R, \quad \sum_{k \in K} \sum_{\omega \in \Omega'_k} \theta_{i,\omega} y_{k,\omega} \geq 1 \quad (\lambda_i) \quad (5.2)$$

$$\forall k \in K, \quad \sum_{\omega \in \Omega'_k} y_{k,\omega} \leq 1 \quad (\kappa_k) \quad (5.3)$$

$$\forall k \in K, \forall \omega \in \Omega'_k, \quad y_{k,\omega} \geq 0 \quad (5.4)$$

The constraints are the same as in the Set Covering formulation from Section 3.2.1. Because we consider the Linear Relaxation of the original problem, the definition domain of variables $y_{k,\omega}$ only requires positivity. Still, combined with the constraints in Equation (5.3), this ensures that all decision variables stay below one, so that:

$$\forall k, \forall \omega, \quad 0 \leq y_{k,\omega} \leq 1$$

The variables defined in parentheses on the right side of each constraint are used to denote the dual variables associated with each constraint. In this case, we have one dual variable λ_i associated with each request $i \in R$ and one dual variable κ_k for each vehicle $k \in K$. These dual variables will intervene in the computation of the reduced cost and the formulation of the Pricing Subproblem.

The RMP is a restriction of the LP relaxation of the original formulation. This means that a feasible solution to the RMP is always feasible for the original LP, but an optimal solution to the RMP might not be optimal for the original LP, as only a subset of all possible solutions is considered in the RMP.

5.2.2 Pricing Subproblem

Assuming that the set of known routes Ω' is fixed and that the RMP gives an optimal solution y^* , then we also obtain the associated values for the dual variables λ^* and κ^* . We

will denote $\lambda^* = (\lambda_i)_{i \in R}$ and $\kappa^* = (\kappa_k)_{k \in K}$, so that given a request $i \in R$ (respectively a vehicle $k \in K$), the associated dual value obtained from an optimal solution to the RMP is λ_i (resp. κ_k).

The goal of the Pricing Subproblem is to find a route in $\Omega \setminus \Omega'$ which would improve the solution. As each route is bound to a specific vehicle k , we consider the Pricing Subproblem for each vehicle separately. Given a route $\omega \in \Omega_k \setminus \Omega'_k$ which is feasible for vehicle k , we can estimate how this route would impact the objective value of an optimal solution by looking at its reduced cost $\tilde{c}_\omega$, which is defined as follows:

$$\tilde{c}_\omega = c_\omega - \kappa_k - \sum_{i \in R} \theta_{i,\omega} \lambda_i \quad (5.5)$$

From this reduced cost, the Pricing Subproblem for vehicle k can be formulated as the following combinatorial optimization problem:

$$\min_{\omega \in \Omega_k \setminus \Omega'_k} \tilde{c}_\omega \quad (5.6)$$

Here again, this compact formulation hides away most of the complexity of this optimization problem, which lies in actually generating feasible routes with a good solution quality. Indeed, this formulation is not a linear program, as we are optimizing over the discrete set of routes $\Omega_k \setminus \Omega'_k$.

Alternatively, the Pricing Subproblem for vehicle k can be formulated as a variant of the Elementary Shortest Path Problem with Resource Constraints (ESPPRC) on a reduced graph $G_k \subseteq G$. The ESPPRC is an NP-hard problem aiming to find the shortest path in a graph that satisfies constraints defined over a set of resources. These resources vary along the path that is taken. We refer to Irnich and Desaulniers [23] for a classification of variants of the ESPPRC and a more general overview of this problem.

In the case of the Pricing Subproblem for the EADARP, we want to find a feasible path starting at v_k^{ini} that minimizes the reduced cost $\tilde{c}_\omega$. This problem is called *elementary* because we do not allow a pickup or dropoff location to be visited more than once. To model the different constraints of the EADARP regarding customer load, battery consumption and time windows, constrained resources are introduced in the ESPPRC. Our formulation differs from classical ESPPRC formulations because we do not have a pre-defined sink node where the path must end. Instead, we can consider any location where the vehicle is empty as a valid final location in a path.

As the Pricing Subproblem is formulated independently for each vehicle $k \in K$, we consider the subgraph $G_k = (V_k, A_k)$ extracted from the instance graph G and defined as follows. The set of nodes V_k of G_k is defined based on V as:

$$V_k = \{v_k^{\text{ini}}\} \cup P \cup D \cup S \quad (5.7)$$

Among the set of starting locations of vehicles, we only keep the starting location of vehicle k . All other location nodes (pickups, dropoffs, and charging stations) are kept from the original instance.

For the set of edges, we can further restrict allowed edges based on the constraints that define a feasible route. We define the set A_k as the union of the following edge sets:

- The initial vehicle location v_k^{ini} is connected to all pickup locations and all charging stations, but not to the dropoff locations. Indeed, the vehicle always starts empty, so there is no passenger to drop off on board.
- Every charging station $s \in S$ is connected only to the pickup locations. Indeed, a vehicle cannot have any passengers on board while charging, so the next location can never be a dropoff location. Furthermore, we forbid any edge directly linking two charging stations. This avoids any potentially infinite loop between charging stations. This decision is motivated by the assumption that there always exists a pickup location that is reachable from a given charging station for a vehicle with a full battery.
- A pickup location $v_i^\uparrow$ is connected to all other pickup and dropoff locations, but not to any charging station. A vehicle leaving $v_i^\uparrow$ always has at least the customer from request i on board, so it cannot go directly to a charging station.
- A dropoff location $v_i^\downarrow$ is connected to all other pickup, dropoff, and charging locations. A dropoff might be the end of a fragment, in which case the vehicle is allowed to drive to a charging station to recharge. Or it might be an intermediate stop in the fragment, in which case the vehicle might serve other requests at the next location in the route.

Formally, A_k is defined as:

$$\begin{aligned}
 A_k = & \{(v_k^{\text{ini}}, v_i^\uparrow) \mid i \in R\} \cup \{(v_k^{\text{ini}}, s) \mid s \in S\} \\
 & \cup \{(s, v_i^\uparrow) \mid s \in S, i \in R\} \\
 & \cup \{(v_i^\uparrow, v_j^\uparrow) \mid i \neq j \in R\} \cup \{(v_i^\uparrow, v_j^\downarrow) \mid i, j \in R\} \\
 & \cup \{(v_i^\downarrow, v_j^\uparrow) \mid i \neq j \in R\} \cup \{(v_i^\downarrow, v_j^\downarrow) \mid i \neq j \in R\} \cup \{(v_i^\downarrow, s) \mid i \in R, s \in S\}
 \end{aligned} \tag{5.8}$$

In this reduced graph $G_k = (V_k, A_k)$, the Pricing Subproblem has the objective of finding a feasible route that minimizes the reduced cost defined in Equation (5.5).

A feasible route is a path in G_k starting at v_k^{ini} that respects the operational constraints of the EADARP. The constraints relevant to defining the feasibility of a route are based on the constraints of the EADARP defined in Section 3.1.3. A feasible route has to satisfy the following constraints:

- The route starts empty at location v_k^{ini} . At its last location, the vehicle is empty.
- The route should remain battery- and load-feasible at all locations in the path.

- If a request i is picked up in the route, it has to be dropped off at a later point in the path.
- The route has to respect the maximum user ride time and pickup time windows of every request that it serves.
- Waiting and recharging are only allowed when no customers are on board. Furthermore, vehicles always charge to their full battery capacity.

5.3 Labeling Algorithm to Solve the Pricing Subproblem

A standard approach in the literature to solve variants of the ESPPRC consists of developing a labeling algorithm. In this technique, we progressively construct all feasible paths starting from the source node. Each location in the path gets an associated label, which contains values that are relevant to compute the state of each resource. A feasible path is extended along an edge in the graph by computing a new label for the next location in the path based on the current label. This is achieved by defining Resource Extension Functions (REFs) for each component of the label. To avoid an exponential growth of the set of feasible paths, domination criteria are introduced. They are used to compare two labels at a given location. If one of the labels cannot deliver any better solution than the other, then we can safely discard it from the list of labels to extend without cutting off any optimal solution. Each of these components of the labeling algorithm will be presented in the next sections.

To find the most profitable route for the Pricing Subproblem, we adapt the labeling algorithm from Desaulniers et al. [17] to fit the EADARP. As the EADARP only considers full recharges and does not impose any limit on the number of charging stations that are visited, we will base this labeling algorithm on the forward MF (Multiple Full recharges) variant.

5.3.1 Label Definition

Given a location $u \in V_k$ in the reduced graph G_k , we define the label L_u as:

$$L_u = \{\text{load}_u, \text{batt}_u, \text{time}_u, \text{f_time}_u, \text{cost}_u, \text{late}_u, \text{f_late}_u, \text{pick}_u, \text{serv}_u, \text{f_reqs}_u, \text{drive}_u, \text{pred}_u\} \quad (5.9)$$

The label components are defined as follows:

- load_u is the load of the vehicle after processing the current location.
- batt_u is the state of charge of the vehicle when entering location u . From this we can compute the recharging time needed at the next charging stop to get back to a full battery.

- time_u is the earliest possible arrival time at the current location. For pickup locations, this is the time when the vehicle actually picks up a customer, including some potential waiting time. For charging stations, the arrival time is considered to be the time when the vehicle starts recharging.
- f_time_u is the earliest possible arrival time at the first location in the current fragment. This value can be modified within a fragment because of additional waiting time that is pushed back to the start of the fragment.
- cost_u is the static reduced cost along the path, not including the penalty for lateness. This reduced cost is composed of the travel costs along the path, as well as the costs incurred through dual variables from picking up requests.
- late_u is the accumulated lateness along the path up to the last finished fragment. This value is only updated at the end of a fragment, when the lateness in this fragment is fixed.
- f_late_u is the minimum accumulated lateness in the current fragment. This value may change whenever waiting is added at a pickup location.
- pick_u is the set of requests that have been picked up along the path (but may still need to be dropped off).
- serv_u is the set of requests that have been fully served along the path (i.e. both picked up and delivered).
- f_reqs_u is the set of requests that have been visited within the current fragment. We need to keep track of these requests to be able to recompute the incurred lateness when some waiting time is added.
- drive_u is a mapping keeping track of the direct travel time from the start of the fragment to any pickup location in the current fragment. This travel time does not include any waiting or charging time. Whenever a request i is picked up at the location $v_i^\uparrow$ with a label $L_{v_i^\uparrow}$, we can compute the associated direct travel time $\text{drive}(i)$ as follows:

$$\text{drive}(i) = \text{time}_{v_i^\uparrow} - \text{f_time}_{v_i^\uparrow}$$

This value can be used to compute the time spent by each customer on board of the vehicle and the lateness created by some waiting later in the fragment.

- pred_u is a reference to the predecessor of L_u . The predecessor is the label which was extended to obtain L_u . The initial label is the only one which does not have a predecessor. We use this reference when trying to construct the actual route based on a given label.

Having defined the label, we can initialize it at location v_k^{ini} by setting all components to 0 (or $\emptyset$ for the sets), besides some components which have a relevant initial value. $\text{cost}_{v_k^{\text{ini}}}$

is initialized to $-\kappa_k$, the dual value associated with vehicle k . $\text{time}_{v_k^{\text{ini}}}$ is initialized to t_k^{avail} , the time at which vehicle k becomes available. Finally, $\text{batt}_{v_k^{\text{ini}}}$ is initialized to $B_{k,0}$, the battery level of vehicle k when it becomes available. As such, the initial label at location v_k^{ini} is:

$$L_{v_k^{\text{ini}}} = \{0, B_{k,0}, t_k^{\text{avail}}, 0, -\kappa_k, 0, 0, \emptyset, \emptyset, \emptyset, \emptyset\} \quad (5.10)$$

5.3.2 Label Extension

Now, we can define the Resource Extension Functions (REFs), which are used to update the state of the label when driving along an edge. Starting from a label L_u at location u , we want to extend it by creating a new label L_v in a neighboring location v such that $(u, v) \in A_k$.

In the special case where v is a pickup location, we may have some additional waiting time wait_v . In that case, there exists a request $i \in R$ such that $v = v_i^\uparrow$. We can compute the waiting time based on the current label L_u as follows:

$$\text{wait}_v = \max\{0, t_i^{\text{start}} - (\text{time}_u + d_{u,v} + \begin{cases} (Q - \text{batt}_u)/\alpha & \text{if } u \in S \\ 0 & \text{otherwise} \end{cases})\}$$

Based on the current label L_u , we can update the label components of L_v . If location v is a pickup or a dropoff location, we denote $i \in R$ the index of the associated request, such that $v = v_i^\uparrow$ or $v = v_i^\downarrow$.

In Equations (5.11) to (5.22), the extension for each label component is described, with a few sentences explaining the idea behind the mathematical formulas.

$$\text{load}_v = \begin{cases} \text{load}_u + l_i & \text{if } v \in P \\ \text{load}_u - l_i & \text{if } v \in D \\ \text{load}_u & \text{otherwise} \end{cases} \quad (5.11)$$

The customer load is updated whenever the vehicle enters a pickup or dropoff location by adding or subtracting the load from the associated request.

$$\text{batt}_v = \begin{cases} Q - b_{u,v} & \text{if } u \in S \\ \text{batt}_u - b_{u,v} & \text{otherwise} \end{cases} \quad (5.12)$$

The state of charge of the vehicle is updated based on the battery $b_{u,v}$ consumed to drive from u to v . If location u was a charging station, the vehicle departs from u after a full recharge, i.e. with a battery level of Q .

$$\text{time}_v = \begin{cases} \text{time}_u + d_{u,v} + \text{wait}_v + (Q - \text{batt}_u)/\alpha & \text{if } v \in P \wedge u \in S \\ \text{time}_u + d_{u,v} + \text{wait}_v & \text{if } v \in P \wedge u \notin S \\ \text{time}_u + d_{u,v} + (Q - \text{batt}_u)/\alpha & \text{if } v \notin P \wedge u \in S \\ \text{time}_u + d_{u,v} & \text{otherwise} \end{cases} \quad (5.13)$$

In the most general case, the arrival time is updated by adding the driving time from location u to location v . If u is a charging station, we also have to add the charging time that is needed to recharge the vehicle. If v is a pickup location, we have to account for some potential waiting time in the case where the vehicle arrives at the location before the start of the time window at v . Note that, in Equation (5.13), the third condition $v \notin P \wedge u \in S$ cannot actually happen in our case, as the only outgoing edges that are allowed from a charging station in G_k lead to a pickup location. We have left it in the label extension for completeness sake, as the edge restrictions are specific to our modelisation.

$$f_time_v = \begin{cases} time_v & \text{if } load_u = 0 \\ f_time_u + wait_v & \text{if } v \in P \wedge load_u \neq 0 \\ f_time_u & \text{otherwise} \end{cases} \quad (5.14)$$

The starting time of the current fragment is reset whenever a new fragment is started, that is whenever $load_u = 0$. Otherwise, this component is only updated at pickup locations internal to the current fragment, where some additional waiting time might have to be pushed back to the start of the fragment. Indeed, if a vehicle arrives too early at a pickup location v that is not the first location in a fragment, the vehicle is not allowed to wait at this location because of the constraints listed in Section 5.2.2. As such, the waiting time that would be necessary at v is pushed back before the start of the current fragment, where waiting is allowed because the vehicle is empty. This additional waiting time might induce some lateness in pickup locations that were already visited, which is handled by Equation (5.20).

$$pick_v = \begin{cases} pick_u \cup \{i\} & \text{if } v \in P \\ pick_u & \text{otherwise} \end{cases} \quad (5.15)$$

$$serv_v = \begin{cases} serv_u \cup \{i\} & \text{if } v \in D \wedge i \in pick_u \\ serv_u & \text{otherwise} \end{cases} \quad (5.16)$$

The set of picked up requests (respectively of served requests) is updated when entering a pickup (resp. a dropoff) location. The index of the newly processed request is added to the set. Note that $pick_v$ contains all requests that have been picked up at any point in the route until location v is reached, not just those whose customers are currently on board of the vehicle.

$$f_reqs_v = \begin{cases} \{i\} & \text{if } load_u = 0 \wedge v \in P \\ \emptyset & \text{if } load_u = 0 \wedge v \notin P \\ f_reqs_u \cup \{i\} & \text{if } load_u \neq 0 \wedge v \in P \\ f_reqs_u & \text{otherwise} \end{cases} \quad (5.17)$$

The set f_reqs_v keeps track of which requests are served by the current fragment. This information is needed to be able to correctly compute the lateness induced by any waiting time added in the fragment. This set is reinitialized at the start of a new fragment, i.e. when $load_u = 0$ and updated with the index of a new request whenever a pickup location is entered.

$$drive_v = \begin{cases} \{(i \mapsto 0)\} & \text{if } load_u = 0 \wedge v \in P \\ \emptyset & \text{if } load_u = 0 \wedge v \notin P \\ drive_u \cup \{(i \mapsto time_v - f_time_v)\} & \text{if } load_u \neq 0 \wedge v \in P \\ drive_u & \text{otherwise} \end{cases} \quad (5.18)$$

The direct driving time within the fragment is also reset at the start of a new fragment. Within a fragment, it is updated whenever a pickup location is entered by computing the driving time from the start of the fragment to this location. This is done by computing $time_v - f_time_v$. Indeed, f_time_v is updated with any waiting time that has been added within the fragment, and recharging is not allowed within a fragment, so the vehicle is always driving within a fragment. For the case $load_u = 0 \wedge v \in P$, i.e. for the pickup location that is the starting location of the fragment, the value of $time_v - f_time_v$ simplifies to 0 because of the first case in Equation (5.14).

$$cost_v = \begin{cases} cost_u + d_{u,v} - \lambda_i & \text{if } v \in P \\ cost_u + d_{u,v} & \text{otherwise} \end{cases} \quad (5.19)$$

The static cost is updated with the travel cost from u to v . In the case where v is a pickup location, the dual cost of the associated request is also taken into account.

$$f_late_v = \begin{cases} 0 & \text{if } load_v = 0 \\ \sum_{r \in f_reqs_v} \max\{0, f_time_v + drive_v[r] - t_r^{end}\} & \text{if } v \in P \\ f_late_u & \text{otherwise} \end{cases} \quad (5.20)$$

The value for f_late_v is reset at the last location in a fragment, i.e. when the condition $load_v = 0$ is satisfied. The lateness in the current fragment is recomputed whenever a pickup location is entered by using the updated value of f_time_v . It is updated by summing up the individual lateness at each of the pickup locations traversed in the current fragment (which are stored in f_reqs_v).

$$late_v = \begin{cases} late_u + f_late_u & \text{if } load_v = 0 \\ late_u & \text{otherwise} \end{cases} \quad (5.21)$$

The lateness within a fragment is only fixed after entering the last location in this fragment. This is verified by the condition $load_v = 0$. At this location, the accumulated

lateness is updated by adding the lateness from the current fragment, which was computed in f_late_u . Note that we use the value taken from the current label L_u , because f_late_v has already been reset in Equation (5.20). This remains correct because the last location in a fragment cannot be a pickup (otherwise we would still have to drop the request off), so there cannot be additional lateness created at the last location in a fragment.

$$pred_v = L_u \quad (5.22)$$

Finally, we set L_u as the predecessor of the new label L_v .

5.3.3 Feasibility Conditions

As we want the transitions in the graph G_k to be valid with regard to the constraints listed in Section 5.2.2, the extension from L_u to L_v needs to respect some feasibility conditions. Should any of the feasibility conditions be unsatisfied, the label extension is aborted, and the new label L_v is discarded.

A feasible label L_v needs to respect the load and capacity limitations of the vehicle, and the vehicle should always be empty at a charging stop. This can be verified by checking that the following conditions hold:

$$0 \leq load_v \leq C, \quad 0 \leq batt_v \leq Q, \quad (5.23)$$

$$v \in S \implies load_v = 0 \quad (5.24)$$

We also have to ensure that the maximum user ride time is respected, by verifying the following condition:

$$\forall r \in f_reqs_u \setminus serv_u, \quad time_v - (f_time_v + drive_v[r]) \leq d_r^{\max} \quad (5.25)$$

Here, $f_reqs_u \setminus serv_u$ is the set of customers that were on board of the vehicle when entering location v , including any potential customer that would be dropped off in location v if $v \in D$. This ensures that the maximum user ride time is also respected for a customer that is dropped off at location v . Also note that $f_time_v + drive_v[r]$ is the actual pickup time for request r , taking into account all waiting times that may have been added in the fragment.

For the pickup and dropoff locations, we also have to check that no location is visited multiple times by verifying that the following conditions hold. Once again, we assume that the request associated with the pickup or the dropoff has index $i \in R$.

$$v \in P \implies (i \notin pick_u \wedge i \notin serv_u), \quad (5.26)$$

$$v \in D \implies (i \in pick_u \wedge i \notin serv_u) \quad (5.27)$$

The label extension from L_u to L_v is considered feasible if and only if all of the conditions in Equations (5.23) to (5.27) are satisfied.

5.3.4 Dominance Criterion

Dominance is a notion used in labeling algorithms to detect labels that are redundant and can be safely discarded from the algorithm. A dominance criterion is used to find labels that can never lead to a solution with minimum reduced cost. Such labels can be safely discarded without impacting the quality of the final solution. Using a dominance criterion can drastically reduce the number of labels that need to be extended to find the optimal solution [17].

In practice, a dominance check is done between two labels at the same location to determine whether one of the labels makes the other redundant. If one of the labels is strictly less efficient than the other one, it can be discarded safely. Considering two labels L^1 and L^2 both at the same location $u \in V_k$, L^1 is dominated by L^2 if any feasible extension from L^1 that subsequently leads to a feasible final solution can be mirrored starting from L^2 and the mirrored extension leads to a final solution that is at least as good as the one from L^1 .

In general, a dominance criterion can be applied at any location to compare labels at this location. For the Labeling Algorithm that is presented here, the fact that the lateness of a route can change within a fragment depending on locations in the fragment which have not yet been reached makes the definition of a strong dominance criterion valid at any location quite complex. Instead, we will only consider dominance for labels linked to locations where the lateness is fixed, that is, at the first location in a fragment. In this case, many label components are fixed, so that we can more easily determine whether a label is dominated.

This dominance criterion is admittedly weaker than a general dominance criterion applicable to any pair of labels. Experimental results show that, on small instances where the labeling algorithm terminates, it is enough to discard between 30% and 50% of the generated labels. These results are discussed more in depth in the paragraph relating to the CMSA-CG approach in Section 7.4.3.

The start of a fragment can be identified by the fact that the vehicle is empty when entering the first location in a fragment. Also, fragments always start with either a charging station or a pickup location. A dropoff location cannot be the first location in a fragment, because this would mean having picked up the associated customer at some location earlier in the fragment. As such, we can identify whether a label is the first in a fragment by looking at the two possible cases:

- Labels at a charging station are always the first (and only) label in a fragment.
- For a pickup location $u \in P$ associated with request $i \in R$, such that $u = v_i^\uparrow$, a label L_u at location u is the start of a fragment if and only if $\text{load}_u = l_i$. This condition ensures that the vehicle was empty before picking up customer i .

In the rest of this section, we consider two labels L_u^1 and L_u^2 at a location $u \in P \cup S$ and will express the dominance criterion to determine if L_u^1 is dominated by L_u^2 . The

label components for L_u^1 (resp. L_u^2) are denoted with the exponent $\cdot^1$ (resp. with the exponent $\cdot^2$). For example, load_u^1 denotes the load component for L_u^1 .

Dominance at a charging station

Here we have $u \in S$. We also know from the REFs in Equations (5.11) to (5.21) that the following holds for both labels:

$$\begin{aligned} \text{load}_u^1 &= \text{load}_u^2 = 0 \\ \text{f_time}_u^1 &= \text{time}_u^1 & \text{f_time}_u^2 &= \text{time}_u^2 \\ \text{f_late}_u^1 &= \text{f_late}_u^2 = 0 \\ \text{pick}_u^1 &= \text{serv}_u^1 & \text{pick}_u^2 &= \text{serv}_u^2 \\ \text{f_reqs}_u^1 &= \text{f_reqs}_u^2 = \emptyset \\ \text{drive}_u^1 &= \text{drive}_u^2 = \emptyset \end{aligned}$$

In this case, L_u^1 is dominated by L_u^2 if *all of the following* conditions hold simultaneously:

$$\text{time}_u^1 + (Q - \text{batt}_u^1)/\alpha \geq \text{time}_u^2 + (Q - \text{batt}_u^2)/\alpha \quad (5.28)$$

$$\text{cost}_u^1 + \rho \cdot \text{late}_u^1 \geq \text{cost}_u^2 + \rho \cdot \text{late}_u^2 \quad (5.29)$$

$$\text{serv}_u^1 \supseteq \text{serv}_u^2 \quad (5.30)$$

Let us explain how each of these conditions arise. For Equations (5.28) to (5.30) to be a valid dominance criterion, we have to make sure that the dominance does not cut off any label that could lead to an optimal solution. To do this, we have to verify that any feasible label extension of L_u^1 can be mirrored by starting from L_u^2 instead and that this yields solutions that are at least as good.

Consider the state of the solution in label L_u^1 . Because u is a charging station, the next location in any feasible extension of L_u^1 is a pickup location v that verifies $v \in P \setminus \text{serv}_u^1$. Furthermore, the vehicle can leave u at the earliest after having fully recharged, at time $\text{time}_u^1 + (Q - \text{batt}_u^1)/\alpha$.

If label L_u^2 verifies Equations (5.28) and (5.30), then any feasible extension from L_u^1 can be mirrored by starting from L_u^2 instead. Indeed, Equation (5.30) gives $P \setminus \text{serv}_u^1 \subseteq P \setminus \text{serv}_u^2$, so going to location v is also feasible for L_u^2 . And because of Equation (5.28), we know that the vehicle can leave u from L_u^2 earlier than from L_u^1 , so it can just add some waiting time to mirror the departure time. Because we are at a charging station, this additional waiting time cannot create more lateness in the next fragment than would be created starting from L_u^1 .

Finally, because of Equation (5.29), we know that L_u^1 cannot achieve a better final objective value than L_u^2 . Because we are at the start of a fragment, all lateness from previous fragments has been fixed and will not change anymore. Any decision taken from

label L_u^1 can be mirrored in L_u^2 with the same impact in both labels on cost_u and late_u . As such, the gap in total cost that exists at location u will persist for the rest of the path, which is why any label extended from L_u^1 cannot become better than the label obtained by the same extensions starting from L_u^2 .

Dominance at a pickup location

We assume that $u \in P$ and that the associated request has index $i \in R$, such that $u = v_i^\uparrow$. Both L_u^1 and L_u^2 are the first label in a fragment if we have $\text{load}_u^1 = \text{load}_u^2 = l_i$.

From the REFs in Equations (5.11) to (5.21), we also know that the following holds for both labels:

$$\begin{aligned} \text{f_time}_u^1 &= \text{time}_u^1 & \text{f_time}_u^2 &= \text{time}_u^2 \\ \text{pick}_u^1 &= \text{serv}_u^1 \cup \{i\} & \text{pick}_u^2 &= \text{serv}_u^2 \cup \{i\} \\ \text{f_reqs}_u^1 &= \text{f_reqs}_u^2 = \{i\} \\ \text{drive}_u^1 &= \text{drive}_u^2 = \{(i \mapsto 0)\} \end{aligned}$$

Compared to the case above for charging stations, note that f_late_u does not have a fixed value. Indeed, it might be that the vehicle is already late at the first location in a fragment. Still, we can compute the exact value of f_late_u for both labels as follows (for $m \in \{1, 2\}$):

$$\begin{aligned} \text{f_late}_u^m &= \sum_{r \in \text{f_reqs}_u^m} \max\{0, \text{f_time}_u^m + \text{drive}_u^m[r] - t_r^{\text{end}}\} \\ &= \sum_{r \in \{i\}} \max\{0, \text{time}_u^m + \text{drive}_u^m[r] - t_r^{\text{end}}\} \\ &= \max\{0, \text{time}_u^m + \text{drive}_u^m[i] - t_i^{\text{end}}\} \\ &= \max\{0, \text{time}_u^m - t_i^{\text{end}}\} \end{aligned}$$

Having computed the value for f_late_u , we can note that it only depends on time_u and that it is an increasing function of time_u .

In this case, L_u^1 is dominated by L_u^2 if *all of the following* conditions hold simultaneously:

$$\text{batt}_u^1 \leq \text{batt}_u^2 \tag{5.31}$$

$$\text{time}_u^1 \geq \text{time}_u^2 \tag{5.32}$$

$$\text{cost}_u^1 + \rho \cdot (\text{late}_u^1 + \text{f_late}_u^1) \geq \text{cost}_u^2 + \rho \cdot (\text{late}_u^2 + \text{f_late}_u^1) \tag{5.33}$$

$$\text{serv}_u^1 \supseteq \text{serv}_u^2 \tag{5.34}$$

Here again, we have to ensure that every decision taken from L_u^1 can be mirrored in L_u^2 with solutions that are at least as good.

Equation (5.34) guarantees that all valid next locations for L_u^1 are also valid for L_u^2 , regardless of battery and time considerations. Finally, Equations (5.31) and (5.32) ensure that the battery and time constraints are respected when mirroring decisions from L_u^1 . L_u^2 has at least as much battery as L_u^1 , so any location reachable from L_u^1 is also reachable from L_u^2 . For the label component time_u , we can add some waiting time in L_u^2 to replicate any decision taken from L_u^1 . This additional waiting time can have an impact on the objective value of L_u^2 , but we can compute an upper bound on how much the objective would increase. Indeed, in the worst case we would depart from u in L_u^2 at time time_u^1 (to mirror a decision taken in L_u^1). In that case, we know that the lateness in the current fragment is equal to f_late_u^1 . As such, the objective value of L_u^2 after the waiting would become $\text{cost}_u^2 + \rho \cdot (\text{late}_u^2 + \text{f_late}_u^1)$, which is exactly the value that was chosen for the right-hand side of Equation (5.33). This guarantees that solutions given by L_u^2 are at least as good as those given by L_u^1 .

This reasoning on the actual lateness in a fragment would not be possible when comparing locations that are internal to a fragment, because any additional waiting within a fragment needs to be pushed back to the start of the fragment and may generate additional lateness, which could have different effects on the labels in L_u^1 and L_u^2 .

5.3.5 Overview of the Labeling Algorithm

The Pricing Subproblem tries to find the feasible route with the lowest negative reduced cost. The full procedure to execute the labeling algorithm is described in pseudo-code in Algorithm 5.1. The labeling algorithm starts from the initial label $L_{v_k^{\text{ini}}}$ and extends new feasible labels until all feasible extensions have been reached. To reduce the number of labels that are actually considered without losing solution quality, each new label is compared for dominance with the other labels at the same location. As the dominance criterion in Section 5.3.4 is only defined for fragment starts, the function `DOMINATES()` that is called at lines 14 and 17 returns false at any other locations. Then, the labeling algorithm has to extract all feasible routes from the labels available at the end of the extension. To achieve this, the labeling algorithm identifies which labels represent a valid feasible route. Such labels will be called final labels.

Based on the definition of fragments that was introduced in Section 3.1.4, only those labels which correspond to the end of a fragment are valid final labels. This can be detected in the labeling algorithm at the moment where a new label is generated by extending an existing label. If the new label is feasible, we can verify whether it is a final label and mark it as such for easy lookup.

Once all possible extensions have been executed, the set of final labels is considered. Given a final label L_u at some location $u \in V_k$, we can compute the reduced cost $\tilde{c}_\omega$ of the route ω corresponding to L_u in constant time as follows:

$$\tilde{c}_\omega = \text{cost}_u + \rho \cdot \text{late}_u \quad (5.35)$$

Algorithm 5.1 Generate all non-dominated labels

Input: a graph G_k and the dual values κ, λ from the RMP
Return: all feasible non-dominated labels at every location in the instance

```
1:  $labels : \text{Dict}[\text{Location}, \text{Array}[\text{Label}]] \leftarrow \emptyset$ 
2:  $L_{v_k^{\text{ini}}} \leftarrow \{0, B_{k,0}, t_k^{\text{avail}}, 0, -\kappa_k, 0, 0, \emptyset, \emptyset, \emptyset, \emptyset\}$   $\triangleright$  Initial label
3:  $stack \leftarrow \{L_{v_k^{\text{ini}}}\}$ 
4:  $labels[v_k^{\text{ini}}] \leftarrow \{L_{v_k^{\text{ini}}}\}$ 
5:  $final\_labels \leftarrow \emptyset$ 
6: while  $stack \neq \emptyset$  do
7:    $L_u \leftarrow \text{POPFIRST}(stack)$ 
8:   for all  $v \in \text{NEIGHBORS}(u)$  do
9:      $L_v \leftarrow \text{EXTENDLABEL}(L_u, u, v)$ 
10:    if  $L_v$  not feasible then  $\triangleright$  Check feasibility
11:      continue
12:    dom_flag  $\leftarrow$  false  $\triangleright$  Check dominance with other labels at v
13:    for all  $L'_v \in labels[v]$  do
14:      if  $\text{DOMINATES}(L_v, L'_v)$  then
15:         $\text{REMOVE}(labels[v], L'_v)$ 
16:         $\text{REMOVE}(stack, L'_v)$ 
17:      else if  $\text{DOMINATES}(L'_v, L_v)$  then
18:        dom_flag  $\leftarrow$  true
19:      break
20:    if dom_flag = false then  $\triangleright$  Add new label
21:       $\text{APPEND}(labels[v], L_v)$ 
22:       $\text{APPEND}(stack, L_v)$ 
23:      if load $_v = 0$  then  $\triangleright$   $L_v$  is a final label
24:         $\text{APPEND}(final\_labels, L_v)$ 
25: return  $labels, final\_labels$ 
```

Let us prove that the right-hand side of Equation (5.35) is exactly equal to the actual reduced cost $\tilde{c}_\omega$ that was defined in Equation (5.5). The REFs for cost_u and late_u are additive. The term $-\kappa_k$ in the definition of $\tilde{c}_\omega$ was added in the initialization of the labeling algorithm in $\text{cost}_{v_k^{\text{ini}}}$. The driving times are computed in the REF for cost_u in Equation (5.19). In the first case of this equation, we also take into account the dual values λ_i for every request that is served by ω . Finally, the lateness is updated at the end of a fragment, when no additional waiting time can be added to the fragment, as shown in Equation (5.21). To obtain the correct coefficient in the actual cost of ω , we simply have to multiply late_u by the penalty factor ρ . As such, the value obtained by computing $\text{cost}_u + \rho \cdot \text{late}_u$ is equal to the reduced cost of the associated route $\tilde{c}_\omega$.

From this, we can iterate over all final labels to find the one with the lowest negative reduced cost. This procedure is described in Algorithm 5.2. To speed up the Column Generation algorithm, we might want to add more than one column to the RMP. In that case, we can extract all labels with a negative reduced cost, instead of only considering the minimum cost label.

Algorithm 5.2 Extract the route with minimum reduced cost

Input: the *labels* obtained by running the labeling algorithm and a list of all *final_labels*
Return: the route yielding the minimum negative reduced cost

```

1: best_cost  $\leftarrow 0$ 
2: best_route  $\leftarrow \emptyset$ 
3: for all  $L_u \in \text{final\_labels}$  do
4:    $\text{cost}_L \leftarrow \text{cost}_u + \rho \cdot \text{late}_u$ 
5:   if  $\text{cost}_L < \text{best\_cost}$  then
6:      $\text{best\_cost} \leftarrow \text{cost}_L$ 
7:      $\text{best\_route} \leftarrow \text{CONSTRUCTROUTE}(L_u, \text{labels})$ 
8:  $\triangleright$  If  $\text{best\_cost} = 0$ , then no improving route was found and  $\text{best\_route} = \emptyset$ 
9: return best_route
    
```

Having found the columns that are to be added to the RMP, we can extract the corresponding routes by applying the procedure described in Algorithm 5.3. This procedure runs a backwards pass on the labels to find all the locations that are traversed by the route. The predecessor of a given label is obtained at line 7 by following the reference stored in pred_u . As all labels were obtained starting from the initial label, we move up in the hierarchy of labels until reaching $L_{v_k^{\text{ini}}}$. Afterwards, the algorithm builds the concrete route based on a forward pass on these labels, using the label components to compute the arrival time at each location.

A naive approach would iterate over all stops and use the value of time_u as the time of arrival of the vehicle at each location along the route. This would not correctly account for waiting times within a fragment.

Algorithm 5.3 Construct a route based on the associated final label

Input: a final label L_u and the set of explored labels $labels$
Return: the route corresponding to L_u

```

1:  $stops \leftarrow []$ 
2:  $frag\_times \leftarrow []$ 
3: while  $u \neq v_k^{ini}$  do
4:    $PUSH(stops, L_u)$ 
5:   if  $load_u = 0$  then  $\triangleright$  Last location in a fragment
6:      $PUSH(frag\_times, f\_time_u)$   $\triangleright$  Store starting time for current fragment
7:      $L_u \leftarrow GET(labels, pred_u)$ 
8:    $REVERSE(stops)$   $\triangleright$  Consider stops in ascending order
9:    $REVERSE(frag\_times)$ 
10:   $\omega \leftarrow [(v_k^{ini} \mapsto t_k^{avail})]$ 
11:   $cnt_f \leftarrow 0$   $\triangleright$  Counter to keep track of fragments
12:   $t \leftarrow t_k^{avail}$   $\triangleright$  Time at the current location
13:   $v \leftarrow v_k^{ini}$   $\triangleright$  Previous location
14:  for all  $L_u \in stops$  do
15:    if  $u \in S$  then
16:       $PUSH(\omega, (u \mapsto frag\_times[cnt_f]))$ 
17:       $cnt_f \leftarrow cnt_f + 1$ 
18:    else
19:      if  $u \in P \wedge u = v_i^\uparrow \wedge load_u = l_i$  then  $\triangleright$  Start of a fragment
20:         $t \leftarrow frag\_times[cnt_f]$ 
21:         $cnt_f \leftarrow cnt_f + 1$ 
22:      else  $\triangleright$  Internal stop in a fragment
23:         $t \leftarrow t + d_{v,u}$ 
24:         $PUSH(\omega, (u \mapsto t))$ 
25:         $v \leftarrow u$ 
26:  return  $\omega$ 

```

Considering a fragment composed of pickup and dropoff stops, only the last location in the fragment knows the actual time at which the fragment will start. Indeed, locations earlier in the fragment do not know about potential waiting times that will be added at later pickup locations. This is why we keep track of the starting time of each fragment in the backward pass of the algorithm at line 6 in Algorithm 5.3.

Then, in the forward pass that builds the actual route, we recompute the time of arrival of the vehicle at each location in the route based on the time of arrival at the first location in the fragment which was stored above and on the driving times between consecutive locations in the route. Within a fragment, we know that the vehicle cannot do anything else than drive between locations, because waiting is only allowed when the vehicle is empty.

5.4 Integrating Column Generation into CMSA

Having defined the Column Generation procedure for the EADARP, we now want to use this method in combination with the CMSA framework described in Chapter 4. The principle is to embed Column Generation as a subroutine into the Solve step of CMSA. This hybridization should give us the best of both worlds by exploiting the exact insights from Column Generation to guide the CMSA heuristic into more profitable directions. This approach is inspired by the work of Akbay et al. [3], who have developed a similar approach for the Variable Sized Bin Packing Problem.

To develop this hybrid approach, we will exploit the similarities between the Solve step of CMSA discussed in Section 4.2.3 and the Restricted Master Problem presented in Section 5.2.1. By taking the CMSA pool of components as the set of known routes in the RMP, Column Generation can propose new routes to add to the pool based on the current state of the algorithm.

5.4.1 Updated Solve Step

To integrate Column Generation into the CMSA heuristic, we update the Solve step described at line 12 in Algorithm 4.1 by adding an intermediate step where the Restricted Master Problem and the Pricing Subproblem are solved to obtain new routes to merge into the CMSA pool. The updated procedure is described in Algorithm 5.4. Note that at line 3, the labeling algorithm returns a set of routes, instead of only returning the route with the lowest negative reduced cost. This is an adaptation of Algorithm 5.2 to populate the CMSA pool with more routes and will be discussed in more detail later in this section.

We start by solving the RMP to obtain the current value of the dual variables $(\lambda_i)_{i \in R}$ and $(\kappa_k)_{k \in K}$ described in Equations (5.2) and (5.3). We have to recompute these values at each CMSA iteration, because they depend on the set of known routes Ω' , which is updated in the Merge and Adapt steps whenever routes are added to or removed from the pool.

Algorithm 5.4 Solve step for the hybrid CMSA-CG algorithm

Input: an EADARP instance $inst$, the current state of the CMSA algorithm: $pool$ and $incumbent$

- 1: $duals \leftarrow \text{SOLVERMP}(pool)$ $\triangleright$ Obtain dual variables
- 2: **for all** $k \in K$ **do**
- 3: **for all** $r_k \in \text{RUNLABELING}(inst, k, duals)$ **do** $\triangleright$ Solve Pricing Subproblem
- 4: | $pool \leftarrow pool \cup \{r_k\}$
- 5: $new_sol \leftarrow \text{SOLVEMASTERILP}(inst, pool)$ $\triangleright$ Improve incumbent
- 6: **if** new_sol is better than $incumbent$ **then**
- 7: $incumbent \leftarrow new_sol$

Having computed the new dual values, we generate new routes by applying the labeling algorithm presented in Section 5.3. The labeling algorithm is executed separately for each vehicle, because the Pricing Subproblem aims to find the best route for a given vehicle.

As exact Column Generation can be too expensive, we propose a heuristic version of the labeling algorithm that introduces a bound on the runtime of the labeling algorithm but loses the optimality guarantee of the exact procedure. These adjustments to the labeling algorithm are presented in detail in Section 5.4.2.

In contrast to the standalone Column Generation procedure whose objective is to generate the optimal solution to the Pricing Subproblem, the goal of the Labeling Algorithm within the CMSA framework is to enrich the pool with new routes to improve the incumbent, without any guarantee on optimality. Thus, adding more than one route at each call of the labeling algorithm can offer more possibilities in the Solve step. Instead of extracting only the best route in Algorithm 5.2, we can collect an arbitrary number of routes or all routes with a negative reduced cost and add them to the pool. We are assured that this will not make the CMSA pool grow too large, because the Adapt step always makes sure that the pool size remains within the specified limits.

The rest of the Solve step remains unchanged. Once the routes proposed by Column Generation have been added to the pool, the restricted Set Covering formulation is solved based on the current state of the pool to improve the incumbent.

5.4.2 Heuristic Labeling Algorithm

When running the labeling algorithm as a subroutine in a CMSA framework, we want to ensure that the time spent solving the Pricing Subproblem remains balanced with the other steps of CMSA. This can be achieved by introducing a bound on the runtime of the labeling algorithm. In the while loop at line 6 in Algorithm 5.1, we can add a condition that breaks out from the loop early if the given time bound is exceeded.

This timeout interrupts the labeling algorithm early and looks at the subset of final

labels that have been generated up to this point to find routes with a negative reduced cost. To avoid cutting away too many promising routes, we can adapt the order in which labels are considered for extension. In Algorithm 5.1, labels are extended in chronological order, starting from the oldest labels that were added to the stack. This is similar to a Breadth-First Search exploration in the graph G_k . Instead, we can influence the search to find routes with a negative reduced cost faster by considering the labels according to their partial reduced cost, extending labels that have already attained a lower reduced cost first in the hope that they will lead to final labels that have a negative reduced cost. This is achieved by changing the underlying data structure for the set of unextended labels from a stack to a priority queue that sorts the labels according to their partial reduced cost.

These practical adjustments to the labeling algorithm focus on keeping the runtime of the Column Generation procedure bounded while still exploring the most promising labels first to generate routes with a negative reduced cost, even if these routes do not reach the optimal reduced cost.

5.4.3 Dual Repair Operator

Aside from integrating Column Generation directly into the CMSA framework, we can also use the concepts of CG to develop a new repair operator. We want to use the values of the dual variables $(\lambda_i)_{i \in R}$ to guide the construction heuristic towards promising solutions, without needing to compute the exact solutions of the Pricing Subproblem.

Recall the repair operators that were described in Section 4.2.1. The random and chronological repair operators are based on a sorting function that defines the order in which unserved requests are inserted into a partial solution. Recall now that the Restricted Master Problem gives solutions to the LP relaxation of the Set Covering formulation, along with the values of the dual variables based on the current state of the CMSA pool. The higher the dual value λ_i of a given request $i \in R$, the more potential there is for improvement for this request. As such, we can use the dual values $(\lambda_i)_{i \in R}$ as an ordering function to insert the most promising requests first.

Formally, we define the dual repair operator using the same scheme as described in Algorithm 4.2. The ordering function *order* sorts the requests in descending order of their dual value λ_i . To add more randomization to this operator, we can shuffle all requests that have $\lambda_i = 0$.

To be able to use the dual repair operator, we need to have access to the current value of the dual variables $(\lambda_i)_{i \in R}$. In the case of the CMSA-CG hybrid algorithm, these dual values are already computed when solving the RMP. To be able to use the dual repair even in the standalone CMSA algorithm, we can add a step to solve the RMP in the Solve step of CMSA. The additional computational cost of solving the RMP is expected to be negligible, because solving LPs is easier and faster than using Branch-and-Cut to solve the Set Covering ILP, and because the formulation has already been loaded in memory

to solve the restricted Set Covering formulation, needing only minimal modifications to remove the integrality constraints.

The dual repair operator should allow for a more guided construction heuristic, at the cost of computing the dual values at each CMSA iteration. We can make this feature easily configurable by adding a boolean parameter to control the activation of dual repair. For the standalone CMSA, the solving of the RMP in the Solve step is only done if the parameter is enabled. With this, a tuning procedure on test instances can decide whether this additional repair operator provides benefits to the performance of the developed approaches. The tuning of this parameter for the final experimental results is discussed in Section 7.2.

Hybridizing CMSA with an LNS-based approach

This chapter looks into hybrid algorithms combining the LNS approach from Tomandl et al. [45] with the CMSA framework developed in Chapter 4. Section 6.1 introduces this LNS approach. Then, Section 6.2 presents the hybrid algorithm that was developed to combine both techniques. Finally, Section 6.3 investigates an improvement to the LNS approach for large instances using concepts inspired by CMSA.

6.1 An LNS Approach for the EADARP

We will start this chapter by giving a brief introduction to the LNS approach developed by Tomandl et al. [45]. Although this approach was developed for the dynamic EADARP, it can also be applied to the static formulation considered in this thesis. This approach consists of a construction heuristic to find an initial solution and a set of destroy and repair operators that are applied repeatedly on the incumbent solution to find improvements until a termination criterion is reached. Algorithm 6.1 presents the LNS applied to the EADARP.

For this LNS approach, the construction heuristic reinserts all requests into an empty solution to generate a feasible solution. For instances with $n \leq 50$, the greedy insertion operator is used to start from a high-quality solution directly. For bigger instances, chronological insertion is used instead, because using the greedy operator for a full solution construction becomes too expensive.

The destroy operator randomly removes a fixed number of requests from the current solution. Here, we will use the same value as in [45], that is, three requests are removed at each LNS iteration. The removed requests are then reinserted in the solution by one of the three repair operators presented in Section 4.2.1 (random, greedy, or chronological

Algorithm 6.1 Large Neighborhood Search for the EADARP

Input: an EADARP instance $inst$
Return: the best incumbent solution found

```
1:  $incumbent \leftarrow \text{CONSTRUCT}(inst)$   $\triangleright$  Initialize solution
2: while termination criteria not met do
3:    $lms\_sol \leftarrow \text{DESTROY}(incumbent)$ 
4:    $operator \leftarrow \text{CHOOSEREPAIR}()$ 
5:    $lms\_sol \leftarrow \text{REPAIR}(lms\_sol, operator)$ 
6:   if  $lms\_sol$  is better than  $incumbent$  then
7:      $incumbent \leftarrow lms\_sol$ 
8: return  $incumbent$ 
```

insertion). The selection for the repair operators is done at random, uniformly sampling each of the available operators. Should a destroy-repair cycle find a better solution than the current one, the incumbent is updated accordingly.

As it removes a relatively small number of requests from the current solution, this technique is able to quickly perform a destroy-repair cycle. Thus, the LNS approach can explore many different solutions even on larger instances.

As this thesis focuses on the static formulation of the DynEADARP, all additional considerations developed in [45] to account for dynamic aspects of the problem were left out, including the heuristics and reinforcement learning model that were introduced to guide the charging behaviour of the vehicles.

6.2 Combining CMSA and LNS

This section investigates a hybrid approach combining the CMSA framework described in Chapter 4 with the LNS approach developed by Tomandl et al. [45]. The aim of this approach is to keep the overall framework of CMSA and use the LNS as a subroutine that acts as a local search procedure to improve the incumbent and generate new feasible routes.

Compared to the general CMSA framework presented in Algorithm 4.1, we want to use the LNS in the Construct step to propose more routes and potentially improve the incumbent. Algorithm 6.2 presents the updated hybrid framework. The Merge, Solve, and Adapt steps are identical to the standalone CMSA heuristic, while the Construct has been updated to integrate the LNS approach.

6.2.1 LNS Subroutine

The main difference in the Construct step of the outer CMSA framework in the hybrid algorithm lies in the call to the LNS approach at line 11. This function call launches an LNS instance which starts exploring new solutions based on the current incumbent.

Algorithm 6.2 CMSA-LNS hybrid framework for the EADARP

Input: an EADARP instance *inst* and a *config*
Return: the best incumbent solution found

```

1: pool  $\leftarrow \emptyset$   $\triangleright$  Pool of components for the CMSA
2: incumbent  $\leftarrow \emptyset$   $\triangleright$  Current best solution
3: while termination criteria not met do
4:    $\triangleright$  Construct  $\triangleleft$ 
5:   new_routes  $\leftarrow \emptyset$ 
6:   if incumbent =  $\emptyset \vee \text{RAND}() < \text{config.pshaking}$  then  $\triangleright$  shaking
7:     cons_sol, cons_routes  $\leftarrow \text{CONSTRUCT}(inst, config)$ 
8:     if cons_sol is better than incumbent then
9:       incumbent  $\leftarrow cons\_sol$ 
10:    PUSHALL(new_routes, cons_routes)
11:    lns_sol, lns_routes  $\leftarrow \text{LNS}(inst, incumbent, config)$   $\triangleright$  LNS
12:    if lns_sol is better than incumbent then
13:      incumbent  $\leftarrow lns\_sol$ 
14:    PUSHALL(new_routes, lns_routes)
15:     $\triangleright$  Merge  $\triangleleft$ 
16:    for all r  $\in$  new_routes do
17:      pool  $\leftarrow pool \cup \{r\}$ 
18:     $\triangleright$  Solve  $\triangleleft$ 
19:    ilp_sol  $\leftarrow \text{SOLVEMASTERILP}(inst, pool)$ 
20:    if ilp_sol is better than incumbent then
21:      incumbent  $\leftarrow ilp\_sol$ 
22:     $\triangleright$  Adapt  $\triangleleft$ 
23:    ADAPTPOOL(pool, config)
24: return incumbent

```

Usually in an LNS, the incumbent is initialized by applying a construction heuristic to generate a feasible solution, as shown at line 1 in Algorithm 6.1. Afterwards, the destroy-repair cycles are performed until termination of the algorithm. In the case of the CMSA-LNS algorithm presented in Algorithm 6.2, we want the LNS to be able to focus on destroy-repair cycles and avoid having to generate this initial solution. To achieve this, we bypass the construction of an initial solution by giving a feasible solution to the LNS, namely by passing the current incumbent of the CMSA framework as parameter to the LNS at line 11 of Algorithm 6.2. By using the incumbent as the starting solution for the LNS, we skip the expensive construction heuristic that is needed to initialize the LNS, thus directly moving to destroy-repair cycles and allowing for more solutions to be explored within the given runtime. In the very first iteration of the hybrid algorithm, we make sure that the incumbent is initialized before the first call to the LNS by always

generating a new solution when the incumbent is empty, regardless of the chosen value for p_{shaking} , as shown at line 6 of the algorithm.

As we want to keep the time spent in the LNS balanced with the time spent in the rest of the CMSA framework, the configuration parameters stored in *config* also contain a limit on the runtime of the LNS instance. We introduce two configurable parameters that define the convergence criteria for the internal LNS. $t_{\text{max}}^{\text{lms}} \in \mathbb{R}_+^*$ defines the maximum runtime of the LNS, while $i_{\text{max}}^{\text{lms}} \in \mathbb{N}^*$ determines the maximum number of LNS iterations that will be executed. If either of these two termination conditions is met, the internal LNS stops and returns the best found solution. These parameters can be tuned to best fit the instance size. The tuning process is described in Section 7.2.

Compared to a standalone LNS, which only aims at improving the incumbent, the current approach also brings benefits for the outer CMSA loop. We take advantage of the fact that the LNS explores multiple solutions to gather all routes which have been generated at any point of the internal LNS, even in solutions which did not improve the incumbent. These routes can then be added to the CMSA pool to give more possibilities in the Solve step.

To reduce the overhead of creating a new LNS instance from scratch at each CMSA iteration, we only allocate the memory for the LNS subroutine once at the start of the algorithm and can reuse the same LNS instance by updating the relevant parameters and setting its initial solution to the current incumbent.

6.2.2 Shaking

Exclusively using the LNS in the Construct step creates the risk of converging towards the same local optima as the pure LNS approach. To avoid this situation, we want to ensure that the Construct step of CMSA remains diverse enough. This is why the call to the construction heuristic from the standalone CMSA approach is preserved in the hybrid algorithm. Still, as we expect the LNS subroutine to be able to explore more solutions than the construction heuristic, we introduce a configurable parameter to control how often the construction heuristic should be executed.

With these modifications, the construction heuristic acts like a shaking move. At each CMSA iteration, with a probability given by the configurable parameter $p_{\text{shaking}} \in [0, 1]$, we generate a new feasible solution independently from the incumbent. Setting the highest value of $p_{\text{shaking}} = 1.0$ is equivalent to always generating a new random solution in the Construct step before calling the LNS. This setting focuses on finding more diverse components for the CMSA pool. The lower the value of p_{shaking} , the fewer calls to the expensive construction heuristic are needed, especially for larger instances. As a trade-off, this might make the components in the CMSA pool less diverse. The process to tune this parameter and find fitting values for the experiments is described in Section 7.2.

6.3 Enhancing LNS for Large Instances

For large instances, the main bottleneck of all approaches remains the construction of new solutions. The various improvements discussed in Section 4.3 and in the shaking step of the hybrid algorithms do help reduce the time spent to generate new feasible solutions, but preliminary benchmarks show that this is not sufficient to compete with the pure LNS approach on the largest instances.

Instead, we have investigated another approach specifically for large instances, where the LNS is enhanced by continuously collecting the generated routes and regularly solving the Set Covering ILP from the Solve step of CMSA on the accumulated routes to try to improve the incumbent. We call this approach ‘LNS+Solve’.

The extreme case would be to execute the standalone LNS while collecting all generated routes. Once the LNS is finished, the Set Covering ILP formed by all collected routes is solved. The risk with this extreme approach is that the number of collected routes might grow too large and make the exact solving of the ILP intractable.

In practice, we have developed this LNS+Solve approach as a variant of the CMSA-LNS approach with specific values for the parameters p_{shaking} , $t_{\text{max}}^{\text{lns}}$ and $i_{\text{max}}^{\text{lns}}$, setting the value of each parameter as follows.

$$p_{\text{shaking}} = 0; \quad t_{\text{max}}^{\text{lns}} = 60; \quad i_{\text{max}}^{\text{lns}} = -1$$

By setting $p_{\text{shaking}} = 0$, we ensure that the construction heuristic is only called once to generate the initial solution. By choosing the values $t_{\text{max}}^{\text{lns}} = 60$ and $i_{\text{max}}^{\text{lns}} = -1$, we let the LNS run its course and only solve the Set Covering ILP once every minute. Once the ILP has been solved, we empty the CMSA pool in the Adapt step to avoid accumulating routes and making the solving of the ILP too expensive.

Experimental Analysis

Having presented the different approaches developed during this thesis in the previous chapters, this chapter will now investigate how they perform on test instances. First, we will look into the experimental environment and the test instances in Section 7.1. Then, Section 7.2 will discuss the tuning of configurable parameters mentioned throughout the previous chapters. Finally, Section 7.3 will describe the actual experiments and Section 7.4 will investigate the performance of each approach, comparing them with one another and looking into interesting results for each individual technique.

7.1 Test Environment

The algorithms discussed in this thesis were implemented with Julia¹ version 1.12.5, based on the MHLlib² library. The commercial solver Gurobi³ version 13.0.2 was used in single-threaded mode to solve the MILP formulations.

All experiments were performed on the compute cluster of the Algorithms and Complexity Group (which we will refer to as the AC Cluster), on a 2.53 GHz Intel Xeon E5540 Quad Core processor. Each individual run was performed on a single thread and restricted to 10GB of memory.

As we are using the programming language Julia, there might be some artifacts from the just-in-time compilation scheme in the first call of an algorithm in a Julia process, leading to worse performance than the other runs. To avoid this bias, each algorithm is run once to ensure that the program is fully compiled before performing the actual measurements.

¹<https://julialang.org/>

²<https://github.com/ac-tuwien/MHLlib.jl>

³<https://www.gurobi.com/>

The benchmarks presenting the final results were performed on the `uniform` benchmark instances from [45]. The small instances with 10, 20 or 50 requests were first introduced by [22]. This instance set was extended by [45] to generate instances with up to 5000 requests. For each instance size, 30 different instances have been generated. In these instances, requests arrive according to a homogeneous Poisson process within a time horizon T^{plan} of 24 hours. Table 7.1 presents an overview of the considered instance sizes.

	Small			Medium				Large		
n	5	10	20	50	100	200	500	1000	2000	5000
n_K	2	2	2	2	4	8	20	40	78	180
n_S	3	3	3	3	3	3	3	3	3	3

Table 7.1: Instance sizes

Observations on the small `uniform` benchmark instances with $n \in \{10, 20\}$ have shown the requests to be highly spread out over the planning horizon of 24 hours. To extend the set of available instances, we randomly generate new instances with $n \in \{5, 10, 20\}$ requests according to the same methodology, but with a planning horizon that is reduced proportionally to the number of requests. We take instances with $n = 50$ requests and $T^{\text{plan}} = 24$ hours as model and generate new random instances where requests are more tightly concentrated. This additional set of instances is called `packed` instances and is composed of thirty new instances of each size. Aside from this change to the planning horizon, these `packed` instances are generated by applying the same methodology as [22, 45].

$$\begin{aligned}
 n = 5 & \quad T^{\text{plan}} = 3h \\
 n = 10 & \quad T^{\text{plan}} = 5h \\
 n = 20 & \quad T^{\text{plan}} = 10h
 \end{aligned}$$

For medium and large instances, we will only consider the `uniform` benchmark instances from Tomandl et al. [45].

7.2 Parameter Tuning

The performance of the algorithms presented in Chapters 4 to 6 can be influenced by the values chosen for some parameters. To obtain the best results possible, we want to find adequate values for these parameters. This is achieved by tuning the parameters and testing different combinations to find the best-fitting one. Parameter tuning can be done manually by comparing test results for different parameter settings. In this thesis, we will use `irace`⁴ [28] to automatically find the best parameter values by running the iterated race method on test instances.

⁴<https://github.com/MLopez-Ibanez/irace>

7.2.1 Tunable Parameters

Among the algorithms developed in the previous chapters, there are configurable parameters in the standalone CMSA approach and in the CMSA-LNS hybrid algorithm. Table 7.2 summarizes the different parameters and their significance. For more detail on each parameter, we refer to the linked sections in the column ‘Reference’.

Parameter	Domain	Reference	Description
$k_{\max}$	$\{-1\} \cup \{1, \dots, n_K\}$	Section 4.3.1	Max number of routes to consider for insertion in the repair operators.
n_{destroy}	$\{-1\} \cup \{1, \dots, n\}$	Section 4.3.2	Number of requests to destroy to obtain a partial solution for the Construct step.
r^{rnd}	$\{\text{true}, \text{false}\}$	Section 4.2.1	Whether to enable random repair.
r^{dual}	$\{\text{true}, \text{false}\}$	Section 5.4.3	Whether to enable dual repair.
p_{shaking}	$[0, 1]$	Section 6.2.2	Probability of generating a new solution in the CMSA-LNS Construct step.
$t_{\max}^{\text{lms}}$	$\mathbb{R}_+^*$	Section 6.2.1	Max runtime for the internal LNS in the CMSA-LNS algorithm.
$i_{\max}^{\text{lms}}$	$\{-1\} \cup \mathbb{N}^*$	Section 6.2.1	Max number of internal LNS iterations in the CMSA-LNS algorithm.

Table 7.2: Configurable parameters considered for tuning

The first four parameters were defined for the standalone CMSA approach, while the last three were introduced specifically for the CMSA-LNS hybrid approach. For the parameters $k_{\max}$, n_{destroy} and $i_{\max}^{\text{lms}}$, we introduce the special value -1 , which will be used to indicate that this parameter is turned off. For $k_{\max}$ and n_{destroy} , this special value is equivalent to setting $k_{\max} = n_K$ and $n_{\text{destroy}} = n$ respectively. Setting $i_{\max}^{\text{lms}} = -1$ is equivalent to only controlling the LNS subroutine in the CMSA-LNS algorithm by its runtime limit $t_{\max}^{\text{lms}}$, performing as many iterations as possible within the given runtime.

Additionally, we forbid the case where both r^{rnd} and r^{dual} are set to `false`. This is done to avoid the situation where chronological insertion is the only repair operation available, as chronological insertion is less randomized than the other proposed insertion procedures.

7.2.2 Tuning Strategy

In theory, we could perform a separate tuning run for each algorithm and each instance size. In practice, we will reduce the number of actual tuning runs, because each tuning run can be computationally expensive.

The parameters defined in the standalone CMSA algorithm are used for all other approaches. As all approaches use the same CMSA framework as a base, we will only tune the parameters once on the standalone CMSA algorithm and use the obtained tuned values for all other approaches. The parameters specific to the CMSA-LNS algorithm are tuned in a second run, using the tuned values of the CMSA parameters as a baseline.

To further reduce the number of tuning runs, we also group instances into three size categories: small, medium, and large instances. These categories are defined arbitrarily based on trends observed in preliminary benchmark runs. See Table 7.1 for the full distribution of considered instance sizes. To avoid bias in the tuning results, irace is executed on randomly generated instances that are independent from the test instances that will be used to measure the performance of the algorithms. In the small and medium categories, for every size that belongs to this category, we generate 10 random tuning instances using the same methodology as [22, 45]. For the large instances, preliminary results discussed in Section 7.4.1 show that the proposed algorithms are not viable for instances of size $n = 5000$. As such, tuning instances were only generated for instances with size $n \in \{1000, 2000\}$.

Instance	$k_{\max}$	n_{destroy}	r^{rnd}	r^{dual}
Small	$\{-1\}$	$\{-1, 5, 10\}$	$\{\text{true}, \text{false}\}$	$\{\text{true}, \text{false}\}$
Medium	$\{-1, 5, 10\}$	$\{-1, 20, 100\}$	$\{\text{true}, \text{false}\}$	$\{\text{true}, \text{false}\}$
Large	$\{-1, 10, 20\}$	$\{-1, 100, 500\}$	$\{\text{true}, \text{false}\}$	$\{\text{true}, \text{false}\}$

Table 7.3: Restricted Domain for the standalone CMSA parameters

Instance	$t_{\max}^{\text{lns}}$	$i_{\max}^{\text{lns}}$	p_{shaking}
Small	$\{5, 10, 20\}$	$\{-1, 10, 50\}$	$\{0.1, 0.4, 0.7, 1.0\}$
Medium	$\{5, 10, 20\}$	$\{-1, 10, 50\}$	$\{0.1, 0.4, 0.7, 1.0\}$
Large	$\{5, 10, 20\}$	$\{-1, 10, 50\}$	$\{0.1, 0.4, 0.7, 1.0\}$

Table 7.4: Restricted Domain for the CMSA-LNS parameters

To reduce the space of possible parameter combinations in the tuning runs, we will select a subset of significant values for each parameter, instead of allowing a potentially infinite number of parameter settings. The significant values are chosen arbitrarily, based on manual testing of different parameter values to get an intuition of the influence of each parameter.

Finally, we define the parameter combinations that will be considered in the tuning scenarios generated by irace. Tables 7.3 and 7.4 define the discrete domain of allowed values for each parameter.

For small instances, the parameter $k_{\max}$ in the CMSA algorithm is disabled by default, because there are only 2 vehicles available in these instances. As such, we consider the speedup obtained by only looking at a single vehicle to be negligible compared to the negative impact on the solution quality.

For the CMSA-LNS hybrid approach, the available parameter values do not depend on the instance size, but we still want to tune each instance category separately, as they could have a different behavior.

7.2.3 Tuning Results

The tuning runs were performed on the AC Cluster using irace 3.5 with the default configuration parameters and a budget of 200 experiments per tuning run. The individual calls to the CMSA algorithm were allowed a runtime of 1 minute for the small instances and 5 minutes for the medium and large instances. The final results are reported in Tables 7.5 and 7.6. Let us provide an intuition about some of these results.

Instance	$k_{\max}$	n_{destroy}	r^{rnd}	r^{dual}
Small	-1	10	true	false
Medium	-1	20	false	true
Large	-1	100	false	true

Table 7.5: Tuning results for the standalone CMSA parameters

Instance	$t_{\max}^{\text{lms}}$	$i_{\max}^{\text{lms}}$	p_{shaking}
Small	10	10	0.4
Medium	10	10	1.0
Large	10	50	1.0

Table 7.6: Tuning results for the CMSA-LNS parameters

Observing the two speedup strategies controlled by $k_{\max}$ and n_{destroy} , we can note that the $k_{\max}$ parameter is always set to the disabled setting $k_{\max} = -1$, while n_{destroy} is set to the intermediate value of $n_{\text{destroy}} = 10$ on small instances and to the lowest value on medium and large instances. To interpret these results, we have investigated the influence of these parameters more deeply in Section 7.4.1. From these results, we can deduce that enabling $k_{\max}$ deteriorates the objective value more significantly, while setting a lower value for n_{destroy} has a relatively small impact on the quality of the obtained solutions, but makes the Construct step substantially faster, allowing CMSA to execute more iterations.

We can also observe that the random repair operator is preferred on small instances, but that the tuning for medium and large instances disables it in favor of dual repair. On the larger instances, CMSA is able to perform fewer iterations. Therefore, using the comparatively smarter dual repair can lead to more promising solutions.

Tuning results for the CMSA-LNS parameters find a balance between CMSA and LNS. The high value of p_{shaking} for medium and large instances indicates that this approach needs a diverse pool of solutions, while the parameter $t_{\max}^{\text{lms}}$ converges towards the intermediate value $t_{\max}^{\text{lms}} = 10$ for all instances. The lower value of p_{shaking} for small instances indicates that the algorithm focuses more on the LNS, while still regularly generating new solutions to avoid local optima.

7.3 Description of the Experiments

We will detail here the configuration of each algorithm on each of the three instance categories defined previously. The description of the experiments is summarized in Table 7.7. On each instance, ten independent runs were performed for each algorithm.

Instance	Runtime	Algorithms
Small	60 s	CMSA, CMSA-CG, CMSA-CG ⁺ , CMSA-LNS, LNS+Solve, LNS
Medium	5 min	CMSA, CMSA-LNS, LNS+Solve, LNS
Large	10 min	CMSA, CMSA-LNS, LNS+Solve, LNS

Table 7.7: Overview of realized experiments

Let us describe the list of algorithms that will be tested and define the associated abbreviations.

- ‘CMSA’ is the standalone CMSA approach that was presented in Chapter 4.
- ‘CMSA-CG’ is the CMSA-based approach integrating Column Generation in the Solve step to propose new solutions in the CMSA pool by running a labeling algorithm to solve the Pricing Subproblem. This approach is described in detail in Chapter 5. For the experiments, we have set a runtime limit of 5 seconds for each call to the labeling algorithm.
- ‘CMSA-CG⁺’ uses the same hybridization mechanism as CMSA-CG, but it relies on an alternative labeling algorithm that was developed concurrently with this thesis by Heinzl [20]. This approach uses Set Partitioning instead of the Set Covering formulation introduced for the CMSA-CG approach and develops more specific pruning heuristics to improve the labeling algorithm. We have imposed a limit of 5 seconds per call to the labeling algorithm, same as for the CMSA-CG approach.
- ‘CMSA-LNS’ is the hybrid algorithm developed by combining the standalone CMSA algorithm with the LNS approach from [45]. Here, the Construct step of CMSA is modified by running the LNS algorithm to generate new routes and improve the incumbent solution.
- ‘LNS+Solve’ is an alternative to the CMSA-LNS algorithm that was developed specifically for large instances. This approach mainly focuses on running the LNS algorithm, collecting all generated routes and regularly solving the Set Covering ILP for the current routes to improve the solution. It is described in Section 6.3.
- ‘LNS’ is the approach developed by Tomandl et al. [45]. It is presented in Section 6.1. This approach is used as an external baseline to compare with the approaches developed in this thesis.

To ensure fairness between all algorithms, the stopping criterion for each run is defined exclusively by a limit on the runtime. In a practical application, the definition of stopping criteria would depend on the available resources as well as on the convergence of each approach, which can be measured by looking at the number of iterations since the last improvement to the incumbent.

The concrete value for this runtime limit is dependent on the category of instance and is reported in Table 7.7. This limit is handled as a soft constraint. This means that, given a runtime limit of $t^{\max}$ seconds, the algorithm will never start a new iteration after $t^{\max}$ seconds of runtime have been reached, but at the same time, the program will not be abruptly interrupted and will finish the ongoing iteration, sometimes overstepping the $t^{\max}$ seconds mark by a few seconds. In our experiments, most runs finish within 10 seconds of the runtime limit $t^{\max}$, which we consider to be a tolerable excess.

Preliminary experiments have shown that solving the ILP in the Solve step takes a negligible part of the runtime, even when the number of components in the pool grows. As such, no age nor size limits have been imposed on the CMSA pool for any of the CMSA-based approaches.

As results on small instances have shown that the Column Generation approach cannot deliver improved results within a reasonable time for instances with $n \geq 20$ requests, the CMSA-CG and CMSA-CG⁺ algorithms are not evaluated on medium and large instances.

Unless otherwise specified, all experiments were executed on the static instances, considering that all requests are known before starting the optimization.

7.4 Experimental Results

This section will present and analyze the results obtained from executing the algorithms described in this thesis on test instances. First, Section 7.4.1 will investigate specific results obtained prior to the final benchmarks. These results support design decisions that were made in the earlier chapters. Section 7.4.2 will focus on the overall performance of each algorithm, comparing them with one another and with the LNS approach from Tomandl et al. [45]. Finally, Section 7.4.3 will look into side-by-side comparisons of the developed algorithms, analyzing the differences obtained by hybridization compared to the standalone algorithms.

7.4.1 Preliminary Results

Before looking into the results obtained by running the developed algorithms on benchmark instances, we will first investigate some preliminary results that support the claims made in earlier chapters regarding specific components of the presented algorithms. This section will also look into preliminary results obtained on instances of size $n = 5000$, on which none of the tested algorithms are viable, including the LNS approach.

Post-processing in the Solve Step

In Section 3.2.2, we have mentioned the need for a post-processing step to obtain valid feasible solutions when solving the Set Covering ILP formulation of the EADARP. We have declared that there are only a few ILP solutions which will be affected by this post-processing.

To confirm this affirmation, we have measured the total number of solutions that had to be modified by applying this post-processing step over all experiments performed in the benchmarks described earlier. The results presented in Table 7.8 report the number of times where a solution delivered by the Set covering ILP contained at least one duplicate request. When considering the results, recall that there are 300 runs per approach for most of the instance sizes, with 600 runs on instances with $n \in \{10, 20\}$, as we consider both packed and uniform instances.

Instance (n_K, n)	Small			Medium				Large	
	(2, 5)	(2, 10)	(2, 20)	(2, 50)	(4, 100)	(8, 200)	(20, 500)	(40, 1000)	(78, 2000)
CMSA-CG	0	0	0	–	–	–	–	–	–
CMSA-CG ⁺	0	2	3	–	–	–	–	–	–
CMSA	0	0	2	0	20	6	11	135	47
CMSA-LNS	0	0	0	2	13	14	46	134	56
LNS+Solve	0	0	0	0	1	10	64	665	386

Table 7.8: Total number of calls to the post-processing step of the Set Covering ILP

Looking at the reported results, we can see that this post-processing is only needed for a very small proportion of realized experiments. It is only for the largest instances with $n \geq 1000$ that the number of calls to this procedure becomes more substantial. Comparing these results with the number of performed iterations, which are discussed later in Tables 7.12, 7.13, 7.15 and 7.17, this confirms that the post-processing represents a negligible part of the developed algorithms.

Repartition of the runtime over the steps of CMSA

In Section 7.3, the decision was made not to set any age or size limit to the CMSA pool. To support this decision, we have measured the time spent in each step of the CMSA metaheuristic. Table 7.9 reports the proportional runtime of each step of the standalone CMSA algorithm averaged over all experiments performed on instances of the given size. There we can clearly see that, starting from medium instances, the Construct step takes by far the most time in this approach, accounting for over 95% of the runtime.

As the Solve step remains comparatively small, this justifies the decision that was taken in Section 7.3 to not impose any limits on the size of the CMSA pool. Indeed, making the pool smaller would deteriorate the quality of the delivered solutions while having very little impact on the number of iterations that can be performed within the given runtime. For small instances, where the Solve step takes a greater proportion of the

Instance (n_K, n)	Small			Medium				Large	
	(2, 5)	(2, 10)	(2, 20)	(2, 50)	(4, 100)	(8, 200)	(20, 500)	(40, 1000)	(78, 2000)
Construct	33.5	34.2	75.3	96.7	98.9	99.0	98.0	98.7	99.2
Merge	0.4	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Solve	66.1	65.7	24.7	3.3	1.1	1.0	2.0	1.3	0.8
Adapt	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 7.9: Proportional runtime of each CMSA step (in %)

runtime, we do not set any limit either, as experiments have shown that CMSA is still able to converge towards its best possible solution within the allotted runtime.

Influence of the parameters $k_{\max}$ and n_{destroy}

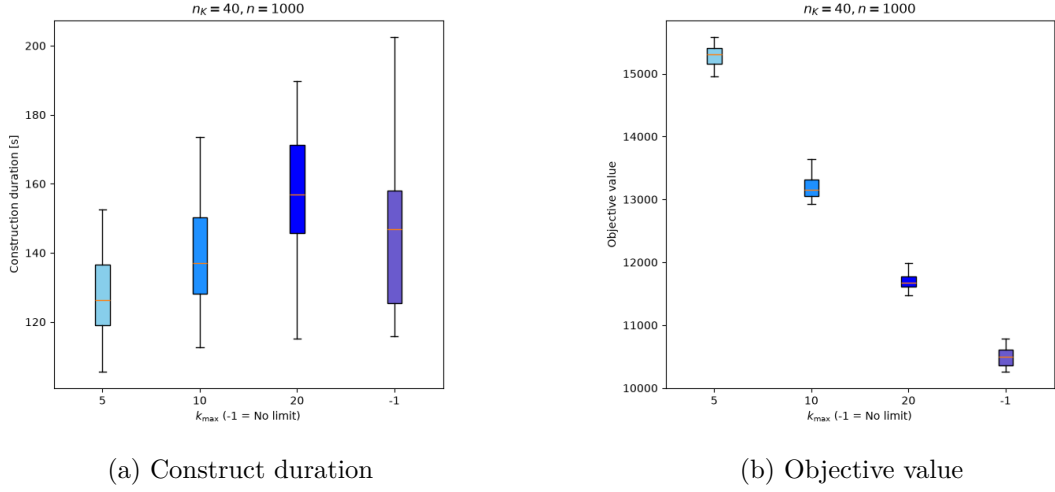
In Section 4.3, we have introduced two separate strategies to speed up the Construct step of CMSA. In this section, we want to validate these strategies by studying them in isolation from other factors. We want to investigate the influence of the parameters $k_{\max}$ and n_{destroy} , and be able to measure the trade-off between speedup and loss of solution quality.

In the strategy using the parameter $k_{\max}$, we generate a new feasible solution from scratch by inserting each request at the best possible position among $k_{\max}$ randomly selected routes, instead of considering all n_K available vehicles. To measure the influence of the chosen value for $k_{\max}$, we perform a single call to the construction heuristic starting from an empty solution and measure its duration. By also measuring the objective value of the generated solution, we can quantify the trade-off between speedup and solution quality.

Figure 7.1 shows the results obtained by performing the experiment described above on ten randomly generated instances of size ($n_K = 40, n = 1000$). For each instance, ten independent runs were performed. To avoid any bias, only the chronological insertion operator is enabled for these experiments. The figure on the left (fig. 7.1a) shows the duration of the Construct step, while the one on the right (fig. 7.1b) represents the objective value of the constructed solution.

Overall, the results align with the theoretical expectations: the lower the value for $k_{\max}$, the faster the construction, but the worse the obtained solution. Still, we can observe that the loss of solution quality is considerable, compared to a relatively small gain in the speed of the construction step. This indicates that, contrary to the expectations, this strategy only brings rather low improvements for the considered instances.

In particular, we observe that for $k_{\max} = 20$, the Construct step is slower than for $k_{\max} = -1$. Looking deeper into the internal operations of the insertion heuristic, we have been able to narrow the reason for this result down to the on-the-fly charging stop insertion strategy (OTFCSI) maintaining the battery feasibility of the constructed routes, which was mentioned in Section 4.2.1. This OTF strategy concentrates most of the runtime of the construction heuristic. Looking at the number of calls to this part of the construction heuristic, reported in Table 7.10, the number of partial routes which have

Figure 7.1: Influence of the $k_{\max}$ parameter on the construction heuristic

to be completed by updating charging stops goes down for smaller values of $k_{\max}$. But the total time spent in the OTF insertion remains approximately constant, showing that the time spent for each insertion grows. This explains why the overall runtime of the construction heuristic does not go down, even though less routes are actually considered for insertion.

$k_{\max}$	5	10	20	-1
Total number of calls	152792	262515	425763	615976
Total time spent [s]	124.87	134.69	148.21	132.05
Average time per call [ms]	0.817	0.513	0.348	0.214

Table 7.10: Statistics of OTFCSI for different values of $k_{\max}$

In the case of the n_{destroy} parameter, we realize very similar experiments. We only have to account for the fact that the n_{destroy} strategy only applies when a solution has already been found. Indeed, this technique works by partially destroying an existing solution, so that there are fewer requests to insert during the construction heuristic. As such, to measure the impact of this parameter, we will run the CMSA algorithm for exactly two iterations and measure the time spent in the Construct step of the second iteration.

Once again, ten independent runs are realized on ten randomly generated instances of size $(n_K = 40, n = 1000)$ to obtain the results shown in Figure 7.2. The runtime of the construction heuristic in the second CMSA iteration is shown on the left for different values of n_{destroy} (fig. 7.2a), while the figure on the right (fig. 7.2b) shows the objective value of the final solution.

Similarly to $k_{\max}$, the observed results align with the expectations, with a lower n_{destroy} value making the second call to the construction heuristic faster. The results shown

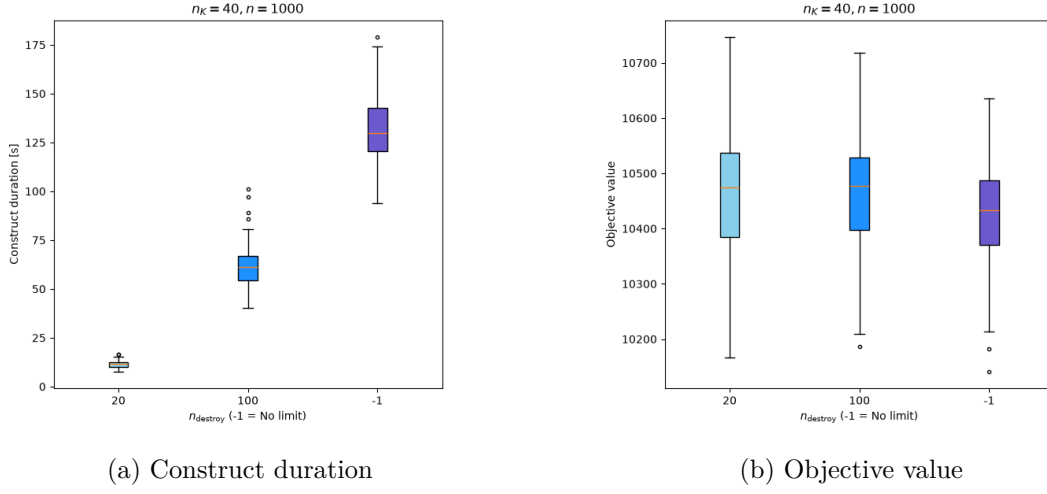


Figure 7.2: Influence of the n_{destroy} parameter on the construction heuristic

in the figures fully confirm this observation, with the experiments with lower values of n_{destroy} showing a very important gain in the speed of the construction heuristic and a relatively small impact on the solution quality.

Overall, these two benchmarks confirm the fact that the two developed strategies can contribute to making the Construct step of CMSA faster. They also show that an actual benchmark is needed to be able to compare different approaches and to confirm whether results align with the expectations. This also justifies why we need to tune such parameters to find the best-fitting value when performing actual experiments.

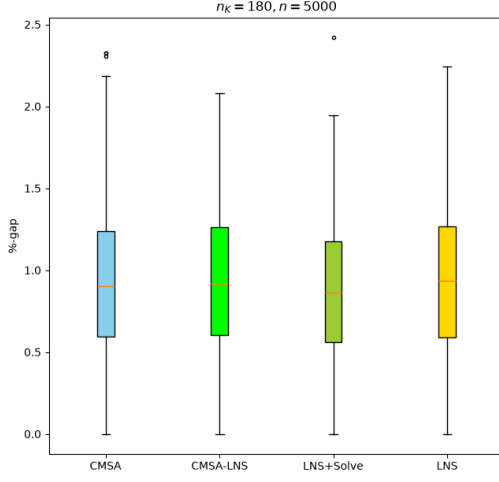
Instances of size $n = 5000$

For the largest considered instance size with $n = 5000$, preliminary benchmark results presented in Figure 7.3 show that none of the considered algorithms are viable, even the standalone LNS approach, which is used for comparison.

The boxplots in Figure 7.3a show the obtained relative objective gap (called %-gap) between each experiment and the best solution found by all algorithms on a given instance. In contrast to the raw objective value of a solution, which is instance-specific, this relative gap can be easily compared across multiple instances. It is computed as presented in Equations (7.1) and (7.2).

$$\% \text{-gap}(run, inst) = \frac{obj(run, inst) - \min_obj(inst)}{\min_obj(inst)} \cdot 100 \quad (7.1)$$

$$\text{where } \min_obj(inst) = \min_{r \in \text{RUNS}(inst)} obj(r, inst) \quad (7.2)$$



(a) Observed %-gap

		CMSA		CMSA-LNS	
%-gap	σ	0.906	0.470	0.926	0.465
I^{best}	I^{total}	1	1	1	1
		LNS+Solve		LNS	
%-gap	σ	0.868	0.433	0.950	0.450
I^{best}	I^{total}	1	1	1	1

(b) Summarized results

Figure 7.3: Obtained results for instances of size $n = 5000$

In Figure 7.3b, we have reported the total number of iterations I^{total} performed by each algorithm, as well as the iteration I^{best} where the best found solution was found.

For instances with $n = 5000$, Figure 7.3b shows that all investigated approaches only perform a single iteration. They stop directly after constructing the initial solution. Looking in more detail at the execution logs, we can see that constructing the initial solution takes around 1000 seconds, which already oversteps the runtime limit of 10 minutes. For this instance size, the construction heuristic used throughout this thesis, based on inserting requests at the best possible position, is not viable.

These experiments also allow us to measure the dispersion of the results obtained by the construction heuristic. Applying the request insertion procedure to generate new solutions delivers solutions with an average relative %-gap of approximately $0.9\% \pm 0.45$.

7.4.2 Overall Results

This section will discuss the results obtained by running the algorithms described in Section 7.3 on the benchmark instances. First, we investigate the performance of CMSA by looking at how often the Solve step finds improvements in the experiments. Then, we will look into the performance of each of the proposed approaches on the different instance categories.

Improvements to the incumbent by the Solve steps of CMSA

In this section, we investigate how much the Solve step contributes to the final solution of the CMSA algorithm. In the CMSA metaheuristic, the objective can improve either by the Construct step finding a better solution than the current incumbent, or by finding a new best solution in the Solve step by solving the restricted Set Covering ILP. As this

second mechanism is one of the specificities of CMSA, we want to investigate it more deeply.

Let us start by illustrating how both Construct and Solve steps can improve the solution in an experiment. In Figure 7.4, we have represented the evolution of the objective value of the best found solution at each iteration of the CMSA algorithm for an experiment performed on the instance `n00100-m0004-uniform-test00002`. The markers in the figure highlight which CMSA step gave the last improvement. The initial solution is always given by the construction heuristic, in the Construct step. Afterwards, both steps can find an improved solution.

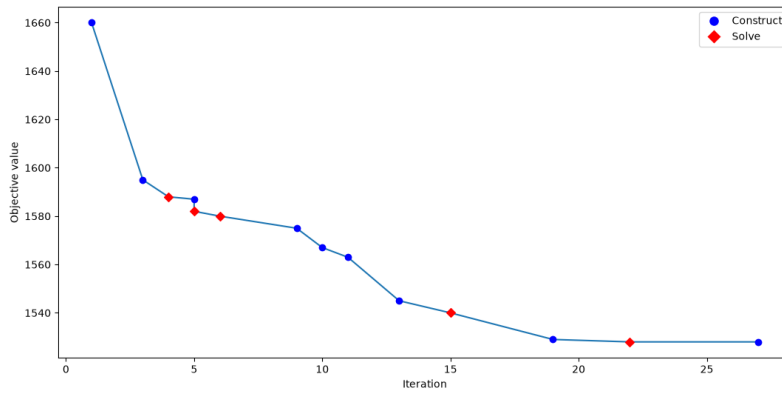


Figure 7.4: Evolution of the objective value per iteration for an experiment on the instance `n00100-m0004-uniform-test00002`

At iteration 5, we can observe that both steps can give an improvement in the same iteration. This is the case when the Construct step finds a new incumbent, but the Solve step immediately improves on it thanks to existing components in the CMSA pool.

As the Solve step is one of the specificities of the CMSA metaheuristic, we want to investigate how often solving the Set Covering ILP finds an improved solution in our experiments. The results presented in Table 7.11 measure how often an improvement was found by the Solve step for each instance size. These results show that the number of improvements depends on both the algorithm that is used and the instance size.

Instance (n_K, n)	Small			Medium				Large	
	(2, 5)	(2, 10)	(2, 20)	(2, 50)	(4, 100)	(8, 200)	(20, 500)	(40, 1000)	(78, 2000)
CMSA-CG	0.570	0.043	0.045	—	—	—	—	—	—
CMSA-CG ⁺	0.510	1.122	0.062	—	—	—	—	—	—
CMSA	0.000	0.090	0.325	0.343	3.540	5.877	5.140	7.410	5.157
CMSA-LNS	0.000	0.000	0.045	0.110	0.863	2.147	1.760	6.133	4.117
LNS+Solve	0.000	0.000	0.002	0.010	0.060	0.073	0.037	0.023	0.010

Table 7.11: Average number of improvements found by the Solve step per run

On smaller instances with $n \leq 50$, we tend to have very few improvements for all approaches, as the construction heuristic is already able to explore a large set of solutions and find high-quality solutions. In this case, the diversity of solutions brought by the Construct step of CMSA contributes more to the quality of the final solution than the Solve step, which is unable to improve the found solutions. The only exception regards the approaches using Column Generation, for which adding new promising routes based on the current pool makes the Solve step consistently find improvements.

For medium to large instances with $n \geq 100$, we can see that the number of improvements found by the Solve step grows considerably for both the CMSA and CMSA-LNS approaches. These instances deliver results similar to the one depicted in Figure 7.4, with the Solve step regularly finding improvements over the course of the algorithms.

Overall, while not occurring at every iteration, we can see the importance of the Solve step to improve the solution. It allows the construction and repair heuristics to restart from an improved solution, exploring solutions that would have remained unreachable otherwise.

The obtained results for the LNS+Solve approach show that very few experiments are improved by the addition of the Solve step. This indicates that the set of routes explored by the LNS cannot be recombined to reach a better solution, and that the LNS+Solve approach does not obtain a significant enhancement from the addition of the Solve step to the LNS. These results will be discussed in more detail in Section 7.4.3.

Having investigated this point specific to the CMSA metaheuristic, let us now analyze the obtained results for the different instance sizes.

Small instances

On small instances, the construction heuristic always delivers a feasible solution within a few milliseconds, making it easy to obtain many solutions within the allotted runtime of one minute. Here, convergence is achieved before the end of the allotted runtime for most approaches. For this instance size, the main limitation is the ability of each approach to explore the solution space and find high-quality solutions. This is confirmed by the results reported in Tables 7.12 and 7.13. Indeed, the difference between the columns I^{total} , measuring the number of performed iterations, and I^{best} , indicating the iteration where the best solution was found, shows that most algorithms find their best solution relatively quickly and spend the rest of the runtime unsuccessfully trying to improve it. The exceptions to this observation are: (1) the LNS+Solve approach, which is expected because this approach only realizes a single iteration of the outer CMSA loop within the given runtime of one minute, and (2) the CMSA-CG algorithm for $n \in \{10, 20\}$ and the CMSA-CG⁺ approach for $n = 20$, which encounter scalability issues that will be discussed in more detail below and in Section 7.4.3.

On the very small instances with $n = 5$, we can see in Figure 7.5 that all algorithms reach the best found solution on most instances, with a %-gap approximating zero. Here,

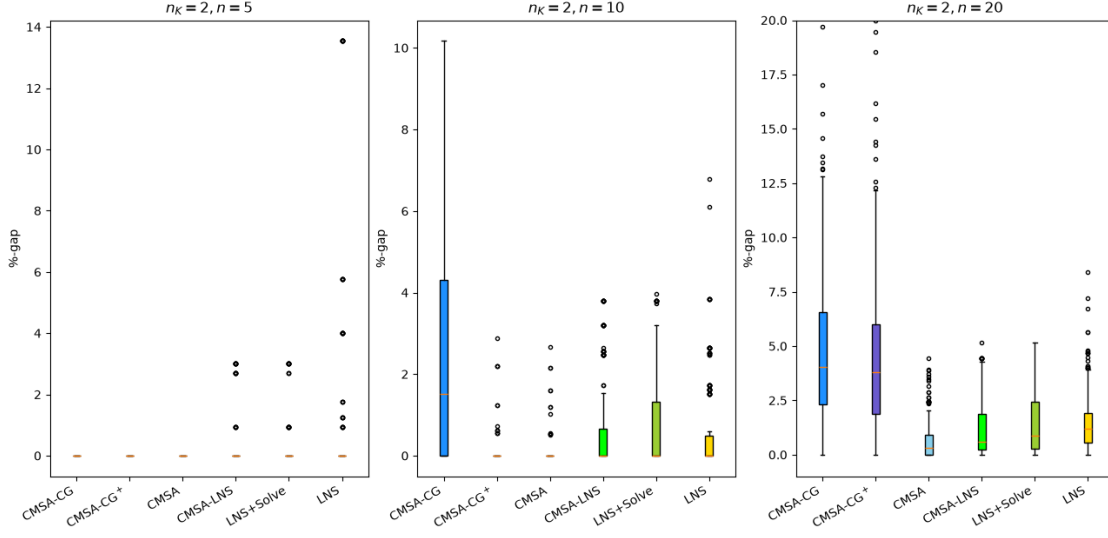


Figure 7.5: Relative objective gap on small packed instances

Instance	CMSA-CG		CMSA-CG ⁺		CMSA		CMSA-LNS		LNS+Solve		LNS	
(n_K, n)	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ
(2, 5)	0.000	0.000	0.000	0.000	0.000	0.000	0.194	0.691	0.158	0.614	0.785	2.625
(2, 10)	2.174	2.270	0.054	0.303	0.054	0.294	0.581	1.055	0.708	1.137	0.481	1.007
(2, 20)	4.940	4.263	4.641	4.494	0.662	0.832	1.160	1.248	1.389	1.341	1.487	1.331
(n_K, n)	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}
(2, 5)	2	220	2	4171	12	11932	2	3261 (32608)	1	1 (42339)	10	43253
(2, 10)	3	6	5	175	109	1063	5	577 (5768)	1	1 (6630)	169	6567
(2, 20)	4	6	4	6	59	191	14	82 (823)	1	1 (934)	133	874

Table 7.12: Obtained results on small packed instances

we can observe that CMSA and the two CMSA-CG approaches obtain more consistent results than the approaches relying more heavily on the LNS, with fewer outliers. A deeper look into the results shows that there are specific instances where the LNS and LNS-hybrids are unable to reach the best solution. For the LNS and LNS+Solve approaches, combining this observation with the analysis of the impact of the Solve step described in Section 7.4.2, we can conclude that the Construct step of CMSA is more diverse than the LNS approach and that the greedy repair operator used by the LNS to initialize its solution makes it unable to reach the best found solution on some instances.

Let us look more precisely at the approaches using Column Generation in combination with CMSA. For small instances with $n = 5$, the fact that the CG approaches perform a high number of iterations without finding an improvement indicates that the found solution is likely optimal. Here, we can note that CMSA-CG and CMSA-CG⁺ have a significantly lower average value for I^{best} than the standalone CMSA and that the Solve step finds an improvement for these approaches in more than half of the experiments, indicating that CG guides the search towards the best found solution faster than the construction heuristic of CMSA.

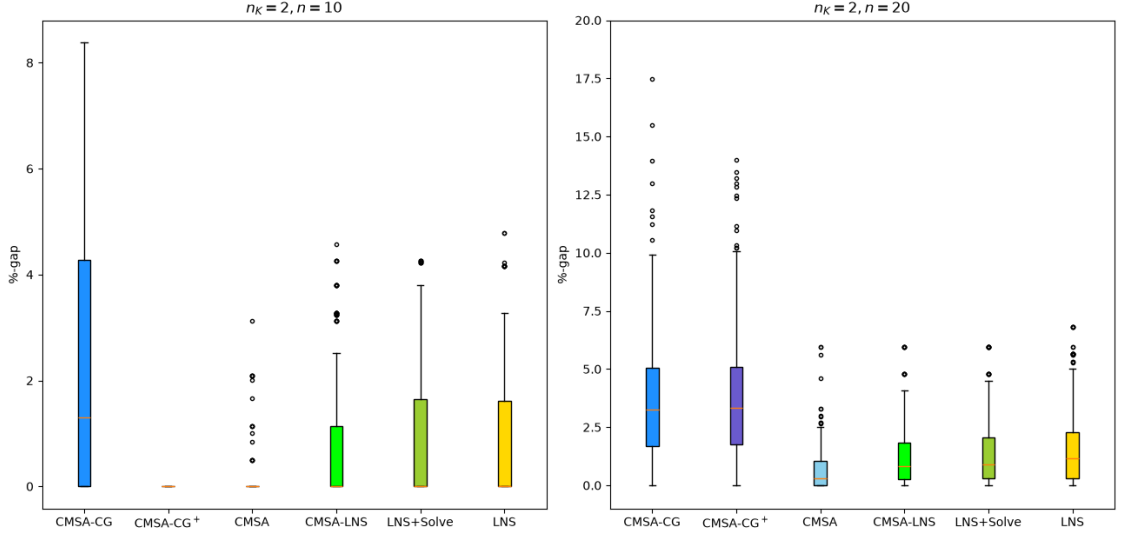


Figure 7.6: Relative objective gap on small uniform instances

Looking at the bigger instances with $n \in \{10, 20\}$, the results of the CMSA-CG algorithm worsen. The low number of performed iterations I^{total} for this approach in Tables 7.12 and 7.13 indicates that the labeling algorithm developed in Section 5.3 cannot terminate within the given runtime. Most of the runtime is spent in the labeling algorithm, which does not find any improving solution within the given time and makes it so that the CMSA-CG framework executes fewer iterations than the other approaches. The CMSA-CG⁺ approach overcomes this limitation on instances with $n = 10$, even outperforming all other approaches on small uniform instances in Figure 7.6. This shows that more advanced strategies in the pruning of labels can make Column Generation based techniques useful for small instances. This is further confirmed by looking at the improvements brought by the Solve step for CMSA-CG⁺ on instances of size $n = 10$, which emphasizes that Column Generation consistently finds improving routes to add to the CMSA pool. But even the heuristic strategies developed in this advanced labeling algorithm fail to scale to instances with $n = 20$ requests. Here, we can see that both CG-based approaches are outperformed by the other techniques, with a considerably higher average relative gap. For these instances, the labeling algorithm takes up time, but cannot propose improving solutions to the CMSA metaheuristic.

To be able to more accurately compare the number of iterations performed by the CMSA-LNS and LNS+Solve approaches, we have indicated in parentheses in Table 7.12 the number of internal LNS iterations performed by these algorithms. Comparing this number for CMSA-LNS to the number of iterations performed by the LNS, we can see an indication of the time spent in the Solve step of CMSA. For the smallest instances with $n = 5$, CMSA-LNS performs around 25% fewer iterations than LNS, but this gap becomes smaller for instances of size $n \in \{10, 20\}$, where the difference in the total number of iterations performed by both approaches becomes smaller than 10%.

Instance	CMSA-CG		CMSA-CG ⁺		CMSA		CMSA-LNS		LNS+Solve		LNS	
(n_K, n)	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ
(2, 10)	2.112	2.262	0.000	0.000	0.087	0.384	0.782	1.231	0.889	1.262	0.773	1.203
(2, 20)	4.259	7.105	4.423	6.312	0.682	0.923	1.243	1.363	1.406	1.463	1.605	1.627
(n_K, n)	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}
(2, 10)	3	7	4	343	95	962	13	524 (5242)	1	1 (5851)	123	5867
(2, 20)	4	7	4	7	55	174	15	74 (738)	1	1 (786)	171	764

Table 7.13: Obtained results on small uniform instances

p -value	CMSA-CG	CMSA-CG ⁺	CMSA	CMSA-LNS	LNS+Solve	LNS
CMSA-CG	–	1.000	1.000	1.000	1.000	1.000
CMSA-CG ⁺	0.000	–	1.000	1.000	1.000	1.000
CMSA	0.000	0.000	–	0.000	0.000	0.000
CMSA-LNS	0.000	0.000	1.000	–	0.000	0.000
LNS+Solve	0.000	0.000	1.000	1.000	–	0.195
LNS	0.000	0.000	1.000	1.000	0.805	–

Table 7.14: Pairwise Wilcoxon rank test for small instances

To determine a clear hierarchy between the different approaches, we have performed a pairwise one-sided Wilcoxon signed-rank tests with a confidence threshold of 95%, on the objective value of the solutions found for both packed and uniform instances. The results of this statistical test are reported in Table 7.14. In the table, we have highlighted in green the cells where the Wilcoxon test found a significant difference between the approaches. If a cell in the table has a p -value that is lower than the chosen threshold of 0.05, then the algorithm in the row of this cell performs significantly better than the one in the column. Otherwise, the results of the Wilcoxon test cannot find significant differences between the two approaches for the chosen confidence level.

The result of the Wilcoxon test confirms the intuition that could be gathered from the values reported in Figures 7.5 and 7.6. The standalone CMSA approach outperforms all others, followed by the CMSA-LNS algorithm. On this instance size, the Wilcoxon test is unable to make a significant distinction between the LNS+Solve and LNS approaches. This is consistent with the design of the LNS+Solve approach. Indeed, this approach runs the standard LNS and collects generated routes, solving the Set Covering ILP once every minute. For small instances, this means that the ILP is only solved once at the end of the algorithm, and the results in Table 7.11 have shown that the Solve step only has a limited impact for small instances. As such, the LNS+Solve is not significantly different from the standalone LNS on these instances. Finally, the two CG-based approaches perform worst, with the CMSA-CG⁺ having significantly better results than CMSA-CG. This could be deduced from Figures 7.5 and 7.6, as these two approaches have worse results than the others for instances of size $n = 20$.

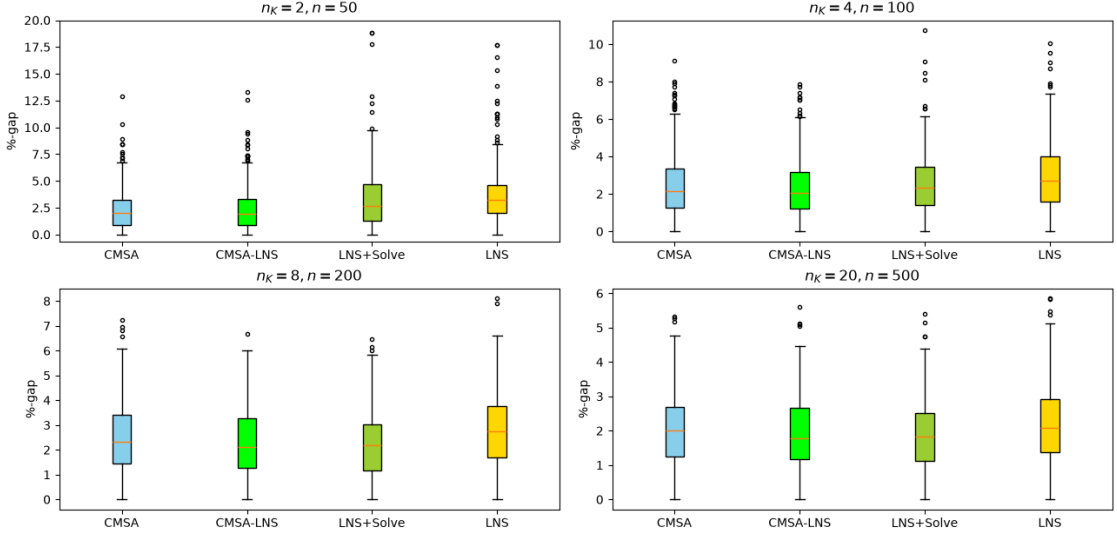


Figure 7.7: Relative objective gap on medium instances

Medium instances

Comparing the number of iterations reported in Table 7.15 for medium instances, we can observe that the CMSA and CMSA-LNS approaches perform significantly fewer iterations than the LNS algorithm. This highlights one of the main differences between the CMSA and LNS metaheuristics, namely that CMSA continuously constructs new solutions from scratch to obtain a diverse pool of components, while LNS only generates a completely new solution once to initialize the incumbent and then performs much smaller destroy-repair cycles. Even with this difference in iterations, we can observe in Figure 7.7 that CMSA still outperforms LNS on all medium instances. These results confirm the potential of CMSA, with the Solve step playing a significant role as discussed in Section 7.4.2.

Regarding the CMSA-LNS algorithm, we can note that for $n \geq 100$, the number of LNS iterations performed by this approach drops noticeably lower than that of the standalone LNS and of LNS+Solve. Still, the hybridization is successful, as CMSA-LNS performs best on instances of size $n = 100$ and is a close second on all other instance sizes. A deeper look into the role of LNS in generating routes in the outer CMSA framework is presented in Section 7.4.3. Comparing the number of improvements found by the Solve step for CMSA-LNS and LNS+Solve shows the importance of the Construct step of CMSA to generate a pool of diverse components. Indeed, LNS+Solve only finds a few improvements through the Solve step, while CMSA-LNS consistently benefits from it.

The overall results visible in Table 7.15 are consolidated by performing a Wilcoxon rank test for the medium instances, whose results are reported in Table 7.16. These tests confirm that CMSA-LNS significantly outperforms all other approaches for medium instances. Also, both CMSA and LNS+Solve perform significantly better than the

Instance	CMSA		CMSA-LNS		LNS+Solve		LNS	
(n_K, n)	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ
(2, 50)	2.356	1.957	2.484	2.199	3.665	3.843	3.833	3.237
(4, 100)	2.500	1.769	2.367	1.624	2.520	1.631	2.941	1.839
(8, 200)	2.516	1.488	2.282	1.330	2.243	1.322	2.805	1.496
(20, 500)	2.029	1.092	1.949	1.068	1.869	1.032	2.165	1.156
(n_K, n)	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}
(2, 50)	43	61	12	22 (177)	3	5 (241)	88	161
(4, 100)	30	34	13	16 (113)	4	5 (193)	108	141
(8, 200)	23	25	12	13 (84)	5	5 (169)	105	117
(20, 500)	21	22	11	11 (73)	4	4 (155)	102	107

Table 7.15: Obtained results on medium instances

p -value	CMSA	CMSA-LNS	LNS+Solve	LNS
CMSA	–	0.979	0.561	0.000
CMSA-LNS	0.021	–	0.042	0.000
LNS+Solve	0.439	0.958	–	0.000
LNS	1.000	1.000	1.000	–

Table 7.16: Pairwise Wilcoxon rank test for medium instances

standalone LNS, but the Wilcoxon test is unable to find substantial differences between these two approaches.

Let us now focus on the convergence of the different approaches with the higher runtime limit of 5 minutes. For instances with $n = 50$ requests, the different approaches all seem to converge within the given runtime. For CMSA-LNS, LNS+Solve and LNS, we have $I^{\text{best}} \approx 0.5 \cdot I^{\text{total}}$. Even for CMSA, the algorithm spends the last 20 iterations without finding an improvement, indicating that all considered approaches are likely to have converged. For the larger instances with $n \geq 100$ requests, all approaches are limited by the runtime of 5 minutes, as the difference between I^{best} and I^{total} remains small.

Finally, the results presented in Table 7.15 also show the variability of the approaches presented here. Indeed, the average %-gap of the best approach on all instances is around 2%, which indicates that although a given approach performs best, the best found solution on each instance is not reached consistently. This can be explained by the randomization of the different operators used in CMSA and LNS, which makes it so that two runs on the same instance can obtain vastly different results.

Large instances

On large instances, we can see that the CMSA approach only performs a few iterations on all instances, as shown in Table 7.17. The CMSA-LNS hybrid algorithm also suffers from the same weakness, even when considering the total number of internal LNS

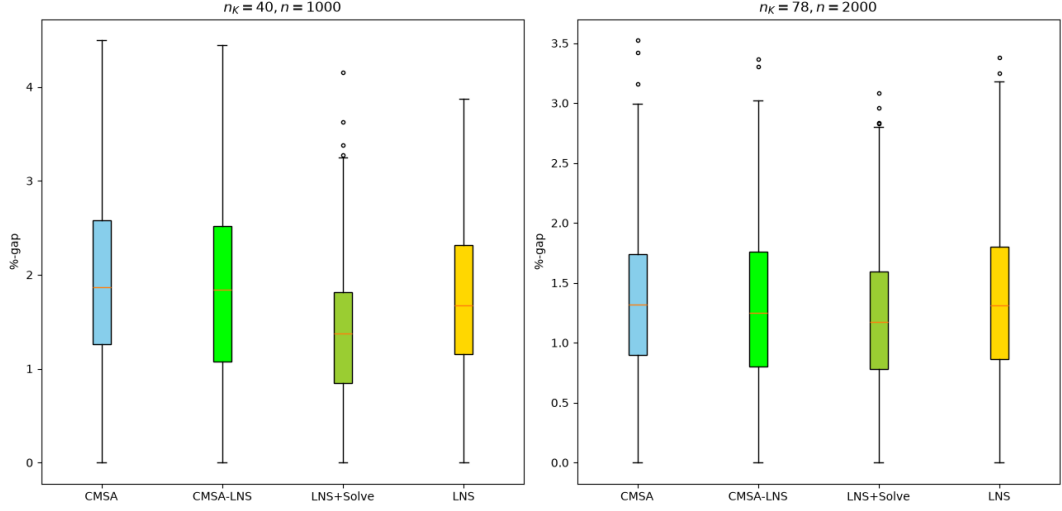


Figure 7.8: Relative objective gap on large instances

Instance	CMSA		CMSA-LNS		LNS+Solve		LNS	
(n_K, n)	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ
(40, 1000)	1.882	0.901	1.797	0.930	1.387	0.798	1.728	0.851
(78, 2000)	1.342	0.671	1.311	0.655	1.216	0.656	1.371	0.698
(n_K, n)	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}	I^{best}	I^{total}
(40, 1000)	9	9	8	8 (51)	8	8 (268)	187	192
(78, 2000)	6	6	6	6 (36)	5	5 (189)	136	140

Table 7.17: Obtained results on large instances

iterations. This is particularly significant when compared with the number of LNS iterations performed in the LNS+Solve and LNS approaches. Comparing the results of CMSA-LNS and LNS, we can observe that the CMSA-LNS approach remains competitive even with the significantly lower number of performed LNS iterations, indicating that the Construct and Solve steps from CMSA have a substantial impact on the quality of the final solution even within a few iterations.

Regarding the obtained results presented in Figure 7.8, we can observe that the LNS+Solve approach performs the best on this instance category. This is confirmed by the pairwise Wilcoxon tests reported in Table 7.18, where the LNS+Solve approach obtains significantly better results than all other approaches. A more specific investigation into the results of LNS+Solve is discussed in Section 7.4.3.

Regarding the other approaches, we can note that even with the limitation on the number of performed iterations, the CMSA and CMSA-LNS algorithms still perform relatively well in comparison to the LNS. This can be attributed in part to the speedup strategy enabled by setting $n_{\text{destroy}} = 100$, which still allows the approaches to perform a few iterations. Another contributing factor is the Solve step, which consistently finds improvements on

p -value	CMSA	CMSA-LNS	LNS+Solve	LNS
CMSA	–	0.903	1.000	0.896
CMSA-LNS	0.097	–	1.000	0.303
LNS+Solve	0.000	0.000	–	0.000
LNS	0.104	0.697	1.000	–

Table 7.18: Pairwise Wilcoxon rank test for large instances

these instances, as shown in Table 7.11. The Wilcoxon rank test presented in Table 7.18 is unable to clearly determine an order among the CMSA, CMSA-LNS, and LNS approaches at the standard threshold of $p \leq 0.05$, as the obtained results are quite similar between these approaches. Still, a finer look at the p -values obtained in the Wilcoxon test shows that at a confidence level of 90%, the CMSA approach performs worse than CMSA-LNS and LNS, as shown by the cells highlighted in yellow in Table 7.18. This indicates that for large instances, the faster destroy-repair cycles of the LNS-based approaches bring a small advantage over the Solve step of CMSA.

Similar to the results for medium instances, we can observe that all approaches are limited by the allotted runtime of 10 minutes. None of them have reached convergence, as $I^{\text{best}} \approx I^{\text{total}}$ for all of them in Table 7.17.

7.4.3 Side-by-side Comparison

In this section, we want to look into the contribution of the hybridization strategies when compared with the standalone algorithms. As such, each of the following paragraphs will focus on two approaches and investigate what advantages were brought by the hybridization. We will first compare standalone CMSA with CMSA-CG, then look into the differences between CMSA and CMSA-LNS, before investigating how LNS+Solve compares to the standalone LNS approach.

Integrating Column Generation into CMSA

In this section, we want to look deeper into the results of the CMSA-CG approach to investigate the advantages and limitations of the Column Generation technique applied to the EADARP. In the results discussed for small instances in Section 7.4.2, we have observed that the CMSA-CG algorithm successfully runs on instances with $n = 5$ requests, but encounters important scalability issues on instances with $n \geq 10$ requests. To better understand this approach, we have performed more extensive tests on randomly generated packed instances with $3 \leq n \leq 10$ requests and $n_K = 2$ vehicles. With these tests, we can more precisely measure the limitations of the CMSA-CG algorithm.

Let us start by looking more precisely at the internal operation of the CMSA-CG approach. In Section 5.4.2, we mentioned that the CMSA-CG algorithm does not perform exact Column Generation, instead running the labeling algorithm until a given runtime bound is reached. In the experiments performed in this chapter, this bound has always been set

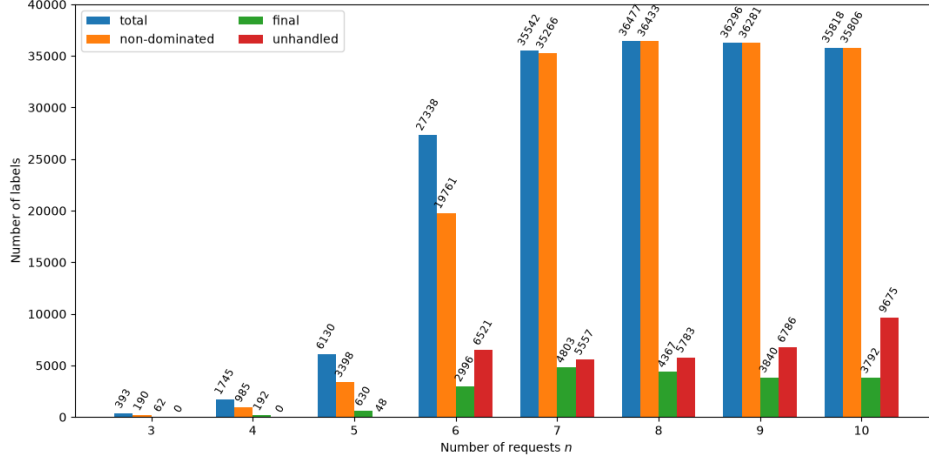


Figure 7.9: State of the explored labels at the end of the labeling algorithm

to 5 seconds per call to the labeling algorithm, to ensure that a reasonable number of CMSA iterations can be executed.

The line ‘Proportion of timeouts [%]’ in Table 7.19 reports the proportion of calls to the labeling algorithm that were interrupted by the runtime bound. From these results, we can observe that the approach only scales up to $n = 5$ requests, with the number of timeouts rapidly growing to reach 100% for $n \geq 8$. The results for $n \in \{6, 7\}$ also indicate that the runtime of the labeling algorithm depends partially on the considered instances, as there are some rare instances where the Pricing Subproblem can be solved exactly even for $n = 7$, while failing on other instances with $n = 6$ requests.

Instance size n	3	4	5	6	7	8	9	10
Proportion of timeouts [%]	0.00	0.00	0.04	50.50	99.64	100.00	100.00	100.00
Total number of routes with negative reduced cost	24	52	86	162	179	119	82	69

Table 7.19: Relevant metrics about the labeling algorithm of CMSA-CG

In Figure 7.9, we have represented, for each instance size, a snapshot of the state of the explored labels at the end of the labeling algorithm. Looking at the evolution of the total number of generated labels for $3 \leq n \leq 6$, represented by the blue bar in the diagram, we can observe that, when the labeling algorithm is able to terminate, the number of generated labels grows exponentially with the number of requests in the instance. This explains why the CMSA-CG approach performs so poorly on instances of size $n \in \{10, 20\}$, as the labeling algorithm is forced to stop early by the imposed limit on the runtime. The stagnation in the number of generated labels for $n \geq 7$ confirms this observation, with the number of extended labels being limited by the runtime and hardware limitations of our experiments.

We can also compare the number of non-dominated labels, represented by the orange bar in Figure 7.9, with the total number of generated labels. When the labeling algorithm can terminate naturally, we observe that the dominance criterion that was discussed in Section 5.3.4 is able to discard between 40% and 60% of the generated labels, considerably bringing down the number of labels to be extended. For the larger instances where the labeling algorithm has to be interrupted, the number of dominated labels is reduced to almost zero. This observation is explained by the fact that the heuristic labeling algorithm that was introduced in Section 5.4.2 focuses on the most promising labels first. As such, most of the considered labels remain non-dominated. To confirm this hypothesis, we have let the labeling algorithm run until termination on the test instances of size $n \in \{6, 7, 8\}$ and measured the time spent in the labeling algorithm, as well as the proportion of labels discarded thanks to the dominance criterion. These results, reported in Table 7.20, support the observation above, as the proportion of discarded labels remains higher than 50% on all instances when the labeling algorithm is allowed to terminate.

Instance size n	3	4	5	6	7	8
Runtime [s]	0.00	0.03	0.54	16.78	184.97	506.01
Total generated labels	393	1745	6130	75163	249630	463142
Total non-dominated labels	190	985	3398	41730	148396	288362
Proportion of discarded labels [%]	51.65	43.55	44.57	55.51	59.44	62.26

Table 7.20: Impact of the dominance criterion on the exact labeling algorithm

Aside from the non-dominated labels, the green bar shows the number of labels that are valid final labels. Recall that in our labeling algorithm, we generate a new label whenever we extend an existing label along a feasible edge. Among all extended labels, only a subset are labels representing a feasible route, namely those where the vehicle is empty. Comparing the number of valid final labels with the total number of labels shows that only a small proportion of extended labels correspond to feasible routes. The rest are labels at internal locations in a fragment where the vehicle is not empty, which is why they are not valid labels for feasible routes.

The red bar in Figure 7.9 shows the average number of labels that are still waiting to be extended when the algorithm is interrupted at the end of the 5-second runtime. For the smallest instances with $n \leq 5$ requests, this number is zero (or near zero for $n = 5$), confirming that the labeling algorithm is able to terminate naturally. For the larger instances, this number remains approximately constant at a value of around 6000 unhandled labels, because of the computation bounds set in our experiments. This number is not representative of the number of labels that would have been generated, had the labeling algorithm been allowed to terminate. Indeed, even though the number of unhandled labels is constant, the actual number of generated labels reported in Table 7.20, when the labeling algorithm is allowed to terminate, grows exponentially for larger instances.

Table 7.19 also reports the average number of routes with negative reduced cost that the CMSA-CG algorithm finds over the course of the program execution. For higher values of n , these results show that the Column Generation subroutine can still find some routes with negative reduced cost, even when the labeling algorithm is forced to stop early. The experimental results discussed in Section 7.4.2 show that these found routes do not suffice to make the CMSA-CG algorithm applicable to larger instances.

Finally, we want to look deeper into the performance of Column Generation and focus on the advantages brought by CG over the standalone CMSA approach. For this comparison, we will focus on the CMSA-CG⁺ approach, because results for this approach are more representative of the capabilities of a tailored Column Generation approach, consistently able to solve the Pricing Subproblem on small instances within the given runtime. As such, CMSA-CG⁺ is a good indicator of the advantages brought by integrating CG into CMSA.

Regarding the best found solution, CMSA-CG⁺ and CMSA find the same best solution on almost all instances. Still, it is remarkable that, among the 80 randomly generated test instances, there are two instances where CMSA ends up with a worse solution than CMSA-CG⁺. This highlights the fact that the construction heuristic used in CMSA remains a heuristic technique and that, even on small instances, there are cases where this technique is unable to reach the best solution.

Instance size n		3	4	5	6	7	8	9	10
CMSA	I^{best}	2	4	64	17	20	74	101	46
	Time [s]	0.00	0.01	0.30	0.11	0.18	1.01	2.73	1.50
	Solve Improv.	0.00	0.00	0.02	0.01	0.04	0.09	0.16	0.00
CMSA-CG ⁺	I^{best}	2	2	2	3	3	4	3	4
	Time [s]	0.00	0.01	0.01	0.03	0.08	0.43	0.26	0.64
	Solve Improv.	0.43	0.32	0.74	0.70	0.84	1.03	1.02	0.91

Table 7.21: Compared performance of CMSA-CG⁺ and CMSA

Aside from the obtained solutions, which remain quite similar for both approaches, we have also investigated how fast each of them converges towards the best found solution. In Table 7.21, we have reported the iteration at which the best solution was found and the associated time spent since the start of the experiment, averaged over all experiments performed in this section. In the line ‘Solve Improv.’, we have also reported the frequency at which the Solve step finds improvements, which was previously discussed in Section 7.4.2. As expected, the CMSA-CG⁺ approach benefits largely from the Solve step, showing that the routes found by Column Generation actually improve the incumbent. Comparatively, the routes generated by the Construct step play a more limited role, as shown by the Solve improvements for CMSA.

For the smallest instances with $n \in \{3, 4\}$, the difference in convergence speed is not substantial, but as the size of the instances grows, we can clearly observe that CMSA-CG⁺ converges significantly faster towards the best solution, both in terms of iterations and

in terms of runtime. These results show the high potential of CG to find improving routes most fitting with the current state of the CMSA pool. Combining this guided route generation with the Solve step of CMSA leads to a much faster convergence of the algorithm.

Combining CMSA with LNS

The goal of integrating LNS into the CMSA framework is two-fold. First, the main focus of this hybridization strategy lies in the use of the LNS to quickly explore many solutions and collect all encountered routes to populate the CMSA pool and give more possibilities to the Solve step. Secondly, using the LNS also enables a more focused search to improve the incumbent, similar to local search operations performed in many search-based metaheuristics. We want to look more deeply at the results of the CMSA-LNS approach compared with the standalone CMSA to more accurately observe the impact that both of these factors have on the obtained results.

Looking at the number of iterations performed by both approaches reported in Tables 7.12, 7.13, 7.15 and 7.17, we can observe that performing the internal call to the LNS lowers the total number of iterations that can be performed by the CMSA-LNS algorithm. As such, this also lowers the randomization of the pool brought by the shaking step, as this step is called less often. On the other hand, this allows the LNS to run and try to improve the incumbent. Running the internal LNS has a very limited impact on smaller instances with $n \leq 50$ requests, as CMSA has a lower average %-gap than CMSA-LNS on these instances. Still, the hybridization becomes more advantageous on larger instances. Indeed, for instances with size $n \geq 100$, the CMSA-LNS algorithm outperforms CMSA. As such, using the LNS to improve the incumbent in the CMSA framework is a successful part of the hybridization.

Instance (n_K, n)	Small			Medium				Large	
	(2, 5)	(2, 10)	(2, 20)	(2, 50)	(4, 100)	(8, 200)	(20, 500)	(40, 1000)	(78, 2000)
Final pool size – CMSA	44	733	1294	1246	725	576	723	1623	2346
Final pool size – CMSA-LNS	29	140	500	678	487	449	657	1579	2310
Total routes generated by LNS ⁵	37758	7743	1207	315	175	144	162	129	98
Total LNS-routes added in the pool	26	127	248	214	110	60	43	27	19

Table 7.22: Comparing routes added in CMSA and in CMSA-LNS

Regarding the other objective of this CMSA-LNS hybridization, the situation is more nuanced. Exploring many solutions quickly using the LNS to populate the CMSA pool does not bring the expected results. In Table 7.22, we have investigated the contribution of the LNS to the pool of the CMSA-LNS algorithm. The first observation that we can make, is that the final size of the pool in the CMSA-LNS algorithm is actually noticeably lower than for standalone CMSA, although the hybrid approach is able to explore more solutions by performing fast LNS iterations. This can be explained by the fact that the LNS sometimes generates solutions which have already been seen and added to the pool

⁵Includes duplicate routes, where the LNS generates the same route in different iterations

previously. This is confirmed by looking at the difference between the total number of routes generated by the LNS and the number of routes that are actually added to the pool.

Compared to the LNS, which generates relatively similar routes by removing a small number of requests, the Construct step of CMSA is able to generate more diverse routes, making the overall size of the pool higher for CMSA than for CMSA-LNS. The loss in the diversity of routes in the pool is also observable in the lower number of improvements found by the Solve step in CMSA-LNS compared to CMSA, as reported in Table 7.11. As such, although the LNS is able to explore more solutions, this does not directly translate to more diversity in the pool, because the solutions generated by the LNS are more similar to one another, compared to those generated in the Construct step of CMSA.

Overall, the hybridization between CMSA and LNS makes the most significant difference on medium instances. There, the ability of the LNS to improve the incumbent contributes more to finding high-quality solutions than the Solve step of CMSA. But comparing the CMSA and LNS algorithms on this same instance size also highlights that solving the Set Covering ILP still matters, as CMSA significantly outperforms the standalone LNS on all medium instances. For large instances, the results are more nuanced, but looking at the Wilcoxon rank test in Table 7.18, we can observe that CMSA-LNS outperforms CMSA for a confidence threshold of 90%, as the associated p -value is 0.097. This shows that, while less considerable, the hybridization between CMSA and LNS is still profitable on this instance size.

Adding a Solve step to the LNS

In the LNS+Solve approach, we investigated the potential of using the Set Covering ILP to improve the solution found by the LNS, without using the Construct step of CMSA to focus on generating diverse pool components. By looking more closely at the differences between the LNS and LNS+Solve approaches, we can confirm whether this additional Solve step brings significant improvements to the LNS.

On small instances, the results in Section 7.4.2 have shown that the impact of solving the Set Covering ILP for LNS+Solve is limited. This result is not specific to LNS+Solve and is shared by the other approaches, as both CMSA and CMSA-LNS also find very few improvements. The Solve step is only significant for the CG-based algorithms. As such, the results obtained for LNS and LNS+Solve remain very similar, with the Wilcoxon test in Table 7.14 unable to distinguish between the two approaches.

On medium and large instances, LNS+Solve performs significantly better than the standalone LNS. However, a deeper look into the gathered results indicates that this performance cannot be directly attributed to the Solve step that was added in the LNS+Solve approach. Indeed, Table 7.11 shows that only a few experiments actually find improvements thanks to the Solve step. Furthermore, the number of LNS iterations performed by the LNS+Solve approach in Table 7.17 is substantially higher than that of the LNS, which could more directly lead to the improved results of LNS+Solve.

A deeper investigation into these results points out that the additional overhead in the implementation of the LNS+Solve allowed it to have a longer runtime compared to the standalone LNS approach, as reported in Table 7.23. Indeed, in the LNS+Solve approach, the solution initialization is performed independently from the following calls to the LNS subroutine. Because the initialization can take longer than a minute for medium to large instances, the LNS+Solve approach actually spends more time in the LNS subroutine than the LNS.

Instance (n_K, n)	Small			Medium				Large	
	(2, 5)	(2, 10)	(2, 20)	(2, 50)	(4, 100)	(8, 200)	(20, 500)	(40, 1000)	(78, 2000)
LNS	60	60	60	301	301	302	301	601	601
LNS+Solve	60	60	60	309	316	329	324	631	635

Table 7.23: Average runtime of the LNS and LNS+Solve approaches

To verify whether this difference was responsible for the observed results, we have investigated the results of the benchmarks and computed the results obtained by LNS+Solve when removing the last CMSA iteration, which corresponds to approximately 60 seconds of equivalent LNS runtime. The obtained results are reported in Table 7.24. In that case, LNS+Solve actually has a lower average runtime than LNS. Performing new Wilcoxon tests on the updated values, the conclusions from the previous sections still remain valid, with the LNS+Solve approach significantly outperforming LNS on both medium and large instances, as shown in Table 7.25.

Instance (n_K, n)		Medium				Large	
		(2, 50)	(4, 100)	(8, 200)	(20, 500)	(40, 1000)	(78, 2000)
LNS+Solve	Runtime [s]	253	255	268	275	576	576
	I^{best}	3	4	4	3	7	4
	I^{total}	4	4	4	3	7	4
	LNS iterations	197	152	135	127	239	158
LNS	I^{total}	161	141	117	107	192	140

Table 7.24: Updated results for the aborted LNS+Solve approach

Medium	LNS+Solve	LNS	Large	LNS+Solve	LNS
LNS+Solve	–	0.001	LNS+Solve	–	0.072
LNS	0.999	–	LNS	0.928	–

Table 7.25: Updated Wilcoxon tests between LNS+Solve and LNS

Overall, even though the differences between LNS and LNS+Solve cannot be directly attributed to the Solve step of CMSA, the LNS+Solve approach still consistently outperforms the standalone LNS. These results indicate that a finer interaction between CMSA and LNS in the LNS+Solve algorithm is responsible for the improved results. More extensive testing is needed to investigate this point, which falls outside the scope of this thesis.

Conclusion & Summary

This section will conclude this thesis by summarizing the main contributions and answering the Research Questions that were introduced in Section 1.1.

RQ1: What adaptations can be made to the general CMSA framework to deliver improved solutions for the EADARP?

This question was answered by developing and implementing a CMSA framework for the EADARP in Chapter 4. The proposed approach relies on a request insertion procedure to generate new solutions, from which feasible routes can be extracted and collected. The Solve step uses the Set Covering ILP formulation of the EADARP to extract the best possible solution from the set of known routes. Aside from the direct application of the CMSA metaheuristic to the EADARP, this thesis also investigated strategies to mitigate the limitations of the considered construction heuristic, introducing parametrized mechanisms that control the trade-off between faster solution generation and quality of the obtained solution.

The obtained experimental results show that CMSA can be successfully applied to complex optimization problems such as the EADARP, obtaining good results across the board, outperforming the LNS on all small and medium benchmark instances and remaining competitive on large instances. On small instances, the diversity of solutions generated by the Construct step allows CMSA to find solutions unreachable for the LNS. On medium to large instances, the additional step of solving the Set Covering ILP continuously improves the incumbent solution and unlocks new solutions that would have remained unexplored by search-based heuristics. Even as the construction heuristic becomes more expensive on large instances, thus reducing the number of iterations performed by CMSA within the allotted runtime, the mitigation strategies allow CMSA to find high-quality solutions on these instances, remaining competitive compared to the other approaches.

RQ2-3: How can CMSA make use of exact Column Generation to develop a guided search strategy to solve the EADARP? Which algorithms can be used to efficiently solve the Pricing Subproblem obtained when applying Column Generation to the EADARP?

Inspired by the results obtained by Akbay et al. [3], this thesis looked into a hybridization strategy to combine CMSA with Column Generation. This hybridization relies on the similarities between the Solve step of CMSA and the Restricted Master Problem of CG. Based on the pool of known solutions, the RMP and Pricing Subproblem can propose new promising routes to improve the incumbent solution.

Applying this principle to the EADARP introduced the formulation of the Pricing Subproblem as an Elementary Shortest Path Problem with Resource Constraints as shown in Section 5.2.2. Following standard approaches from the literature, we developed a labeling algorithm to solve this problem and find feasible routes with negative reduced cost. A crucial point in the formulation of the Pricing Subproblem for the EADARP lies in the consideration of waiting times and lateness, which are specific to this problem formulation.

The exact solving of the Pricing Subproblem proved more complex than initially expected, with the developed labeling algorithm generating an amount of labels that is exponential in the size of the instance. As such, the developed CMSA-CG hybrid framework is only applicable to small instances. Still, experimental results have shown that approaches relying on Column Generation are able to discover new best-found solutions on small instances when compared to both the CMSA and LNS approaches. We also observed that combining CMSA with Column Generation allowed the experiments to converge significantly faster towards the best found solution. This proves the potential of combining heuristic and exact methods to guide the search towards promising solutions.

RQ4: How can CMSA be combined with the LNS developed by Tomandl et al. [45] to improve its performance?

To overcome the limitations encountered on the standalone CMSA approach, we developed a hybrid heuristic by combining CMSA with the LNS approach of Tomandl et al. [45]. This approach, presented in Chapter 6, uses the ability of the LNS to quickly explore many different solutions to collect more routes, instead of exclusively relying on the slower construction heuristic of CMSA. This hybridization allows both CMSA and LNS to focus on their strengths, while covering each other's weaknesses. The LNS acts as an intensification operator, focusing on improving the current incumbent through search-based mechanisms, while the Solve step of CMSA can regularly improve the incumbent and makes the LNS avoid stagnating around local optima by restarting from a new incumbent. Experimental results show that this hybrid approach outperforms both CMSA and LNS on medium and large instances.

Future Work

This thesis focused on developing different CMSA-based approaches for the EADARP, investigating the applicability of this metaheuristic and its limitations. In the end, there remain many points of interest that did not enter the scope of this thesis, but which raise interesting questions.

The CMSA framework investigated in this thesis relies on using feasible routes as components for the CMSA pool. At the same time, the discussions in this thesis have shown the central role of fragments, and their versatility in the representation and evaluation of routes. As such, it seems promising to focus future work on methods that can benefit more from this formalism.

The experimental results discussed in Chapter 7 have shown that the main limitation of the CMSA heuristics applied to the EADARP lies in the chosen procedure to generate new feasible solutions, thus limiting the number of iterations performed by this heuristic within the given runtime. Future research investigating CMSA for this problem should look into alternative construction heuristics that are able to generate many diverse feasible components, especially for larger instances.

Regarding the integration of Column Generation into CMSA, the obtained results indicate that, although this method shows potential on small instances, exact Column Generation is too expensive to perform on larger instances. This brings the idea of focusing more on heuristic techniques for the labeling algorithm, concentrating on efficiently finding a promising route with negative reduced cost to be able to apply this technique on larger instances.

Finally, the approaches developed in this thesis all focused on the static instances of the DynEADARP, but the procedure described in Section 3.3 provides a way to directly apply these algorithms to solve dynamic instances. Future work should delve deeper into the dynamic aspect of the problem, looking into strategies to apply CMSA for dynamic instances. Here, the problem of finding a compromise between short- and long-term decisions becomes crucial to account for the insertion of future requests while maintaining a good solution quality. A promising idea for the application of CMSA-based methods to dynamic instances consists of using the components obtained by solving the static subproblem at the current time step to provide a starting base for the optimization of the next state, taking into account the newly incoming requests and the decisions that have been fixed in the current state.

Overview of Generative AI Tools Used

In this thesis, AI tools have only been used as spell-checking tools, to suggest minor improvements to some formulations and to provide some help with translations between german and english.

The integrated spell checker of Libre Office Writer¹ has been used for orthograph corrections. Grammarly² has been used as a proofreading tool to improve some grammatical formulations. DeepL³ and Pons⁴ have been used to translate the sections “Abstract”, “Acknowledgements” and “Overview of Generative AI Tools Used”.

¹<https://www.libreoffice.org/>

²<https://www.grammarly.com/>

³<https://www.deepl.com/en>

⁴<https://en.pons.com/translate>

Übersicht verwendeter Hilfsmittel

KI-basierte Hilfsmittel wurden in dieser Arbeit nur zur Rechtschreibprüfung, zur Verbesserung einzelner Ausdrücke und als Hilfsmittel zur Übersetzung zwischen Deutsch und Englisch verwendet.

Für die Korrektur von Rechtschreibfehlern wurde die integrierte Rechtschreibprüfung von LibreOffice Writer⁵ verwendet. Grammarly⁶ wurde für das Korrekturlesen verwendet, um grammatikalische Formulierungen zu verbessern. DeepL⁷ und Pons⁸ wurden verwendet, um die Abschnitte “Kurzfassung”, “Danksagung” und “Übersicht verwendeter Hilfsmittel” zu übersetzen.

⁵<https://www.libreoffice.org/>

⁶<https://www.grammarly.com/>

⁷<https://www.deepl.com/en>

⁸<https://en.pons.com/translate>

List of Figures

3.1	Example of an EADARP instance	14
4.1	The different steps of a CMSA iteration	25
7.1	Influence of the $k_{\max}$ parameter on the construction heuristic	70
7.2	Influence of the n_{destroy} parameter on the construction heuristic	71
7.3	Obtained results for instances of size $n = 5000$	72
7.4	Evolution of the objective value over time	73
7.5	Relative objective gap on small packed instances	75
7.6	Relative objective gap on small uniform instances	76
7.7	Relative objective gap on medium instances	78
7.8	Relative objective gap on large instances	80
7.9	State of the explored labels at the end of the labeling algorithm	82

List of Tables

3.1	Summary of the notations used for the EADARP	13
3.2	Example of the dynamic framework	22
7.1	Instance sizes	62
7.2	Configurable parameters considered for tuning	63
7.3	Restricted Domain for the standalone CMSA parameters	64
7.4	Restricted Domain for the CMSA-LNS parameters	64
7.5	Tuning results for the standalone CMSA parameters	65
7.6	Tuning results for the CMSA-LNS parameters	65
7.7	Overview of realized experiments	66
7.8	Total number of calls to the post-processing step of the Set Covering ILP	68
7.9	Proportional runtime of each CMSA step (in %)	69
7.10	Statistics of OTFCSI for different values of $k_{\max}$	70
7.11	Average number of improvements found by the Solve step per run	73
7.12	Obtained results on small packed instances	75
7.13	Obtained results on small uniform instances	77
7.14	Pairwise Wilcoxon rank test for small instances	77
7.15	Obtained results on medium instances	79
7.16	Pairwise Wilcoxon rank test for medium instances	79
7.17	Obtained results on large instances	80
7.18	Pairwise Wilcoxon rank test for large instances	81
7.19	Relevant metrics about the labeling algorithm of CMSA-CG	82
7.20	Impact of the dominance criterion on the exact labeling algorithm	83
7.21	Compared performance of CMSA-CG ⁺ and CMSA	84
7.22	Comparing routes added in CMSA and in CMSA-LNS	85
7.23	Average runtime of the LNS and LNS+Solve approaches	87
7.24	Updated results for the aborted LNS+Solve approach	87
7.25	Updated Wilcoxon tests between LNS+Solve and LNS	87

List of Algorithms

3.1	Post-Processing step for the Set Covering ILP	18
4.1	CMSA Framework for the EADARP	26
4.2	Request insertion procedure for random and chronological order	27
4.3	Greedy request insertion procedure	27
4.4	Limited request insertion procedure	30
4.5	Construction heuristic with partial destroy	31
5.1	Generate all non-dominated labels	48
5.2	Extract the route with minimum reduced cost	49
5.3	Construct a route based on the associated final label	50
5.4	Solve step for the hybrid CMSA-CG algorithm	52
6.1	Large Neighborhood Search for the EADARP	56
6.2	CMSA-LNS hybrid framework for the EADARP	57

Acronyms

CG Column Generation.

CMSA Construct, Merge, Solve & Adapt.

DARP Dial-a-Ride Problem.

DynEADARP Dynamic Electric Autonomous Dial-a-Ride Problem.

EADARP Electric Autonomous Dial-a-Ride Problem.

ESPPRC Elementary Shortest Path Problem with Resource Constraints.

ILP Integer Linear Program.

LNS Large Neighborhood Search.

LP Linear Program.

MILP Mixed Integer Linear Program.

REF Resource Extension Function.

RMP Restricted Master Problem.

VRP Vehicle Routing Problem.

Bibliography

- [1] Christian Ackermann and Julia Rieck. New optimization guidance for dynamic dial-a-ride problems. In Norbert Trautmann and Mario Gnägi, editors, *Operations Research Proceedings 2021*, pages 283–288, Cham, 2022. Springer International Publishing.
- [2] Mehmet Anıl Akbay, Christian Blum, and Can Berk Kalayci. Application of adapt-cmsa to the electric vehicle routing problem with simultaneous pickup and deliveries. In Alexis Quesada-Arencibia, Michael Affenzeller, and Roberto Moreno-Díaz, editors, *Computer Aided Systems Theory – EUROCAST 2024*, volume 15172 of *Lecture Notes in Computer Science*, pages 90–106, Cham, 2025. Springer Nature Switzerland.
- [3] Mehmet Anıl Akbay, Christian Blum, and Günther R Raidl. A hybrid CMSA–Column generation approach to variable-sized bin packing. In *Frontiers in Artificial Intelligence and Applications*, volume 413 of *Frontiers in Artificial Intelligence and Applications*, pages 4790–4797. IOS Press, 2025.
- [4] Elahe Amiri, Antoine Legrain, and Issmail El Hallaoui. Accelerated column generation: Application in real-time dial-a-ride problem. *Les Cahiers du GERAD ISSN*, 711:1–23, 2024.
- [5] Andrea Attanasio, Jean-François Cordeau, Gianpaolo Ghiani, and Gilbert Laporte. Parallel tabu search heuristics for the dynamic multi-vehicle dial-a-ride problem. *Parallel Computing*, 30(3):377–387, 2004.
- [6] Gerardo Berbeglia, Jean-François Cordeau, and Gilbert Laporte. Dynamic pickup and delivery problems. *European Journal of Operational Research*, 202(1):8–15, 2010.
- [7] Christian Blum, Pedro Pinacho, Manuel López-Ibáñez, and José A. Lozano. Construct, merge, solve & adapt: A new general algorithm for combinatorial optimization. *Computers & Operations Research*, 68:75–88, 2016.
- [8] Claudia Bongiovanni, Mor Kaspi, Jean-François Cordeau, and Nikolas Geroliminis. A machine learning-driven two-phase metaheuristic for autonomous ridesharing operations. *Transportation Research Part E*, 165:102835, 2022.

- [9] Claudia Bongiovanni, Mor Kaspi, and Nikolas Geroliminis. A two phase heuristic approach for the dynamic electric autonomous dial-a-ride problem. In *7th Symposium of the European Association for Research in Transportation*, 2018.
- [10] Claudia Bongiovanni, Mor Kaspi, and Nikolas Geroliminis. The electric autonomous dial-a-ride problem. *Transportation Research Part B: Methodological*, 122:436–456, 2019.
- [11] Maria Bresich, Günther R. Raidl, Caspian Coleman, Pascal Welke, and Steffen Limmer. Search space reduction through machine learning for the electric autonomous dial-a-ride problem. In Giuseppe Nicosia, Varun Ojha, Sven Giesselbach, M. Panos Pardalos, Renato Umeton, La Malfa Emanuele, and La Malfa Gabriele, editors, *Machine Learning, Optimization, and Data Science*, volume 16468 of *Lecture Notes in Computer Science*, pages 185–200, Cham, 2026. Springer Nature Switzerland.
- [12] Maria Bresich, Günther R. Raidl, and Steffen Limmer. Letting a large neighborhood search for an electric dial-a-ride problem fly: On-the-fly charging station insertion. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '24*, page 142–150, New York, NY, USA, 2024. Association for Computing Machinery.
- [13] Jean-François Cordeau and Gilbert Laporte. The dial-a-ride problem: models and algorithms. *Annals of Operations Research*, 153(1):29–46, 2007.
- [14] Jean-François Cordeau. A branch-and-cut algorithm for the dial-a-ride problem. *Operations Research*, 54(3):573–586, 2006.
- [15] Jean-François Cordeau and Gilbert Laporte. A tabu search heuristic for the static multi-vehicle dial-a-ride problem. *Transportation Research Part B: Methodological*, 37(6):579–594, 2003.
- [16] Luca Coslovich, Raffaele Pesenti, and Walter Ukovich. A two-phase insertion technique of unexpected customers for a dynamic dial-a-ride problem. *European Journal of Operational Research*, 175(3):1605–1615, 2006.
- [17] Guy Desaulniers, Fausto Errico, Stefan Irnich, and Michael Schneider. Exact algorithms for electric vehicle-routing problems with time windows. *Operations Research*, 64(6):1388–1405, 2016.
- [18] Thierry Garaix, Christian Artigues, Dominique Feillet, and Didier Josselin. Optimization of occupancy rate in dial-a-ride problems via linear fractional column generation. *Computers & Operations Research*, 38(10):1435–1442, 2011.
- [19] Timo Gschwind and Michael Drexler. Adaptive large neighborhood search with a constant-time feasibility test for the dial-a-ride problem. *Transportation Science*, 53(2):480–491, 2019.
- [20] Michaela Heinzl. A labeling algorithm for the electric autonomous dial-a-ride problem. Bachelor’s thesis, TU Wien, Faculty of Informatics, 2026.

- [21] Sin C. Ho, W.Y. Szeto, Yong-Hong Kuo, Janny M.Y. Leung, Matthew Petering, and Terence W.H. Tou. A survey of dial-a-ride problems: Literature review and recent developments. *Transportation Research Part B*, 111(C):395–421, 2018.
- [22] William Huang, Yi Mei, Gunther Raidl, Fangfang Zhang, Laurenz Tomandl, Steffen Limmer, Mengjie Zhang, and Tobias Rodemann. Genetic programming hyper-heuristic for the dynamic electric dial-a-ride problem. In *2025 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8, 2025.
- [23] Stefan Irnich and Guy Desaulniers. *Shortest Path Problems with Resource Constraints*, pages 33–65. Springer US, Boston, MA, 2005.
- [24] Sten Elling Tingstad Jacobsen, Attila Lischka, Balázs Kulcsár, and Anders Lindman. Learning to dial-a-ride: A deep graph reinforcement learning approach to the electric dial-a-ride problem. *arXiv*, 2026.
- [25] R M Jorgensen, J Larsen, and K B Bergvinsdottir. Solving the dial-a-ride problem using genetic algorithms. *Journal of the Operational Research Society*, 58(10):1321–1331, 2007.
- [26] Nicholas D. Kullman, Martin Cousineau, Justin C. Goodson, and Jorge E. Mendoza. Dynamic ride-hailing with electric vehicles. *Transportation Science*, 56(3):775–794, 2022.
- [27] Steffen Limmer. Bilevel large neighborhood search for the electric autonomous dial-a-ride problem. *Transportation Research Interdisciplinary Perspectives*, 21:100876, 2023.
- [28] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Thomas Stützle, and Mauro Birattari. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.
- [29] Yves Molenbruch, Kris Braekers, and An Caris. Typology and literature review for dial-a-ride problems. *Annals of Operations Research*, 259(1):295–325, 2017.
- [30] Yves Molenbruch, Kris Braekers, An Caris, and Greet Vanden Berghe. Multi-directional local search for a bi-objective dial-a-ride problem in patient transportation. *Computers & Operations Research*, 77:58–71, 2017.
- [31] Sophie Parragh, Jean-François Cordeau, Karl Doerner, and Richard Hartl. Models and algorithms for the heterogeneous dial-a-ride problem with driver-related constraints. *OR Spectrum*, 34:1–41, 2009.
- [32] Sophie N. Parragh, Karl F. Doerner, and Richard F. Hartl. Variable neighborhood search for the dial-a-ride problem. *Comput. Oper. Res.*, 37(6):1129–1138, 2010.

- [33] Sophie N. Parragh, Karl F. Doerner, Richard F. Hartl, and Xavier Gandibleux. A heuristic two-phase solution approach for the multi-objective dial-a-ride problem. *Networks*, 54(4):227–242, 2009.
- [34] Harilaos N. Psaraftis, Min Wen, and Christos A. Kontovas. Dynamic vehicle routing problems: Three decades and counting. *Networks*, 67(1):3–31, 2016.
- [35] Yannik Rist and Michael A. Forbes. A new formulation for the dial-a-ride problem. *Transportation Science*, 55(5):1113–1135, 2021.
- [36] Ulrike Ritzinger, Jakob Puchinger, and Richard Hartl. Dynamic programming based metaheuristics for the dial-a-ride problem. *Annals of Operations Research*, 236:341–358, 2016.
- [37] Stefan Ropke and Jean-François Cordeau. Branch and cut and price for the pickup and delivery problem with time windows. *Transportation Science*, 43:267–286, 2009.
- [38] Stefan Ropke, Jean-François Cordeau, and Gilbert Laporte. Models and branch-and-cut algorithms for pickup and delivery problems with time windows. *Networks*, 49(4):258–272, 2007.
- [39] Hamid R. Sayarshad and H. Gao. A scalable non-myopic dynamic dial-a-ride and pricing problem for competitive on-demand mobility systems. *Transportation Research Part C*, 91:192–208, 2018.
- [40] Verena Stallhofer and Sophie N. Parragh. Event-based models for the electric autonomous dial-a-ride problem. *Transportation Research Part C: Emerging Technologies*, 171:104896, 2025.
- [41] Yue Su, Nicolas Dupin, Sophie N. Parragh, and Jakob Puchinger. A branch-and-price algorithm for the electric autonomous dial-a-ride problem. *Transportation Research Part B: Methodological*, 186:103011, 2024.
- [42] Yue Su, Nicolas Dupin, and Jakob Puchinger. A deterministic annealing local search for the electric autonomous dial-a-ride problem. *European Journal of Operational Research*, 309(3):1091–1111, 2023.
- [43] Amirmahdi Tafreshian, Mojtaba Abdolmaleki, Neda Masoud, and Huizhu Wang. Proactive shuttle dispatching in large-scale dynamic dial-a-ride systems. *Transportation Research Part B*, 150:227–259, 2021.
- [44] Dhananjay Thiruvady and Rhyd Lewis. Recombinative approaches for the maximum happy vertices problem. *Swarm and Evolutionary Computation*, 75:101188, 2022.
- [45] Laurenz Tomandl, Maria Bresich, Günther R. Raidl, Yi Mei, Steffen Limmer, and Tobias Rodemann. A reinforcement learning guided large neighborhood search for the dynamic electric autonomous dial-a-ride problem. In Giuseppe Nicosia, Varun

- Ojha, Sven Giesselbach, M. Panos Pardalos, Renato Umeton, La Malfa Emanuele, and La Malfa Gabriele, editors, *Machine Learning, Optimization, and Data Science*, volume 16468 of *Lecture Notes in Computer Science*, pages 425–439, Cham, 2026. Springer Nature Switzerland.
- [46] Laurence A Wolsey. *Integer programming*. Wiley, Hoboken, NJ Chichester, West Sussex, second edition edition, 2021.
- [47] Jiang Zhou and Peng Zhang. Simple heuristics for the rooted max tree coverage problem. In Weili Wu and Jianxiong Guo, editors, *Combinatorial Optimization and Applications*, pages 252–264, Cham, 2024. Springer Nature Switzerland.